

Paramiko Security Audit



Quarkslab

Reference 25-11-2415-REP

Version 1.1

Date 2026-04-20

Quarkslab SAS
10 boulevard Haussmann
75009 Paris
France

1. Project information

1.1. Document history

Version	Date	Details	Authors
1.0	2025-11-10	Initial audit	Dahmun Goudarzi Julio Loayza Meneses Alan Marrec Pauline Sauder
1.1	2026-04-20	Control audit	Dahmun Goudarzi Julio Loayza Meneses Alan Marrec Pauline Sauder

1.2. Contacts

1.2.1. Quarkslab

Name	Role	Email
Frédéric Raynal	CEO	fraynal@quarkslab.com
Pauline Sauder	Project Manager	psauder@quarkslab.com
Dahmun Goudarzi	R&D Engineer	dgoudarzi@quarkslab.com
Julio Loayza Meneses	R&D Engineer	jloayzameneses@quarkslab.com
Alan Marrec	R&D Engineer	amarrec@quarkslab.com

1.2.2. Paramiko

Name	Role	Email
Jeff Forcier	Maintainer	jeff@bitprophet.org

1.2.3. OSTIF

Name	Role	Email
Derek Zimmer	Executive Director	derek@ostif.org
Amir Montazery	Managing Director	amir@ostif.org
Helen Woeste	Communication Manager	helen@ostif.org

Contents

1. Project information	1
1.1. Document history	1
1.2. Contacts	1
2. About OSTIF	4
3. Executive Summary	5
3.1. Context	5
3.2. Objectives	5
3.3. Findings Summary	6
3.4. Recommendation and Action Plan	8
3.5. Conclusions	10
4. Control audit	11
5. Reading Guide	12
5.1. Executive summary	12
5.2. Metric definition	12
6. Discovery	15
6.1. Projects Information	15
6.2. Installation	19
6.3. Source Discovery	20
6.4. Tests	23
6.5. PYCA Cryptography	30
6.6. Fuzzing	31
7. Audit Overview	33
7.1. Hypotheses	33
7.2. Threats and Security checks	33
7.3. Methodology	34
8. Paramiko's compliance to SSH RFCs	35
8.1. Compliance and Security considerations	35
8.2. Summary of RFC compliance	68
9. Static analysis	69
9.1. Cryptographic primitives used in Paramiko	69
9.2. Random number generation	91
9.3. Cryptographic primitive implementations in (PYCA) Cryptography	98
9.4. Conclusion	113
10. Testing enhancement	116

10.1. Increasing test coverage	116
10.2. CI improvements	117
10.3. Conclusion	124
11. Dynamic testing	125
11.1. Invalid Ed25519 signature size	125
11.2. DES keys	126
11.3. RSA keys	126
11.4. Constant-timeness	128
11.5. Differential Fuzzing of PYCA Cryptography	132
11.6. Server over UDP	132
11.7. Conclusion	133
12. Build Systems, CI, and Supply chain Review	134
12.1. Paramiko's CI	134
12.2. Fork to Pull Requests	134
12.3. Supply Chain Attacks	137
12.4. Conclusion	137
A. Dependencies script	139
B. Patch to fix GSS-API tests	145
C. Dynamic testing scripts	149
C.1. Base instances	149
C.2. Bad Ed25519 signature	155
C.3. RSA keys	155
C.4. RSA 512-bit server	156
C.5. RSA 1024-bit server	158
C.6. TDES keys	158
C.7. UDP server	159
D. Pysa configuration files	162
D.1. <code>.pyre_configuration</code>	162
D.2. <code>sinks.pysa</code>	162
D.3. <code>taint.config</code>	163

2. About OSTIF

The **Open Source Technology Improvement Fund (OSTIF)** is dedicated to resourcing and managing security engagements for open source software projects through partnerships with corporate, government, and non-profit donors. We bridge the gap between resources and security outcomes, while supporting and championing the open source community whose efforts underpin our digital landscape.

Over the past ten years, OSTIF has been responsible for the discovery of over 800 vulnerabilities, (121 of those being Critical/High), over 13,000 hours of security work, and millions of dollars raised for open source security. Maximizing output and security outcomes while minimizing labor and cost for projects and funders has resulted in partnerships with multi-billion dollar companies, top open source foundations, government organizations, and respected individuals in the space. Most importantly, we've helped over 120 projects and counting improve their security posture.

Our directive is to support and enrich the open source community through providing public-facing security audits, educational resources, meetups, tooling, and advice. OSTIF's experience positions us to be able to share knowledge of auditing with maintainers, developers, foundations, and the community to further secure our infrastructure in a sustainable manner.

We are a small team working out of Chicago, Illinois. Our website is ostif.org. You can follow us on social media to keep up to date on audits, conferences, meetups, and opportunities with OSTIF, or feel free to reach out directly at contactus@ostif.org or our [GitHub](#).

Derek Zimmer, Executive Director

Amir Montazery, Managing Director

Helen Woeste, Communications and Community Manager

Tom Welter, Project Manager



3. Executive Summary

3.1. Context

Paramiko is a pure Python implementation of SSHv2, providing both client and server functions. It forms the basis for the high-level SSH library Fabric, and it is the most popular SSH implementation in Python.

Cryptography is a collection that provides access to cryptographic algorithms and methods for Python programmers. It is a very popular Python/Rust cryptographic library with over 25,000 known dependencies.

This engagement encompasses a comprehensive assessment of the Paramiko library, along with a detailed examination of the Cryptography library's interaction with the rust-openssl bindings, reliable entropy sources, constant-time correctness requirements, code quality, testing procedures, and the CI/CD pipeline. Recommendations for strengthening these areas will be provided.

3.2. Objectives

The Paramiko library is designed to be compact, easy to understand, and limited in functionality to minimize the attack surface. Attacks on SSH implementations are well researched and documented, so OSTIF will direct its efforts towards examining Paramiko's testing, building, and CI systems. This will lead to sustainable enhancements in the library's resilience. It will also examine how the Paramiko initiative ensures that its dependencies are properly implemented. Finally, a manual code review will be conducted to verify correctness and that Paramiko is not vulnerable to known attacks from other SSH implementations.

The Cryptography library boasts a vast array of features and functions, encompassing numerous use cases. Despite its extensive attack surface, this is made possible through the integration of OpenSSL (via rust-openssl). As such, evaluating the Cryptography library amounts to examining the proper utilization of rust-openssl's capabilities, rather than re-examining cryptographic primitives from the ground up. This review of the library should be initially triaged to focus on the use cases that affect Paramiko. All remaining time and resources that are available after the review of the Paramiko use-case should be used in a time-boxed and risk-based approach to evaluate the rest of the Cryptography project.

OpenSSL and rust-openssl themselves are not part of this evaluation, only how they are invoked by Cryptography and Paramiko. If a researcher finds a potential bug in OpenSSL or rust-openssl incidentally during this research, they are free to investigate the issue, report it responsibly and include their findings in the final report. However, this engagement is not a review of OpenSSL nor rust-openssl.

3.3. Findings Summary

During the security audit period, Quarkslab discovered several security issues and vulnerabilities, including the following:

- 0 security issues considered as critical severity;
- 2 security issues considered as high severity;
- 6 security issues considered as medium severity;
- 6 security issues considered as low severity;
- 16 issues considered informative.

ID	Name	Perimeter
HIGH-21	Insecure parameters for digital signatures with RSA	paramiko/ rsakey.py.
HIGH-28	Insecure key sizes accepted for Triple DES	TripleDES in Cryptography
MEDIUM-15	Deprecated group exchange method	paramiko/ kex_gex.py
MEDIUM-16	Insecure minimum modulus size in Diffie-Hellman group exchange	paramiko/ kex_gex.py
MEDIUM-17	Deprecated Diffie-Hellman group	paramiko/ kex_group1.py.
MEDIUM-18	Deprecated GSS-API key exchange methods	paramiko/ kex_gss.py.
MEDIUM-22	Use of 8-byte seed for TripleDES key generation	Encryption.
MEDIUM-24	Wrong type usage in SHA-1 in KexGSSGroup1 and KexGSSGroup14	paramiko/ kex_gss.py
LOW-1	CVE impacting black	Development
LOW-19	Use of MD5 as a Key Derivation Function	paramiko/ pkey.py
LOW-25	Invalid Ed25519 signature causes mishandled exception	ed25519key.py
LOW-27	Invalid Ed25519 signature cause transport thread to crash	paramiko/ ed25519key.py
LOW-29	Insecure RSA key size allowed	RSA Keys in Paramiko and Cryptography
LOW-30	Server can be instantiated over UDP socket	paramiko/ transport.py
INFO-2	Code Quality Improvements for try/except	Code base
INFO-3	Undocumented dependency for testing GSS-API	Tests
INFO-4	Broken tests due to obsolete unittest assertions	Tests
INFO-5	GSS-API is not tested in CI	CI, tests
INFO-6	Enforce servers host keys	RFC compliance

ID	Name	Perimeter
INFO-7	No default support for channel request methods	RFC compliance
INFO-8	Support of deprecated primitive (3DES-CBC)	RFC compliance
INFO-9	Rekeying threshold for data integrity too high	RFC compliance
INFO-10	No DoS protections against physical exhaustion	RFC compliance
INFO-11	No traffic analysis protection by default	RFC compliance
INFO-12	Missing debug message control	RFC compliance
INFO-13	No default support for local security policy	RFC compliance
INFO-14	%C token in SSH config does not match OpenSSH behavior	paramiko/ config.py.
INFO-20	Invalid class method from_private_key_file for AgentKey	paramiko/ agent.py , paramiko/ pkey.py
INFO-23	Mapping Python errors to unwrap_or(false)	Cryptography Rust
INFO-26	Wrong documented type in check_channel_exec_request	paramiko/ server.py

3.4. Recommendation and Action Plan

The suggested actions to take for the various findings are summarized in the table below.

ID	Recommendation
HIGH-21	Remove support for RSA with SHA-1.
HIGH-28	Reject key sizes that are not 24 bytes.
MEDIUM-15	Remove support for <code>diffie-hellman-group-exchange-sha1</code> .
MEDIUM-16	Increase the minimum modulus size to 2048 bits.
MEDIUM-17	Remove support for <code>diffie-hellman-group1-sha1</code> .
MEDIUM-18	Remove the deprecated key exchange methods, replacing them with RFC 8732 additions.
MEDIUM-22	Reject 8-byte input for the key initialization of Triple DES.
MEDIUM-24	Change <code>str(hm)</code> to <code>hm.asbytes()</code> in <code>KexGSSGroup1</code> .
LOW-1	Update <code>black</code> to version 24.3.0.
LOW-19	Warn when using this format, recommend the user to save their keys in PKCS8 or OpenSSH format instead.
LOW-25	Either check the length of the signature before calling <code>verify()</code> or handle the exception.
LOW-27	Handle the exception: either catch the <code>nacl.exception.ValueError</code> exception or check that the signature has the correct length before calling <code>verify()</code> .
LOW-29	Reject RSA keys that are shorter than 2048 bits.
LOW-30	Add a check in <code>Transport.__init__()</code> to verify that <code>sock</code> is a TCP socket.
INFO-2	Fix the try/except pass or continue to have properly handled.
INFO-3	Add <code>k5test</code> to the list of development dependencies. We also recommend mentioning the system packages needed for the GSS-API, such as <code>libkrb5-dev</code> , <code>krb5-admin-server</code> , and <code>krb5-kdc</code> for Debian systems, or <code>krb5-server</code> and <code>krb5-devel</code> for Fedora systems.
INFO-4	Apply the patch provided in Appendix B.
INFO-5	Install the GSS-API dependencies on the CI runner.
INFO-6	Enforce that the server has at least one host key.
INFO-7	<code>check_channel_request</code> should be an abstract method for server developers to override for proper usage.
INFO-8	Use 3DES in SDCTR mode as recommended in RFC 4344 in Section 4.
INFO-9	RFC 4251 recommends 2^{28} in Section 9.3.2.
INFO-10	Add IP-based rate limiting and pre-authentication connection quotas.
INFO-11	RFC 4251 asks for traffic analysis protection by default with the <code>SSH_MSG_IGNORE</code> packet in Section 9.3.9.

ID	Recommendation
INFO-12	Paramiko should implement these controls by default or put the different logging methods as abstract to force developers to properly set them up.
INFO-13	<code>check_auth_password</code> should be an abstract method for server developers to override for proper usage.
INFO-14	Use the full host name for the <code>%l</code> token, and add the <code>%j</code> token, at least to include it in <code>%C</code> .
INFO-20	Override <code>from_private_key_file()</code> in <code>AgentKey</code> to return an error indicating that this method is not supported.
INFO-23	Properly propagate the error instead of converting it into <code>false</code> .
INFO-26	Update the docstring to indicate the correct type.

3.5. Conclusions

Quarkslab has been mandated on behalf of OSTIF to perform the first public security audit of Paramiko performed by an audit firm. Since critical security features of Paramiko involve cryptographic primitives, the scope was expanded to PYCA Cryptography and, more specifically, how Paramiko uses it.

Our work was primarily focused on a detailed, in-depth static analysis, identifying and developing targeted test enhancements, and dynamic testing where possible. We also addressed potential security risks in the CI/CD pipeline to ensure a secure and robust deployment process.

During the audit period, we found a few issues, but nothing that raises Paramiko or Cryptography's security concerns. We believe that all of them can be easily corrected or implemented as best practices.

4. Control audit

As part of the agreement, Quarkslab reviewed the fixes published by Paramiko to address the security issues found during the engagement. The table below shows the issues that were fixed: all HIGH and MEDIUM items were addressed, along with one LOW and two INFO issues.

ID	Fix commit(s)	Fix description
HIGH-21	a448945	Support for using SHA-1 when signing with RSA was removed.
HIGH-28 , MEDIUM-22	pyca/ cryptography	PR 13928 in pyca/cryptography removes implicit key expansion in Triple DES.
MEDIUM-15	9bf5fca	The group exchange methods using SHA-1 were removed.
MEDIUM-16	6fa1556	The minimum modulus size for diffie-hellman-group-exchange-sha256 was increased to 2048.
MEDIUM-17	9bf5fca	The diffie-hellman-group1-sha1 exchange was removed.
MEDIUM-18	1ecc933	Support for GSSAPI was entirely removed.
MEDIUM-24	9bf5fca	The key exchange methods using SHA-1 were removed, including the <code>kex_gss.py</code> file that contained the bug.
LOW-19	acd4bc1	Support for handling keys in OpenSSH format was added. PEM remains the default for compatibility.
INFO-3 , INFO-5	1ecc933	As a consequence of removing GSSAPI, these issues were also solved.

5. Reading Guide

This reading guide describes the different sections present in this report and gives some insights about the information contained in each of them and how to interpret it.

5.1. Executive summary

The executive summary Section 3 presents the results of the assessment in a non-technical way, summarizing all the findings and explaining the associated risks. For each vulnerability, a severity level is provided as well as a name or short description, and one or more mitigation, as shown below.

ID	Name	Category
CRITICAL	Vulnerability Name #1	Injection
HIGH	Vulnerability Name #2	Remote code injection
MEDIUM	Vulnerability Name #3	Denial of Service
LOW	Vulnerability Name #4	Information leak

Each vulnerability is identified throughout this document by a unique identifier `<LEVEL>-<ID>`, where `ID` is a number and `LEVEL` the severity (`INFO`, `LOW`, `MEDIUM`, `HIGH` or `CRITICAL`). Every vulnerability identifier present in the vulnerabilities summary table is a clickable link that leads to the corresponding technical analysis that details how it was found (and exploited if it was the case). Severity levels are explained in Section 5.2.

The executive summary also provides an action plan with a focus on the identified *quick wins*, some specific mitigation that would drastically improve the security of the assessed system.

5.2. Metric definition

This report uses specific metrics to rate the severity, impact and likelihood of each identified vulnerability.

5.2.1. Impact

The impact is assessed regarding the information an attacker can access by exploiting a vulnerability but also the operational impact such an attack can have. The following table summarizes the different levels of impact we are using in this report and their meanings in terms of information access and availability.

CRITICAL	Allows a total compromise of the assessed system, allowing an attacker to read or modify the data stored in the system as well as altering its behavior.
HIGH	Allows an attacker to impact significantly one or more components, giving access to sensitive data or offering the attacker a possibility to pivot and attack other connected assets.
MEDIUM	Allows an attacker to access some information, or to alter the behavior of the assessed system with restricted permissions.
LOW	Allows an attacker to access non-sensitive information, or to alter the behavior of the assessed system and impact a limited number of users.

5.2.2. Likelihood

The vulnerability likelihood is evaluated by taking the following criteria in consideration:

- **Access conditions:** the vulnerability may require the attacker to have physical access to the targeted asset or to be present in the same network for instance, or can be directly exploited from the Internet.
- **Required skills:** an attacker may need specific skills to exploit the vulnerability.
- **Known available exploit:** when a vulnerability has been published and an exploit is available, the probability a non-skilled attacker would find it and use it is pretty high.

The following table summarizes the different level of vulnerability likelihood:

CRITICAL	The vulnerability is easy to exploit even from an unskilled attacker and has no specific access conditions.
HIGH	The vulnerability is easy to exploit but requires some specific conditions to be met (specific skills or access).
MEDIUM	The vulnerability is not trivial to discover and exploit, requires very specific knowledge or specific access (internal network, physical access to an asset).
LOW	The vulnerability is very difficult to discover and exploit, requires highly specific knowledge or authorized access

5.2.3. Severity

The severity of a vulnerability is defined by its impact and its likelihood, following the following table:

		Impact			
					
Likelihood		CRITICAL	CRITICAL	HIGH	MEDIUM
		CRITICAL	HIGH	HIGH	MEDIUM
		HIGH	HIGH	MEDIUM	LOW
		MEDIUM	MEDIUM	LOW	LOW

6. Discovery

6.1. Projects Information

Paramiko is a pure Python implementation of the SSHv2 protocol, providing both client and server capabilities. It serves as the foundation for Fabric, a more advanced SSH library that is recommended for everyday tasks such as running remote shell commands or transferring files. Using Paramiko directly is best for those requiring advanced or low-level control, or who wants to create an SSH server (sshd) inside Python.

The scope of this audit is defined as follows:

- Paramiko critical cryptographic functions;
- (PYCA) Cryptography's OpenSSL Rust Bindings;
- CI/CD CircleCI for Paramiko and Github Actions for (PYCA) Cryptography.

6.1.1. Projects Breakdown

The audited source code has been divided into two distinct projects: `paramiko` and `cryptography`.

Project	Description	URL	Git Commit Hash
<code>paramiko</code>	Source code of the Paramiko implementation of SSHv2.	https://github.com/paramiko/paramiko	AAD0370DB9FD5C22064A673C9602FC48314EB6F4
<code>cryptogra-phy</code>	Source code of the Python Crypto Authority library.	https://github.com/pyca/cryptography	66198C23C97C9594D64540E5A866E4B4121AEC2D

`paramiko` is the primary project implementing the SSHv2 protocol, and the subject of this audit. Mostly written in Python, it contains about 35,000 lines of code, including many tests and a fuzzing harness for the OSS-Fuzz project.

1	\$ tokeni ~/audit/paramiko					Bash
2	=====					
3	Language	Files	Lines	Code	Comments	Blanks
4	=====					
5	Python	83	31595	25216	3244	3135
6	ReStructuredText	28	2465	2155	0	310
7	TOML	1	80	61	9	10
8	XML	1	19	19	0	0
9	Markdown	1	7	0	5	2
10	Autoconf	1	5	5	0	0
11	INI	1	3	3	0	0
12	YAML	1	1	1	0	0
13	=====					
14	Total	117	34175	27460	3258	3457
15	=====					

Listing 1: Number of lines of the `paramiko` project.

`cryptography` is a Python library that serves as the foundation for several critical cryptographic features in Paramiko. In 2021, its backend was largely rewritten in Rust, utilizing existing Rust bindings to OpenSSL. For the purposes of this audit, we concentrate on these OpenSSL bindings, which account for approximately 25,000 lines of code.

- the changelog, which includes fixed vulnerabilities and yanked versions;
- an FAQ section;
- a security section;
- both the high-level recipes (Fernet for symmetric encryption and X.509 for certificate manipulation), along with the low level `hazmat` module for using cryptographic primitives directly.

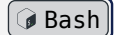
6.1.3. Previous Audits

We could not locate any publicly available security audit reports for either of the two projects.

6.2. Installation

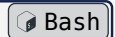
Paramiko can be installed via the PyPI package manager.

```
python -m pip install paramiko
```



It enables the use of GSS-API/SSPI with the help of some additional dependencies:

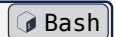
```
python -m pip install paramiko[gssapi]
```



Please note that GSS-API/SSPI support requires an installed implementation of the protocol.

To facilitate its development, Paramiko has listed the tools it uses in `pyproject.toml`. With the release of version 4.0, Paramiko now supports managing dependencies with the package and project manager `uv`:

```
# Create new virtual environment that includes pip (required for invoke)
uv venv --seed
# Install dependencies from uv.lock, including the ones for GSS-API/SSPI
uv sync --extra gssapi
```



Paramiko includes demos of server and client instances in its repository. However, these are outdated and do not work with version 4.0.0. For our testing we have created two demo servers, one that includes all options described in the documentation, and a basic one to get started with password authentication. Both scripts are available at <https://github.com/quarkslab/paramiko-server-demo>.

6.3. Source Discovery

6.3.1. Code Structure

Paramiko's code structure is straightforward: the source code of the package is composed of a singular directory, `paramiko`, with a flattened structure shown in Listing 3.

```
~/paramiko ((4.0.0))> tree -A paramiko
paramiko
├── __init__.py
├── _winapi.py
├── agent.py
├── auth_handler.py
├── auth_strategy.py
├── ber.py
├── buffered_pipe.py
├── channel.py
├── client.py
├── common.py
├── compress.py
├── config.py
├── ecdsakey.py
├── ed25519key.py
├── file.py
├── hostkeys.py
├── kex_curve25519.py
├── kex_ecdh_nist.py
├── kex_gex.py
├── kex_group1.py
├── kex_group14.py
├── kex_group16.py
├── kex_gss.py
├── message.py
├── packet.py
├── pipe.py
├── pkey.py
├── primes.py
├── proxy.py
├── rsakey.py
├── server.py
├── sftp.py
├── sftp_attr.py
├── sftp_client.py
```

```

├─ sftp_file.py
├─ sftp_handle.py
├─ sftp_server.py
├─ sftp_si.py
├─ ssh_exception.py
├─ ssh_gss.py
├─ transport.py
├─ util.py
├─ win_openssh.py
├─ win_pageant.py

```

1 directory, 44 files

Listing 3: Structure of Paramiko's source code.

These different Python source code files implement classes and functions around five main components: the high-level API, session management, the transport layer, the authentication layer, and configuration/Utils. These components interact following the description depicted in Figure 2.



Figure 2: High-level code structure of Paramiko.

6.3.2. Dependencies

Paramiko has four core dependencies:

- `bcrypt` (≥ 3.2): for password hashing and Ed25519 key support (KDF for private key encryption),
 - `cryptography` (≥ 3.3): the primary cryptographic backend providing low-level primitives,
 - `invoke` (≥ 2.0): task automation framework (moved to core dependencies in v4.0.0),
 - `pynacl` (≥ 1.5): for Ed25519 support (Ed25519 key generation, sign, verify);
- and three optional dependencies for GSS-API/SSPI/Kerberos support:
- `pyasn1` ($\geq 0.1.7$): required on all platforms,
 - `gssapi` ($\geq 1.4.1$): for Unix systems,
 - `pywin32` ($\geq 2.1.8$): for Windows systems.

6.3.3. Code Quality

In summary, the code quality of the Paramiko project is excellent. The code is generally well documented with function docstrings used to generate the corresponding documentation pages.

Info

Type-hints (introduced in Python 3.5) are not used for Paramiko. It can be of great help to detect any logical bugs when combined with a static type checker such as `mypy`.

Paramiko employs several tools to ensure a top-notch codebase:

- `codecov`: monitors changes in code coverage by unit tests;
- `flake8`: lints the code;
- `black`: formats the code and maintains a consistent coding style;
- `codespell`: corrects any typos in the comments and documentation;
- `pytest`: runs the unit and integration tests;
- `coverage.py`: generates code coverage reports.

LOW-1		CVE impacting <code>black</code>	
Likelihood		Impact	
Perimeter	Development		
Description			
CVE-2024-21503 impacts <code>black</code> prior to version 24.3.0. As of version 4.0, Paramiko has pinned version 22.12.0 of <code>black</code> as a development dependency.			
Recommendation			
Update <code>black</code> to version 24.3.0.			

To supplement this battery of tools, we used [bandit](#), a tool designed to find common security issues in Python code. It uncovered certain code quality concerns, such as:

- usage of `try/except/pass` in `channel.py:139`, `client.py:382`, `server.py:696`, `sftp_server.py:161`;
- and usage of `try/except/continue` in `primes.py:121`.

INFO-2	Code Quality Improvements for <code>try/except</code>
Perimeter	Code base
Description	
Usage of <code>try/except/pass</code> as well as <code>try/except/continue</code> are made in different portions of Paramiko's code. This can lead to improper check or handling of exceptional conditions as explained in CWE-703 .	
Recommendation	
Fix the <code>try/except</code> pass or continue to have properly handled.	

6.4. Tests

Paramiko has a good collection of tests, with files targeting specific features such as `ssh_config` parsing, and other tests checking the interaction between components such as `test_client.py`. They can be found under the `tests` directory, in a flattened structure as shown in Listing 4.

```
~/paramiko ((4.0.0))> tree -A tests
tests
├── agent.py
├── auth.py
├── badhash_key1.ed25519.key
├── badhash_key2.ed25519.key
├── blank_rsa.key
├── configs
│   ├── basic
│   ├── canon
│   ├── canon-always
│   ├── canon-ipv4
│   ├── canon-local
│   ├── canon-local-always
│   ├── deep-canon
│   └── deep-canon-maxdots
```


- | └─ empty-canon
- | └─ fallback-no
- | └─ fallback-yes
- | └─ hostname-exec-tokenized
- | └─ hostname-tokenized
- | └─ invalid
- | └─ match-all
- | └─ match-all-after-canonical
- | └─ match-all-and-more
- | └─ match-all-and-more-before
- | └─ match-all-before-canonical
- | └─ match-canonical-no
- | └─ match-canonical-yes
- | └─ match-complex
- | └─ match-exec
- | └─ match-exec-canonical
- | └─ match-exec-negation
- | └─ match-exec-no-arg
- | └─ match-final
- | └─ match-host
- | └─ match-host-canonicalized
- | └─ match-host-from-match
- | └─ match-host-glob
- | └─ match-host-glob-list
- | └─ match-host-name
- | └─ match-host-negated
- | └─ match-host-no-arg
- | └─ match-localuser
- | └─ match-localuser-no-arg
- | └─ match-orighost
- | └─ match-orighost-canonical
- | └─ match-orighost-no-arg
- | └─ match-user
- | └─ match-user-explicit
- | └─ match-user-no-arg
- | └─ multi-canon-domains
- | └─ no-canon
- | └─ robey
- | └─ zero-maxdots
- └─ conftest.py
- └─ __init__.py
- └─ _loop.py

- |— pkey.py
- |— _stub_sftp.py
- |— _support
 - |— ecdsa-256.key
 - |— ecdsa-256.key-cert.pub
 - |— ed25519.key
 - |— ed25519.key-cert.pub
 - |— ed448.key
 - |— rsa.key
 - |— rsa.key-cert.pub
 - |— rsa-lonely.key
 - |— rsa-missing.key-cert.pub
- |— test_buffered_pipe.py
- |— test_channelfile.py
- |— test_client.py
- |— test_config.py
- |— test_ecdsa_384.key
- |— test_ecdsa_384_openssh.key
- |— test_ecdsa_521.key
- |— test_ecdsa_password_256.key
- |— test_ecdsa_password_384.key
- |— test_ecdsa_password_521.key
- |— test_ed25519-funky-padding.key
- |— test_ed25519-funky-padding_password.key
- |— test_ed25519_password.key
- |— test_file.py
- |— test_gssapi.py
- |— test_hostkeys.py
- |— test_kex_gss.py
- |— test_kex.py
- |— test_message.py
- |— test_packetizer.py
- |— test_pkey.py
- |— test_proxy.py
- |— test_rsa.key.pub
- |— test_rsa_openssh.key
- |— test_rsa_openssh_nopad.key
- |— test_rsa_password.key
- |— test_sftp_big.py
- |— test_sftp.py
- |— test_ssh_exception.py
- |— test_ssh_gss.py

```
├─ test_transport.py
├─ test_util.py
└─ _util.py

3 directories, 99 files
```

Listing 4: Structure of Paramiko tests

The tests rely on `pytest`, a Python testing framework. They can be run through the `invoke` task included in the repository:

```
invoke test
```

 Bash

Running the command reports that all *selected* tests pass:

```
487 passed, 19 skipped, 54 deselected in 18.29s
```

By default, the `test` task does not include tests marked as slow. Adding the `--include-slow` option runs all tests:

```
~/paramiko ((4.0.0))> invoke test --include-slow
```

 Bash

```
...
539 passed, 21 skipped, 1 warning in 80.56s (0:01:20)
```

6.4.1. Test coverage

There is also a task to run the tests and generate the coverage report:

```
# Run the tests with coverage support -- it includes the slow tests
invoke coverage
# Generate the HTML view
coverage html
# Open on Linux
xdg-open htmlcov/index.html
# Or in MacOS
open htmlcov/index.html
```

 Bash

We note that running the tests with the `coverage` task results in 1 failure from `test_client.py::SSHClientTest::test_cleanup`. A note above the failing assertion indicates that it is known to randomly fail in the CI.

The test coverage of the overall implementation is quite good: as shown in Figure 3, 77%¹ of the code in `paramiko` is covered. Almost all files have a coverage of over 60%, with most leaning towards 90% covered lines. There are however four outliers: less than 30% lines of code are covered in `kex_gss.py`, `primes.py`, and `ssh_gss.py`, and `agent.py` is only at 42%.

¹While Figure 3 says 82%, this corresponds to the coverage of the entire repository *including the tests directory*. The number at the bottom corresponds to the coverage of the files included in the current search, which is limited to the `paramiko` directory.

We explored the coverage to determine if all critical cryptographic functions were properly covered: the findings are in Section 10.

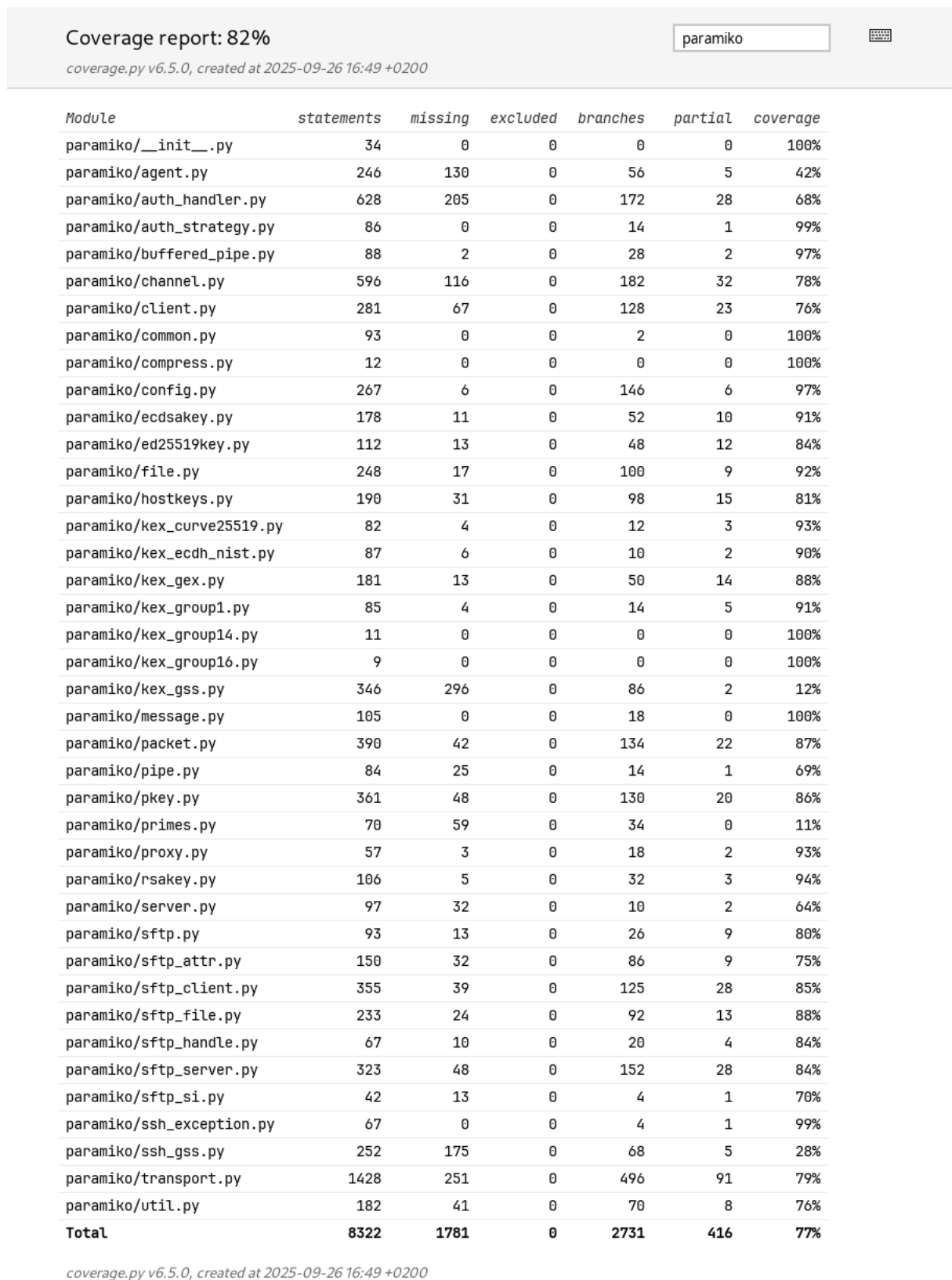


Figure 3: Code coverage of paramiko.

The low coverage of `kex_gss.py`, `primes.py`, and `ssh_gss.py` can be explained by some of the tests that were skipped: while the [documentation](#) indicates that the GSS-API feature requires the installation of an implementation of GSS-API such as Kerberos. It does not mention that the `k5test` Python package is also required to test this feature. Without it, the corresponding tests are silently skipped, as, by default, `pytest` does not show the reason why tests are skipped.

INFO-3	Undocumented dependency for testing GSS-API
Perimeter	Tests
Description	
Using GSS-API with Paramiko on Linux systems requires a working installation of GSS-API itself. This is indicated in the documentation, however, testing this feature also requires the <code>k5test</code> package: if absent, the tests will be skipped, and running the test suite with <code>inv test</code> does not indicate which tests have been skipped.	
Recommendation	
Add <code>k5test</code> to the list of development dependencies. We also recommend mentioning the system packages needed for the GSS-API, such as <code>libkrb5-dev</code> , <code>krb5-admin-server</code> , and <code>krb5-kdc</code> for Debian systems, or <code>krb5-server</code> and <code>krb5-devel</code> for Fedora systems.	

After installing all the necessary dependencies, the GSS-API tests are also executed and some fail:

```
~/paramiko ((4.0.0))> invoke test --include-slow
...
6 failed, 539 passed, 14 skipped, 1 xfailed, 1 warning in 89.14s (0:01:29)
```

Four of these failures are due to the usage of obsolete aliases of `unittest` assertions (e.g. `assertEquals` became `assertEqual`, etc.) that were [removed in Python 3.12](#). The four tests pass when using the correct assertions (see Appendix B).

INFO-4	Broken tests due to obsolete <code>unittest</code> assertions
Perimeter	Tests
Description	
Four GSS-API tests are still using now-obsolete aliases of <code>unittest</code> assertions, such as <code>assertEquals</code> instead of <code>assertEqual</code> . This led the four tests to fail without actually testing the code.	
Recommendation	
Apply the patch provided in Appendix B.	

Paramiko's CI runs the tests on several Python versions, including Python 3.12. The pipeline succeeds even in Python 3.12+ because the dependencies for GSS-API are not installed, so the tests are skipped.

INFO-5	GSS-API is not tested in CI
Perimeter	CI, tests
Description	
The CI runner setup does not go through the steps to install the dependencies necessary to test the GSS-API integration in Paramiko, meaning that this feature is not tested regularly, if at all.	
Recommendation	
Install the GSS-API dependencies on the CI runner.	

6.5. PYCA Cryptography

Since the audit covers only a portion of the PYCA cryptography library, we don't examine its overall code structure, quality, or implementations. Instead, we concentrate on the Rust bindings for OpenSSL in PYCA Cryptography. Our initial assessment should prioritize the use cases that impact Paramiko.

For this purpose, we have listed all the calls to cryptography modules in Paramiko. These calls involve the following primitives:

- Symmetric Encryption Algorithms:
 - AES-128/192/256 in CTR mode for stream encryption [transport.py:243-260](#)
 - AES-128/192/256 in CBC mode for block encryption [transport.py:261-278](#)

- AES-128/256 in GCM mode for authenticated encryption [transport.py:285-298](#)
 - 3DES (Triple DES in CBC mode for legacy compatibility) [transport.py:279-284](#)
- Message Authentication Codes (MACs)
 - HMAC-SHA2-256/512 for modern security [transport.py:190-199](#)
 - HMAC-SHA1 and HMAC-MD5 for legacy compatibility
- Hash Functions
 - SHA-256, SHA-384, SHA-512 for ECDSA and modern key exchange [kex_ecdh_nist.py:6](#)
 - SHA-1 for legacy Diffie-Hellman and DSA operations [kex_group14.py:25](#)
 - MD5 for fingerprint generation [transport.py:30](#)
- Asymmetric Key Algorithms
 - RSA with PKCS#1 v1.5 padding
 - RSA-SHA2-256/512 for modern signatures [transport.py:206-210](#)
 - DSA (Digital Signature Algorithm) for legacy key support [dsskey.py:26](#)
 - P-256, P-384, P-521 curves for modern elliptic curve cryptography [ecdsa.py:26](#)
 - Ed25519 for modern elliptic curve signatures [transport.py:202](#)
- Key Exchange Algorithms
 - Classic DH Group 1 (1024-bit) and Group 14 (2048-bit) [kex_group14.py:28-35](#)
 - ECDH-SHA2-NISTP256/384/521 [kex_ecdh_nist.py:19-23](#)
 - X25519 key exchange with SHA-256 [kex_curve25519.py:20-21](#)
- Random Number Generation
 - X25519 private key generation [kex_curve25519.py:45](#)

6.6. Fuzzing

Both Paramiko and Cryptography are fuzzed as part of the [OSS-Fuzz project](#).

OSS-Fuzz reported four public issues on Paramiko (see [here](#)), none are relevant to security: three are related to build issues and the fourth one is related to coverage issues. The lack of bug reports can be explained in part by poor coverage of the project: only 28% of Paramiko is covered (see [here](#)), thus important functions are not being tested with the fuzzer. For example, the `Packetizer` class is responsible for encrypting outgoing packets with the `send_message()` method and decrypting incoming packets with the `read_message()` method. According to the [coverage report](#) from 21/09/2025, only the `send_message()` method is covered.

In contrast, OSS-Fuzz reported some bugs that are not related to build or coverage issues for Cryptography (see [here](#)):

- <https://issues.oss-fuzz.com/issues/42512733>
- <https://issues.oss-fuzz.com/issues/42516667>
- <https://issues.oss-fuzz.com/issues/42527541>
- <https://issues.oss-fuzz.com/issues/42535212>
- <https://issues.oss-fuzz.com/issues/423078671>

All the crashes, except for the last one, have been fixed.

The coverage for [Cryptography](#) is more reasonable at 73%.

7. Audit Overview

7.1. Hypotheses

The following hypotheses have been considered to evaluate Paramiko's implementation of SSHv2:

SSH RFCs soundness and correctness

For the scope of this audit, all the RFCs implemented by Paramiko for the SSHv2 protocol and its extensions are sound and correct.

Dependency Safety

All Paramiko's dependencies are assumed safe and are out of the scope of this audit (except for PYCA Cryptography, which is part of this audit scope).

Fuzzing as low priority

Fuzzing is not considered a high priority testing approach, as harnesses for Paramiko and Cryptography are already incorporated in projects such as OSS-Fuzz.

Please note, however, that strategies such as differential fuzzing will be employed during the evaluation of Paramiko to assess high-assurance cryptography (see Section 7.2).

Advanced side-channel attacks out-of-scope

Paramiko will not be tested against advanced side-channel attacks. Instead, it only considers the aspect of constant-timeness.

Windows usage out of scope

We will not consider Paramiko implementation and deployment for Windows targets.

7.2. Threats and Security checks

For the audit of Paramiko we concentrate on two areas:

High-assurance cryptography

Ensure that Paramiko's implementation of cryptographic primitives has reliable entropy sources and freshness, memory safety, functional correctness, and constant-time execution.

⚠ CI/CD and build issues

Ensure that Paramiko's build and deployment are safe against supply chain attacks.

7.3. Methodology

The steps involved in the security evaluation of the Paramiko and Cryptography libraries are as follows:

- Verify compliance regarding the implemented RFCs.
- Static (manual and automated) analysis of the relevant source codes (namely critical cryptographic functions).
- Differential fuzzing testing with an in-house tool to verify the correct behavior of the Rust bindings with respect to memory safety and constant-timeness.
- Dynamic testing of entropy sources (with crypto-condor) and sensitive variable tainting for constant-timeness checks. Following the static analysis, other dynamic testing cases should emerge.
- Testing enhancements via the implementation of a crypto-condor plug-in to incorporate in the CI for cryptographic compliance and testing of entropy sources.
- Build-system and CI review (artifacts, permissions, pipelines, dependencies, policies, etc.).

8. Paramiko's compliance to SSH RFCs

The Secure Shell protocol (SSH) is a widely used protocol for securely sending commands to a computer over an insecure network. A comprehensive overview can be found in a Cloudflare blogpost (see [here](#)).

The detailed guidelines and requirements for this protocol were established in various RFCs in 2006. Subsequent errata and updates have been made to ensure its continued security and compatibility with the latest cryptographic primitives.

The Internet Engineering Task Force (IETF) maintains a datatracker page for the working group that developed SSH. There, you will find all the relevant RFCs, including their errata and updates (see [here](#)). The most important RFCs for implementing SSH securely are:

- [RFC 4251](#): Defines the SSH architecture and the main security considerations for each of its components.
- [RFC 4252](#): Defines the SSH authentication protocol for a client to prove its identity to a server over an already secure SSH transport layer connection.
- [RFC 4253](#): Defines the SSH transport layer protocol for the establishment of a secure communication between the client and the server by providing encryption, server authentication, integrity protection, and optional compression over TCP/IP.
- [RFC 4254](#): Defines the SSH connection protocol by enabling multiple logical “channels” (shell sessions, command execution, file transfer, etc.) over a single encrypted connection established by the transport and authentication layers.

8.1. Compliance and Security considerations

Please note that the naming of the different compliance and security checks comes from the RFCs directly.

8.1.1. Overall

8.1.1.1. Host Keys

Servers hold host keys, which they use for verification during key exchange. This way, clients can be sure they are communicating with the correct server. Multiple hosts may have the same host keys.

Within Paramiko's host key system, multiple key types (RSA, ECDSA, Ed25519) are supported, with automatic handling of SHA-2 extensions for RSA keys. The `server_key_dict` stores all available keys, but only the negotiated key type is used during the actual SSH session. On the client side, host key verification is managed separately through the `HostKeys` class, which handles `known_hosts` files.

RFC

Server:

- must have a host key (can be multiple algorithms, but one of them **required** to be a public key algorithm)

Client:

- should not allow connections by default to unverified host keys (“trust on first use”)
- trust models for storage:
 - database of local known hosts
 - public key infrastructure

Success

Server: Requirement for the key to be from a public key algorithm is fulfilled.

Client: The first connection to a new server requires user intervention unless using `AutoAddPolicy` (*client.py:440-466*).

Maintains a `known_hosts` file containing previously seen server keys.

Fail

Server: No code exists to ensure that the server has at least one host key.

INFO-6

Enforce servers host keys

Perimeter

RFC compliance

Description

In the current state of the code, there is no enforcement in the `server mode` for the dictionary that contains the host keys (`server_key_dict` in *transport.py:577*) to have at least one host key, as required by RFC 4251. In fact, `server_key_dict` is used in:

- `add_key` function (*transport.py:841*), which is only called by test functions.
- `get_server_key` function (*transport.py:860*), which is used during key type negotiation with the client, and the agreed-upon key type is stored in `self.host_key_type`.
- `_send_kex_init` function (*transport.py:2396*), which gives the list of the server’s available keys to the other side.

However the following code:

```
available_server_keys = list(
    filter(
```

 Python

```

        list(self.server_key_dict.keys().__contains__,
              ...
              self.preferred_keys,
            )
    )
)

```

can return an empty list and does not ensure that the server has at least one host key (as required by RFC 4251). It's up to the server developer to configure this correctly.

Please note that the Server's host keys are not specified in the `ServerInterface` class from *server.py*.

Recommendation

Enforce that the server has at least one host key.

8.1.1.2. Extensibility

① RFC

All implementations must support a minimal set of algorithms to ensure interoperability.

Success

Paramiko `Transport` class (*transport.py:219-239*) defines a list of preferred algorithms that serve as the minimal supported set for interoperability.

```

 Preferred Python
 _preferred_kex = (
     "ecdh-sha2-nistp256", "ecdh-sha2-nistp384", "ecdh-sha2-nistp521",
     "diffie-hellman-group16-sha512", "diffie-hellman-group-exchange-sha256",
     "diffie-hellman-group14-sha256", "diffie-hellman-group-exchange-sha1",
     "diffie-hellman-group14-sha1", "diffie-hellman-group1-sha1",
 )
 if KexCurve25519.is_available():
     _preferred_kex = ("curve25519-sha256@libssh.org",) + _preferred_kex
 _preferred_gsskex = (
     "gss-gex-sha1-toWM5Slw5Ew8Mqkay+a12g==",
     "gss-group14-sha1-toWM5Slw5Ew8Mqkay+a12g==",
     "gss-group1-sha1-toWM5Slw5Ew8Mqkay+a12g==",
 )
 ...
 cipher_info = {
     "aes128-ctr": {

```

```

    "class": algorithms.AES, "mode": modes.CTR, "block-size": 16, "key-size":
    16
  },
  "aes192-ctr": {
    "class": algorithms.AES, "mode": modes.CTR, "block-size": 16, "key-size":
    24,
  },
  "aes256-ctr": {
    "class": algorithms.AES, "mode": modes.CTR, "block-size": 16, "key-size":
    32,
  },
  "aes128-cbc": {
    "class": algorithms.AES, "mode": modes.CBC, "block-size": 16, "key-size":
    16,
  },
  "aes192-cbc": {
    "class": algorithms.AES, "mode": modes.CBC, "block-size": 16, "key-size":
    24,
  },
  "aes256-cbc": {
    "class": algorithms.AES, "mode": modes.CBC, "block-size": 16, "key-size":
    32,
  },
  "3des-cbc": {
    "class": TripleDES, "mode": modes.CBC, "block-size": 8, "key-size": 24,
  },
  "aes128-gcm@openssh.com": {
    "class": aead.AESGCM, "block-size": 16, "iv-size": 12, "key-size": 16,
    "is_aead": True,
  },
  "aes256-gcm@openssh.com": {
    "class": aead.AESGCM, "block-size": 16, "iv-size": 12, "key-size": 32,
    "is_aead": True,
  },
}

```

8.1.1.3. Policy Issues

④ RFC

The following policy issues should be addressed in the configuration mechanisms of each implementation:

- Encryption, integrity, and compression algorithms: ordered list for preferences.

- Public key algorithms and key exchange method to be used for host authentication.
- The authentication methods that are to be required by the server for each user.
- The operations that the user is allowed to perform using the connection protocol.

Success

- Paramiko implements ordered preference lists for all cryptographic algorithms through the `Transport` class (*transport.py:219-237*). These preferences can be modified through the `SecurityOptions` interface (*transport.py:3175-3253*)

```
class SecurityOptions:
```

 Python

```
"""
```

```
Simple object containing the security preferences of an ssh transport.
These are tuples of acceptable ciphers, digests, key types, and key
exchange algorithms, listed in order of preference.
```

- Paramiko supports configurable public key algorithm preferences through `preferred_keys` and key exchange algorithms through `preferred_kex`. The negotiation process respects these preferences during the `KEXINIT` phase (*transport.py:2563-2601*).
- Paramiko provides the `ServerInterface` class where servers can configure authentication requirements per user through the `get_allowed_auths()` method (*server.py:89-104*).

```
def get_allowed_auths(self, username):
```

 Python

```
"""
```

```
Return a list of authentication methods supported by the server.
This list is sent to clients attempting to authenticate, to inform them
of authentication methods that might be successful.
```

Fail

- Paramiko implements operation control through the `ServerInterface` channel request methods (*server.py:44-87*), which is set to return an error (`OPEN_FAILED_ADMINISTRATIVELY_PROHIBITED`) by default. Server developers need to override this function for proper usage, which means that the configuration mechanisms are not (fully) properly addressed.

```
def check_channel_request(self, kind, chanid):
```

 Python

```
"""
```

```
Determine if a channel request of a given type will be granted, and
return ``OPEN_SUCCEEDED`` or an error code. This method is
called in server mode when the client requests a channel, after
authentication is complete.
```



```
...
The default implementation always returns
``OPEN_FAILED_ADMINISTRATIVELY_PROHIBITED``.
...
"""
return OPEN_FAILED_ADMINISTRATIVELY_PROHIBITED
```

INFO-7	No default support for channel request methods
Perimeter	RFC compliance
Description	
check_channel_request method always returns <code>False</code> by default and does not configure channel request methods.	
Recommendation	
check_channel_request should be an abstract method for server developers to override for proper usage.	

8.1.1.4. Security Properties

ⓘ RFC

- All encryption, integrity, and public key algorithms used are well-known, well-established algorithms.
- All algorithms are used with cryptographically sound key sizes.
- All algorithms are negotiated, and in case some algorithm is broken, it is easy to switch to some other algorithm without modifying the base protocol.

Success

These three properties are properly followed, as seen in a previous subsection.

8.1.1.5. Pseudo-Random Number generation

ⓘ RFC

The protocol ensures that each session key is unique by incorporating random, session-specific data into the hash function that generates it. The number random generation should:

- Ensure that all of the random numbers are of good quality.

- If the random data here (e.g. Diffie-Hellman (DH) parameters) are pseudo-random, then the pseudo-random number generator should be cryptographically secure.
- The amount of entropy available to a given client or server may sometimes be less than what is required. In this case, one must either resort to pseudo-random number generation regardless of insufficient entropy or refuse to run the protocol. The latter is preferable.

Success

- Paramiko does implement session key binding with random, session-specific data in its key derivation process. The core implementation is in the `_compute_key()` method in the `Transport` class (*transport.py:2013-2041*).

```
def _compute_key(self, id, nbytes):
    """id is 'A' - 'F' for the various keys used by ssh"""
    m = Message()
    m.add_mpoint(self.K)
    m.add_bytes(self.H)
    m.add_byte(b(id))
    m.add_bytes(self.session_id)
    # Fallback to SHA1 for kex engines that fail to specify a hex
    # algorithm, or for e.g. transport tests that don't run kexinit.
    hash_algo = getattr(self.kex_engine, "hash_algo", None)
    hash_select_msg = "kex engine {} specified hash_algo {!r}".format(
        self.kex_engine.__class__.__name__, hash_algo
    )
    if hash_algo is None:
        hash_algo = sha1
        hash_select_msg += ", falling back to sha1"
    if not hasattr(self, "_logged_hash_selection"):
        self._log(DEBUG, hash_select_msg)
        setattr(self, "_logged_hash_selection", True)
    out =sofar = hash_algo(m.asbytes()).digest()
    while len(out) < nbytes:
        m = Message()
        m.add_mpoint(self.K)
        m.add_bytes(self.H)
        m.add_bytes(sofar)
        digest = hash_algo(m.asbytes()).digest()
        out += digest
        sofar += digest
    return out[:nbytes]
```

- Paramiko has migrated to using `os.urandom` for all cryptographic random number generation, which provides cryptographically secure randomness from the operating system's entropy pool (*changelog.rst:1568-1569*). This change was implemented across the codebase.

8.1.1.6. Control Character Filtering

RFC

When displaying text to a user, such as error or debug messages, the client software should replace any control characters (except tab, carriage return, and newline) with safe sequences to avoid attacks by sending terminal control characters.

Success

Paramiko implements control character sanitization through the `safe_string()` function in the `util` module (*util.py:112-121*).

8.1.2. Transport Layer

8.1.2.1. Confidentiality

RFC

CBC Mode is theoretically vulnerable to chosen ciphertext attacks because of the high predictability of the start of packet sequences. It should be avoided unless relatively long block sizes are used.

Fail

Paramiko still supports 3DES in CBC mode (namely a block size of 64-bit), which is susceptible to attacks. For instance, the French state agency ANSSI considers 128 bits to be the minimum block size for symmetric ciphers from 2026 onwards.

INFO-8	Support of deprecated primitive (3DES-CBC)
Perimeter	RFC compliance
Description	
Paramiko still supports 3DES in CBC mode.	
Recommendation	
Use 3DES in SDCTR mode as recommended in RFC 4344 in Section 4.	

8.1.2.2. Data integrity

RFC

MACs use 32-bit sequence numbers, which may be leaking information after 2^{32} packets have been sent. The transport protocol recommends rekeying after one gigabyte of data, and the smallest possible packet is 16 bytes. Therefore, rekeying should happen after 2^{28} packets at the very most.

Please note that [RFC 4344](#) asks for rekeying at least once every 2^{32} outgoing packet, which is superseded by this bound.

Fail

Paramiko defines rekeying thresholds in the `Packetizer` class that are more relaxed than the protocol recommendation (*packet.py:72-73*). Both `REKEY_PACKETS` and `REKEY_BYTES` are set to 2^{29} (twice the recommended threshold).

```
# READ the secsh RFC's before raising these values.  if anything,
# they should probably be lower.
REKEY_PACKETS = pow(2, 29)
REKEY_BYTES = pow(2, 29)
```

 Python

INFO-9	Rekeying threshold for data integrity too high
Perimeter	RFC compliance
Description	
Paramiko defines the rekeying threshold to 2^{29} for packets and bytes instead of 2^{28} .	
Recommendation	
RFC 4251 recommends 2^{28} in Section 9.3.2.	

8.1.2.3. Replay

❗ RFC

The transport protocol provides a unique session identifier (bound in part to pseudo-random data that is part of the algorithm and key exchange process) that can be used by higher-level protocols to bind data to a given session and prevent replay of data from prior sessions.

Success

Paramiko does implement this session identifier mechanism. The session identifier is created during the initial key exchange and permanently bound to the session through the `_set_K_H()` method (*transport.py:1986-1993*). The session ID is set to the exchange hash `H` from the first key exchange only (if `self.session_id` is `None`), ensuring it remains constant throughout the session's lifetime even during rekeying operations.

8.1.2.4. Man-in-the-Middle

❗ RFC

This protocol makes no assumptions or provisions for infrastructure or means for distributing the public keys of hosts. It is expected that this protocol will sometimes be used without first verifying the association between the server host key and the server host name. Such usage is vulnerable to man-in-the-middle attacks.

8.1.2.5. Denial of Service

❗ RFC

This protocol is designed to be used over a reliable transport. If transmission errors or message manipulation occur, the connection is closed. The connection should be re-

established if this occurs. Denial of service attacks of this type (wire cutter) are almost impossible to avoid.

In addition, this protocol is vulnerable to denial of service attacks because an attacker can force the server to go through the CPU and memory-intensive tasks of connection setup and key exchange without authenticating. Implementers should provide features that make this more difficult, for example, only allowing connections from a subset of clients known to have valid users.

Success

Paramiko implements this protocol behavior through multiple error handling mechanisms that close connections when transmission errors or protocol violations are detected.

Connection closure on transmission errors: Paramiko closes connections when various types of transmission errors or protocol violations occur. The `SSHClient.connect()` method demonstrates this pattern by catching socket errors and closing the connection (*client.py:387-399*). When socket errors occur during connection establishment, Paramiko explicitly closes the socket and raises appropriate exceptions.

```
except socket.error as e:
    # As mentioned in socket docs it is better
    # to close sockets explicitly
    if sock:
        sock.close()
    # Raise anything that isn't a straight up connection error
    # (such as a resolution error)
    if e.errno not in (ECONNREFUSED, EHOSTUNREACH):
        raise
    # Capture anything else so we know how the run looks once
    # iteration is complete. Retain info about which attempt
    # this was.
    errors[addr] = e
```

Python

Protocol error handling: The Transport layer implements strict protocol validation and closes connections when message manipulation or protocol violations are detected. Recent security fixes show this approach, particularly for the Terrapin Attack (CVE-2023-48795) (*changelog.rst:82-92*).

Channel-level error handling: At the channel level, Paramiko implements error detection and connection cleanup. When errors occur, channels are properly closed and the transport is notified (*channel.py:1221-1224*).

```
e = self.transport.get_exception()
if e is None:
```

Python

```
e = SSHException("Channel closed.")
raise e
```

Connection re-establishment: While Paramiko does not automatically re-establish connections (as this would be application-level logic), it provides the infrastructure for applications to detect connection failures and re-establish them (*demo.py:167-174*).

```
except Exception as e:
    print("*** Caught exception: " + str(e.__class__) + ": " + str(e))
    traceback.print_exc()
try:
    t.close()
except:
    pass
sys.exit(1)
```

Python

Fail

Paramiko does not implement built-in protections against the type of DoS attacks, such as going through the CPU, memory-intensive tasks of connection, etc. The library leaves connection filtering and rate limiting to the application layer or external infrastructure.

INFO-10

No DoS protections against physical exhaustion

Perimeter

RFC compliance

Description

Paramiko leaves connection filtering and rate limiting in the developers' care.

Recommendation

Add IP-based rate limiting and pre-authentication connection quotas.

8.1.2.6. Connection Establishment

① RFC

- Transport Medium: Typically built atop TCP/IP, with SSH using default port 22.
- Version Negotiation: Clients and servers start by exchanging their SSH protocol version strings to establish compatibility.

Success

- Paramiko implements a transport medium built atop TCP/IP with SSH using default port 22 as the standard.

The `Transport` class accepts TCP sockets and builds the SSH protocol on top of them (*transport.py:474-496*). Please note that, in Python, a `socket.SOCK_STREAM` corresponds to a TCP socket.

```
if type(sock) is tuple:
    # connect to the given (host, port)
    hostname, port = sock
    self.hostname = hostname
    reason = "No suitable address family"
    addinfos = socket.getaddrinfo(
        hostname, port, socket.AF_UNSPEC, socket.SOCK_STREAM
    )
    for family, socktype, proto, canonname, sockaddr in addinfos:
        if socktype == socket.SOCK_STREAM:
            af = family
            # addr = sockaddr
            sock = socket.socket(af, socket.SOCK_STREAM)
            try:
                sock.connect((hostname, port))
            except socket.error as e:
                reason = str(e)
            else:
                break
    else:
        raise SSHException(
            "Unable to connect to {}: {}".format(hostname, reason)
        )
```

Paramiko handles non-standard ports correctly, parsing hostname:port combinations and defaulting to port 22 when no port is specified (*transport.py:466-473*).

```
if isinstance(sock, str):
    # convert "host:port" into (host, port)
    hl = sock.split(":", 1)
    self.hostname = hl[0]
    if len(hl) == 1:
        sock = (hl[0], 22)
    else:
```



```
sock = (hl[0], int(hl[1]))
```

- Paramiko implements the SSH protocol version string exchange to establish compatibility between client and server. The version string exchange happens at the very beginning of the SSH connection in the `Transport.run()` method (*transport.py:2174-2179*). The local version string is sent first, followed by reading and validating the remote version.

```
self.packetizer.write_all(b(self.local_version + "\r\n"))
self._log(
    DEBUG,
    "Local version/idstring: {}".format(self.local_version),
) # noqa
self._check_banner()
```

 Python

Paramiko implements the `_check_banner()` method to read and validate the remote SSH protocol version string (*transport.py:2353-2394*)

Warning

At first glance, the socket is enforced to be TCP, but only when the input socket to the `Transport` class is of type `tuple`.

```
if type(sock) is tuple:
    ...
# okay, normal socket-ish flow here...
threading.Thread.__init__(self)
self.daemon = True
self.sock = sock
# we set the timeout so we can check self.active periodically to
# see if we should bail. socket.timeout exception is never propagated.
self.sock.settimeout(self._active_check_timeout)
```

 Python

We tested if a transport layer can be established on top of UDP in Section 11.6.

8.1.2.7. Binary Packet Protocol

RFC

- After version exchange, communication switches to SSH's binary packet format.
- Key Components of Packets:
 - Packet length
 - Padding
 - Flags for encryption and integrity
- Optional compression is negotiated and applied before encryption.

- Protocol includes limits on packet size to guard against malicious or malformed payloads.

Success

Immediately after the version exchange completes, Paramiko switches to binary packet format by starting the handshake and sending the first binary message (*transport.py:2187-2189*).

The `start_handshake()` call signals the transition to binary packet handling, and `_send_kex_init()` sends the first binary SSH message (`MSG_KEXINIT`).

```
self.packetizer.start_handshake(self.handshake_timeout)
self._send_kex_init()
self._expect_packet(MSG_KEXINIT)
```

 Python

Once in binary mode, all subsequent communication uses the `Packetizer.read_message()` and `send_message()` methods for binary packet handling (*transport.py:2194-2195*). The `read_message()` method implements the full SSH binary packet protocol, including (*packet.py:488-590*):

- Packet length parsing from binary headers.
- Encryption/decryption of packet contents.
- MAC verification for message integrity.
- Sequence number tracking.
- Padding validation.

Paramiko implements optional compression negotiation and applies compression before encryption, exactly as specified in the SSH protocol. Compression algorithms are negotiated during the `KEXINIT` exchange alongside other cryptographic parameters (*transport.py:2458-2459*). The `_send_kex_init()` method includes compression preferences in the negotiation message, sending the same compression list for both directions.

```
m.add_list(self.preferred_compression)
m.add_list(self.preferred_compression)
```

 Python

Paramiko defines strict packet size constants in `paramiko/common.py` that enforce protocol limits (*common.py:233-245*). These limits include:

- `DEFAULT_MAX_PACKET_SIZE` = 2^{15} - Standard maximum packet size
- `MIN_PACKET_SIZE` = 2^{12} - Minimum acceptable packet size
- `MAX_WINDOW_SIZE` = $2^{32} - 1$ - Maximum window size per RFC 4254

8.1.2.8. Negotiation and Key Exchange

RFC

- Algorithm Negotiation: Client and server negotiate the following:
 - Key exchange method
 - Host key algorithm
 - Symmetric encryption cipher
 - Message Authentication Code (MAC)
 - Hash function
- Diffie-Hellman Support: Includes `diffie-hellman-group1-sha1` and `diffie-hellman-group14-sha1` as minimal implementations.
- Key Derivation and Session ID:
 - KEX results in two values:
 - Shared secret (K)
 - Exchange hash (H)
 - The first exchange hash H also becomes the session identifier, a persistent unique ID used later in authentication—even if new keys are exchanged.

Success

Paramiko negotiates all five algorithm categories during the KEXINIT exchange in `_send_kex_init()` (*transport.py:2449-2463*). The negotiation process in `_parse_kex_init()` selects compatible algorithms from the preference lists (*transport.py:2567-2583*).

```
m = Message()
m.add_byte(cMSG_KEXINIT)
m.add_bytes(os.urandom(16))
m.add_list(kex_algos)
m.add_list(available_server_keys)
m.add_list(self.preferred_ciphers)
m.add_list(self.preferred_ciphers)
m.add_list(self.preferred_macs)
m.add_list(self.preferred_macs)
m.add_list(self.preferred_compression)
m.add_list(self.preferred_compression)
m.add_string(bytes())
m.add_string(bytes())
m.add_boolean(False)
m.add_int(0)
```

 Python

Paramiko includes both required minimal Diffie-Hellman implementations in its `_kex_info` mapping (*transport.py:330-335*).

```
_kex_info = {
    "diffie-hellman-group1-sha1": KexGroup1,
    "diffie-hellman-group14-sha1": KexGroup14,
```

 Python

The `_set_K_H()` method establishes both values and sets the session ID from the first exchange hash (*transport.py:1986-1993*).

```
def _set_K_H(self, k, h):
    """
    Used by a kex obj to set the K (root key) and H (exchange hash).
    """
    self.K = k
    self.H = h
    if self.session_id is None:
        self.session_id = h
```

 Python

The session ID remains constant even during rekeying operations, as verified in the key derivation process where it is used alongside K and H (*transport.py:2013-2019*). This same pattern is implemented across all key exchange algorithms, including ECDH variants.

```
def _compute_key(self, id, nbytes):
    """id is 'A' - 'F' for the various keys used by ssh"""
    m = Message()
    m.add_mpint(self.K)
    m.add_bytes(self.H)
    m.add_byte(b(id))
    m.add_bytes(self.session_id)
```

 Python

Warning

While Paramiko implements the minimal Diffie-Hellman key exchange groups, these two groups (`KexGroup1` and `KexGroup14`) have been deprecated in RFC 9142.

8.1.2.9. Secure Phase

RFC

- Once keys are negotiated and verified, both sides send **NEWKEYS** messages to signal that subsequent traffic will be encrypted and integrity-protected using the agreed parameters.
- At this point, confidentiality and integrity via encryption and MAC are enabled across the session.

Success

After key negotiation completes, Paramiko sends a **NEWKEYS** message in the `_activate_outbound()` method (*transport.py:2788-2790*).

```
m = Message()
m.add_byte(cMSG_NEWKEYS)
self._send_message(m)
```

 Python

The **NEWKEYS** exchange triggers activation of encryption parameters on both sides:

- Outbound activation: Called when sending **NEWKEYS**, sets up encryption for outgoing traffic
- Inbound activation: Called when receiving **NEWKEYS**, sets up decryption for incoming traffic

The method configures the encryption engine and MAC for outbound traffic (*transport.py:2829-2839*)

```
self.packetizer.set_outbound_cipher(
    block_engine=engine,
    block_size=block_size,
    mac_engine=None if aead else mac_engine,
    mac_size=16 if aead else mac_size,
    mac_key=None if aead else mac_key,
    sdctr=sdctr,
    etm=etm,
    aead=aead,
    iv_out=iv_out if aead else None,
)
```

 Python

8.1.2.10. Forward Secrecy

RFC

SSH has perfect forward secrecy.

Success

Paramiko provides forward secrecy through its implementation of ephemeral key exchange algorithms.

8.1.2.11. Traffic Analysis

❗ RFC

Implementers should use the `SSH_MSG_IGNORE` packet, along with the inclusion of random lengths of padding, to thwart attempts at traffic analysis.

Fail

Paramiko implements the `SSH_MSG_IGNORE` packet functionality with random padding, though it's provided as an optional feature rather than being automatically used for traffic analysis protection.

INFO-11	No traffic analysis protection by default
Perimeter	RFC compliance
Description	
Traffic analysis protection is implemented but is set as optional.	
Recommendation	
RFC 4251 asks for traffic analysis protection by default with the <code>SSH_MSG_IGNORE</code> packet in Section 9.3.9.	

8.1.3. Authentication Layer

8.1.3.1. Weak Transport

❗ RFC

If the transport layer does not provide confidentiality, authentication methods that rely on secret data should be disabled.

Success

Paramiko prevents transmission of secret authentication data over unencrypted connections. The checks occur at the **Transport** layer before any authentication handlers are invoked, ensuring that even if the higher-level code attempts to authenticate prematurely, the secret data will not be transmitted. Protection relies on two key state variables:

- `self.active`: Indicates the transport connection is established and operational
- `self.initial_kex_done`: Indicates that the initial key exchange has been completed and encryption is active

Paramiko explicitly checks for confidentiality before allowing password authentication in the `auth_password()` method (*transport.py:1618-1621*), public key authentication (*transport.py:1691-1693*), or interactive authentication (*transport.py:1748-1750*).

```
if (not self.active) or (not self.initial_kex_done):  
    # we should never try to authenticate unless we're on a secure link  
    raise SSHException("No existing session")
```

 Python

8.1.3.2. Debug Messages

RFC

Debugging messages in SSH may inadvertently disclose confidential server data. Therefore, it is essential to thoughtfully craft and restrict these messages. Debugging should be disabled by default, particularly during user authentication. It is only advisable to enable debugging if an administrator explicitly decides to do so. Developers are advised to minimize sensitive data in such messages and warn administrators of the risks when enabling debugging.

Fail

Paramiko does not implement several key recommendations from the SSH protocol specification:

- No automatic debug disabling during authentication: debug messages are processed normally during authentication phases.
- No administrator warning messages: no warnings are presented when debug logging is enabled.
- No deployment-time debug disabling: debug messages are not disabled by default in a security-conscious manner.
- No active administrator decision requirement: enabling debug logging does not require explicit administrative approval.

INFO-12	Missing debug message control
Perimeter	RFC compliance
Description	
While Paramiko provides the technical infrastructure for controlling debug message logging through its logging system, it does not implement the specific administrative controls and security-conscious defaults recommended by the SSH protocol specification.	
Recommendation	
Paramiko should implement these controls by default or put the different logging methods as abstract to force developers to properly set them up.	

8.1.3.3. Local Security Policy

RFC

The implementer must ensure that the credentials provided validate the professed user and also must ensure that the local policy of the server permits the user the access requested.

Success

Paramiko provides the framework for credential validation through multiple authentication method handlers in the `AuthHandler` class (*auth_handler.py:586-722*).

```

if method == "none":
    ...
elif method == "password":
    ...
elif method == "publickey":
    ...
elif method == "keyboard-interactive":
    ...
elif method == "gssapi-with-mic" and gss_auth:
    ...
elif method == "gssapi-keyex" and gss_auth:
    ...
else:
    ...

```

 Python

Fail

Paramiko provides the base class to define the contract for implementing local policy enforcement, which is set to return always `AUTH_FAILED` (*server.py:125-148*). Server developers need to override this function for proper usage.

```
def check_auth_password(self, username, password):  
    """  
    Determine if a given username and password supplied by the client is  
    acceptable for use in authentication.  
    Return ``AUTH_FAILED`` if the password is not accepted,  
    ``AUTH_SUCCESSFUL`` if the password is accepted and completes  
    the authentication, or ``AUTH_PARTIALLY_SUCCESSFUL`` if your  
    authentication is stateful, and this key is accepted for  
    authentication, but more authentication is required. (In this latter  
    case, `get_allowed_auths` will be called to report to the client what  
    options it has for continuing the authentication.)  
    The default implementation always returns ``AUTH_FAILED``.  
    :param str username: the username of the authenticating client.  
    :param str password: the password given by the client.  
    :return:  
        ``AUTH_FAILED`` if the authentication fails; ``AUTH_SUCCESSFUL`` if  
        it succeeds; ``AUTH_PARTIALLY_SUCCESSFUL`` if the password auth is  
        successful, but authentication must continue.  
    :rtype: int  
    """  
    return AUTH_FAILED
```

INFO-13

No default support for local security policy

Perimeter

RFC compliance

Description

`check_auth_password` method always returns `False` by default and does not configure the server's acceptance of username/password from a client.

Recommendation

`check_auth_password` should be an abstract method for server developers to override for proper usage.

8.1.3.4. Authentication Framework

RFC

- The server orchestrates the process by telling the client which authentication methods are allowed.
- The client may attempt methods in any order of its choosing among those presented.
- The reserved method `"none"` can be sent by the client but is always rejected unless the server intentionally grants access without authentication. It is mainly used to probe supported methods.
- Recommended safeguards include:
 - A timeout (RECOMMENDED: 10 minutes),
 - A limit on failed attempts (RECOMMENDED: 20), beyond which the server should disconnect.

Success

- The server controls which authentication methods are available to clients through the `get_allowed_auths()` method (*server.py:89-104*). By default, this function returns "password".
- In Paramiko's `SSHClient._auth()` method, the client attempts authentication methods in a predefined order, but this is an implementation choice rather than a protocol requirement (*client.py:650-815*).

```
def _auth(self, username, password, pkey, key_filenames,
          allow_agent, look_for_keys, gss_auth, gss_kex, gss_deleg_creds,
          gss_host, passphrase):
    """
    Try, in order:

    - The key(s) passed in, if one was passed in.
    - Any key we can find through an SSH agent (if allowed).
    - Any id_* key discoverable in ~/.ssh/ (if allowed).
    - Plain username/password auth, if a password was given.

    (The password might be needed to unlock a private key [if 'passphrase'
    isn't also given], or for two-factor authentication [for which it is
    required].)
    """
    ...
    # If GSS-API support and GSS-API Key Exchange was performed, we attempt
    # authentication with gssapi-keyex.
    if gss_kex and self._transport.gss_kex_used:
```

```

    ...
    if gss_auth:
        ...
    if pkey is not None:
        ...
    if not two_factor:
        ...
    if not two_factor and allow_agent:
        ...
    if not two_factor:
        ...
    if password is not None:
        ...
    elif two_factor:
        ...
    ...
    raise SSHException("No authentication methods available")

```

- Paramiko provides the `auth_none()` method in the `Transport` class that implements the “none” authentication method (*transport.py:1543-1569*).

```

def auth_none(self, username):
    """
    Try to authenticate to the server using no authentication at all.
    This will almost always fail. It may be useful for determining the
    list of authentication types supported by the server, by catching the
    `BadAuthenticationType` exception raised.
    :param str username: the username to authenticate as
    :return:
        list of auth types permissible for the next stage of
        authentication (normally empty)
    :raises:
        `BadAuthenticationType` -- if "none" authentication isn't allowed
        by the server for this user
    :raises:
        `SSHException` -- if the authentication failed due to a network
        error
    .. versionadded:: 1.5
    """

```

On the server side, default implementation always returns `AUTH_FAILED` in `check_auth_none()` (*server.py:106-123*).

- Paramiko implements authentication timeouts (default set to 30 seconds instead of 10 minutes) in the `Transport` class (*transport.py:568*).

```
if auth_timeout is not None:
    t.auth_timeout = auth_timeout
```

 Python

Failed attempts are tracked via the `auth_fail_count` attribute of the `AuthHandler` class and a limit on the number of attempts is checked (≥ 10).

```
if self.auth_fail_count >= 10:
    self._disconnect_no_more_auth()
```

 Python

8.1.3.5. Authentication Requests

RFC

Every request includes the user name and service name. If either changes mid-session, the server must reset the authentication state.

Success

The server does enforce that username and service name remain consistent throughout the authentication process and resets authentication state if they change. Every authentication request in Paramiko includes both username and service name as required by the SSH protocol (*auth_handler.py:561-563*). The `_parse_userauth_request()` method extracts these fields from every incoming authentication message.

```
username = m.get_text()
service = m.get_text()
method = m.get_text()
```

 Python

Paramiko explicitly checks for username changes mid-session and resets authentication state when detected (*auth_handler.py:573-582*). When a username change is detected (`self.auth_username != username`), Paramiko:

- Logs a warning about the attempted username change
- Calls `_disconnect_no_more_auth()` to reset authentication state and disconnect
- Prevents further authentication attempts

```
if (self.auth_username is not None) and (
    self.auth_username != username
):
    self._log(
        WARNING,
```

 Python

```

        "Auth rejected because the client attempted to change username in mid-
        flight", # noqa
    )
    self._disconnect_no_more_auth()
    return
self.auth_username = username

```

8.1.3.6. Message Numbering

RFC

- SSH authentication messages use numbers between 50 and 79.
- Messages numbered 80 and above are invalid before authentication is complete and must trigger a disconnect.

Success

Paramiko defines authentication messages in the correct range (*common.py:57-72*). The authentication messages are properly defined between 50-79:

- MSG_USERAUTH_REQUEST through MSG_USERAUTH_BANNER use the range 50-54
- MSG_USERAUTH_PK_OK uses 60
- Additional GSSAPI authentication messages use the range 60-67
- HIGHEST_USERAUTH_MESSAGE_ID is explicitly set to 79

```

(
MSG_USERAUTH_REQUEST,
MSG_USERAUTH_FAILURE,
MSG_USERAUTH_SUCCESS,
MSG_USERAUTH_BANNER,
) = range(50, 54)
MSG_USERAUTH_PK_OK = 60
(MSG_USERAUTH_INFO_REQUEST, MSG_USERAUTH_INFO_RESPONSE) = range(60, 62)
(MSG_USERAUTH_GSSAPI_RESPONSE, MSG_USERAUTH_GSSAPI_TOKEN) = range(60, 62)
(
    MSG_USERAUTH_GSSAPI_EXCHANGE_COMPLETE,
    MSG_USERAUTH_GSSAPI_ERROR,
    MSG_USERAUTH_GSSAPI_ERRTOK,
    MSG_USERAUTH_GSSAPI_MIC,
) = range(63, 67)
HIGHEST_USERAUTH_MESSAGE_ID = 79

```

Global requests and channel operations start at 80. The authentication state enforcement mechanism is in the `_ensure_authed()` method (*transport.py:2103-2139*). This method checks if:

- The transport is in server mode
- The message type is above `HIGHEST_USERAUTH_MESSAGE_ID` (79)
- Authentication has not yet completed

If these conditions are met, it generates appropriate error responses that cause disconnection.

```
def _ensure_authed(self, ptype, message):
    """
    ...
    """
    if (
        not self.server_mode
        or ptype <= HIGHEST_USERAUTH_MESSAGE_ID
        or self.is_authenticated()
    ):
        return None
    ...
def run(self):
    ...
    if ptype in self._handler_table:
        error_msg = self._ensure_authed(ptype, m)
        if error_msg:
            self._send_message(error_msg)
        else:
            self._handler_table[ptype](m)
    ...
```

8.1.3.7. Authentication Methods

Public-Key authentication

RFC

- Mandatory support in all implementations.
- The client may first check if a key is acceptable by sending a request without a signature (`flag = FALSE`). The server responds with either:
 - `SSH_MSG_USERAUTH_FAILURE`, or
 - `SSH_MSG_USERAUTH_PK_OK`, indicating that the key is acceptable.

- If accepted, the client sends a signed request (flag = TRUE) including the signature over specific session and protocol data. The server verifies the signature and accepts or rejects accordingly.

Success

- Paramiko provides mandatory public-key authentication support through the `auth_publickey()` method in the `Transport` class (*transport.py:1656-1703*) and the corresponding server-side handling in `AuthHandler._parse_userauth_request()` (*auth_handler.py:612-652*).

Paramiko implements the exact two-phase process described in the SSH protocol:

- Phase 1 (client checks if a key is acceptable): When the server receives a public key request without a signature (`sig_attached = False`), it first validates the key through `check_auth_publickey()` and responds with `SSH_MSG_USERAUTH_PK_OK` if acceptable (*auth_handler.py:637-645*).

```
if not sig_attached:
    # client wants to know if this key is acceptable, before it
    # signs anything... send special "ok" message
    m = Message()
    m.add_byte(cMSG_USERAUTH_PK_OK)
    m.add_string(algorithm)
    m.add_string(keyblob)
    self.transport._send_message(m)
    return

sig = Message(m.get_binary())
blob = self._get_session_blob(
    key, service, username, algorithm
)
if not key.verify_ssh_sig(blob, sig):
    self._log(INFO, "Auth rejected: invalid signature")
    result = AUTH_FAILED
```

- Phase 2 (client sends a signed request): On the client side, Paramiko's `auth_publickey()` method in `AuthHandler` constructs the proper authentication request with the signature flag and session-specific data (*auth_handler.py:392-405*). The signature is created over session-specific data including the session ID, username, service name, and key algorithm.

```
elif self.auth_method == "publickey":
    m.add_boolean(True)
    key_type, bits = self._get_key_type_and_bits(self.private_key)
```

```

algorithm = self._finalize_pubkey_algorithm(key_type)
m.add_string(algorithm)
m.add_string(bits)
blob = self._get_session_blob(
    self.private_key,
    "ssh-connection",
    self.username,
    algorithm,
)
sig = self.private_key.sign_ssh_data(blob, algorithm)
m.add_string(sig)

```

Password Authentication

RFC

The client sends the password (encoded in UTF-8). The server checks it per its policies and either accepts or rejects.

Success

Paramiko handles password authentication on the server side in the `_parse_userauth_request()` method (*auth_handler.py:588-611*). The implementation:

- UTF-8 Decoding: Extracts the password as binary data and attempts UTF-8 decoding (*auth_handler.py:590-596*).
- Non-UTF-8 Fallback: If UTF-8 decoding fails, it falls back to using the raw byte string for compatibility with some clients/servers.
- Policy Validation: Calls the server's `check_auth_password()` method to validate credentials against local policies.

```

elif method == "password":
    changereq = m.get_boolean()
    password = m.get_binary()
    try:
        password = password.decode("UTF-8")
    except UnicodeError:
        # some clients/servers expect non-utf-8 passwords!
        # in this case, just return the raw byte string.
        pass
    if changereq:
        # always treated as failure, since we don't support changing
        # passwords, but collect the list of valid auth types from

```



```

# the callback anyway
self._log(DEBUG, "Auth request to change passwords (rejected)")
newpassword = m.get_binary()
try:
    newpassword = newpassword.decode("UTF-8", "replace")
except UnicodeError:
    pass
result = AUTH_FAILED
else:
    result = self.transport.server_object.check_auth_password(
        username, password
    )

```

Host-based Authentication

RFC

- The client host (not just the user) authenticates itself via its host key and a signature including:
 - Session identifier,
 - User and service names,
 - Host key,
 - Client host name (FQDN),
 - Client-side user name.
- The server must verify that:
 - The host key belongs to the claimed client,
 - The user on that host is allowed,
 - The signature is valid.

Paramiko does not implement host-based authentication.

```

if method == "none":
    ...
elif method == "password":
    ...
elif method == "publickey":
    ...
elif method == "keyboard-interactive":
    ...
elif method == "gssapi-with-mic" and gss_auth:
    ...
elif method == "gssapi-keyex" and gss_auth:
    ...
else:

```

8.1.4. Connection Layer

8.1.4.1. End Point Security

① RFC

Implementers should provide mechanisms for administrators to control which services are exposed to limit the vulnerability of other services. These controls might include controlling which machines and ports can be targeted in port-forwarding operations, which users are allowed to use interactive shell facilities, or which users are allowed to use exposed subsystems.

Success

Paramiko does provide comprehensive mechanisms for administrators to control service exposure through the `ServerInterface` class, which allows fine-grained control over different areas:

- Port Forwarding Control: `check_channel_direct_tcpip_request()` (*server.py:526-564*).
- PTY requests: `check_channel_pty_request()` (*server.py:376-397*).
- Shell request: `check_channel_shell_request()` (*server.py:399-413*).
- Subsystem requests: `check_channel_subsystem_request()` (*server.py:434-462*).
- Window changes: `check_channel_window_change_request()` (*server.py:464-482*).
- X11 forwarding: `check_channel_x11_request()` (*server.py:484-508*).
- Agent forwarding: `check_channel_forward_agent_request()` (*server.py:510-524*).

Please note however that all these functions need to be overridden by administrators.

8.1.4.2. Channel Opening

① RFC

Clients open new channels by sending a channel open request specifying desired type (e.g. “session”, “direct-tcpip”) and initial window size. The server responds by accepting (with a channel number) or rejecting.

Success

Paramiko implements channel opening through the `open_channel()` method in the `Transport` class (*transport.py:1037-1045*). This method:

- Specifies channel type: The `kind` parameter defines the channel type (e.g. “session”, “direct-tcpip”, “forwarded-tcpip”)
- Sets initial window size: Uses `window_size` parameter or defaults to `self.default_window_size`
- Sends `MSG_CHANNEL_OPEN`: Constructs and sends the channel open request message

The message construction includes all required fields (*transport.py:1089-1094*):

- Channel type string
- Local channel ID
- Initial window size
- Maximum packet size

```
m = Message()
m.add_byte(cMSG_CHANNEL_OPEN)
m.add_string(kind)
m.add_int(chanid)
m.add_int(window_size)
m.add_int(max_packet_size)
```

 Python

On the server side, Paramiko handles incoming channel open requests in `_parse_channel_open()` (*transport.py:3019-3143*).

8.1.4.3. Data Transfer

RFC

Once open, channels carry bidirectional data using `CHANNEL_DATA` messages. Each channel uses flow control with window sizes—data must wait until window space is available.

Success

Paramiko implements bidirectional data transfer through the `send()` and `recv()` methods in the `Channel` class (*channel.py:781-799/683-710*). The `send()` method constructs `CHANNEL_DATA` messages for outbound data transfer.

```
m = Message()
m.add_byte(cMSG_CHANNEL_DATA)
m.add_int(self.remote_chanid)
```

 Python

```
return self._send(s, m)
```

The `recv()` method processes incoming data from the remote side.

```
try:
    out = self.in_buffer.read(nbytes, self.timeout)
except PipeTimeout:
    raise socket.timeout()
```

Python

Paramiko maintains separate window sizes for inbound and outbound data flow (*channel.py:120-125*).

```
self.in_window_size = 0
self.out_window_size = 0
self.in_max_packet_size = 0
self.out_max_packet_size = 0
self.in_window_threshold = 0
self.in_window_sofar = 0
```

Python

8.1.4.4. Channel Closing

❗ RFC

A channel may be closed by sending a `CHANNEL_CLOSE` message. Following that, both sides discard channel states.

Success

When a channel is closed, Paramiko sends a `CHANNEL_CLOSE` message through the `_close_internal()` method (*channel.py:1249-1260*). This method constructs a `CHANNEL_CLOSE` message with the remote channel ID and calls `_set_closed()` to discard the local channel state.

```
def _close_internal(self):
    # you are holding the lock.
    if not self.active or self.closed:
        return None, None
    m1 = self._send_eof()
    m2 = Message()
    m2.add_byte(cMSG_CHANNEL_CLOSE)
    m2.add_int(self.remote_chanid)
    self._set_closed()
    # can't unlink from the Transport yet -- the remote side may still
    # try to send meta-data (exit-status, etc)
```

Python

```
return m1, m2
```

When receiving a `CHANNEL_CLOSE` message from the remote side, Paramiko handles it through the `_handle_close()` method (*channel.py:1176-1185*).

```
def _handle_close(self, m):  
    self.lock.acquire()  
    try:  
        msgs = self._close_internal()  
        self.transport._unlink_channel(self.chanid)  
    finally:  
        self.lock.release()  
    for m in msgs:  
        if m is not None:  
            self.transport._send_user_message(m)
```

Python

The `_set_closed()` method implements the state discarding requirement (*channel.py:1226-1236*).

```
def _set_closed(self):  
    # you are holding the lock.  
    self.closed = True  
    self.in_buffer.close()  
    self.in_stderr_buffer.close()  
    self.out_buffer_cv.notify_all()  
    # Notify any waiters that we are closed  
    self.event.set()  
    self.status_event.set()  
    if self._pipe is not None:  
        self._pipe.set_forever()
```

Python

8.2. Summary of RFC compliance

Generally speaking, Paramiko adheres to the assorted RFCs that establish the SSH protocol. The primary cause of problems is that Paramiko delegates some functions to the application level. Indeed, the library employs a philosophy of ‘offer tools, let applications decide policy’, leaving it to applications and system administrators to implement necessary security measures, such as debug message management.

While this is not a direct problem, we think that using abstract methods to define key functions would clearly show system administrators the importance of these functions, and that their implementation is mandatory, improving overall security.

9. Static analysis

9.1. Cryptographic primitives used in Paramiko

During the static analysis phase, we examined the usage of cryptographic primitives by Paramiko, particularly calls to standard and third-party libraries such as `cryptography`, `hashlib`, `bcrypt`, and `pynacl`. This section details the files that directly make use of these libraries. For each instance, we report our discoveries or lack thereof.

① Deprecation of SHA-1 and MD5

We found uses of the deprecated cryptographic primitives SHA-1 and MD5 throughout the code base. MD5 is not part of the hash algorithms approved by the NIST (see [FIPS 180-4](#), [FIPS 202](#), and [SP 800-185](#)) and practical collision attacks have been published (see, for example, [RFC 6151](#)). While SHA-1 is part of FIPS 180-4, the NIST [SP 800-131A Rev. 2](#) deprecates it, disallowing its use for generating digital signatures, with NIST indicating that SHA-1 should be phased out by 2030 for all other uses.

Per the RFCs describing the SSH protocol and extensions, some of these uses are expected to provide full compatibility with existing clients. As such, we noted all uses of these primitives, analyzing whether their use represents an immediate vulnerability, in which case we added a finding.

9.1.1. `config.py`

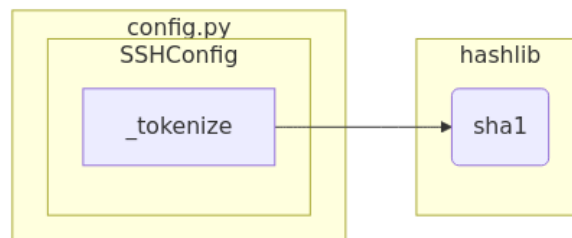


Figure 4: Use of `hashlib` in `config.py`.

`config.py` provides functionality to handle the OpenSSH client configuration file `ssh_config`. The `SSHConfig` class represents the configuration information as stored in the OpenSSH format.

- `hashlib` (Figure 4) is used to hash some configuration tokens with SHA-1 to compute the `%C` token in the `_tokenize` method.

Use of SHA-1

In `config.py`, line 449, `SSHConfig._tokenize()` uses SHA-1 to hash configuration values. The digest is used as a unique identifier for a shared connection through the `ControlPath` option, so using SHA-1 is acceptable and matches the behavior of OpenSSH.

INFO-14**%C** token in SSH config does not match OpenSSH behavior**Perimeter**

paramiko/config.py.

Description

According to `ssh_config(5)`, the `%C` token is defined as follows:

<code>%C</code>	Hash of <code>%l%h%p%r%j</code> .	 Bash
<code>%h</code>	The remote hostname.	
<code>%j</code>	The contents of the ProxyJump option, or the empty string if this option is unset.	
<code>%l</code>	The local hostname, including the domain name.	
<code>%p</code>	The remote port.	
<code>%r</code>	The remote username.	

In Paramiko, the substitution is done by the `_tokenize` method of `SSHConfig` from `paramiko/config.py`:

```
442     local_hostname = socket.gethostname().split(".")[0]
443     local_fqdn = LazyFqdn(config, local_hostname)
444     homedir = os.path.expanduser("~")
445     tohash = local_hostname + target_hostname + repr(port) + remoteuser
446     # The actual tokens!
447     replacements = {
448         # TODO: %???
449         "%C": sha1(tohash.encode()).hexdigest(),
```

The first part in `%C` should be `%l`, the local host name, *including* the domain name. Paramiko strips the domain name from `local_hostname` by taking only the host name with `.split(".")[0]`.

Second, `tohash` is missing the ProxyJump option corresponding to the `%j` token, even though Paramiko does parse the ProxyJump field.

Recommendation

Use the full host name for the `%l` token, and add the `%j` token, at least to include it in `%C`.

9.1.2. `ecdsa.py`

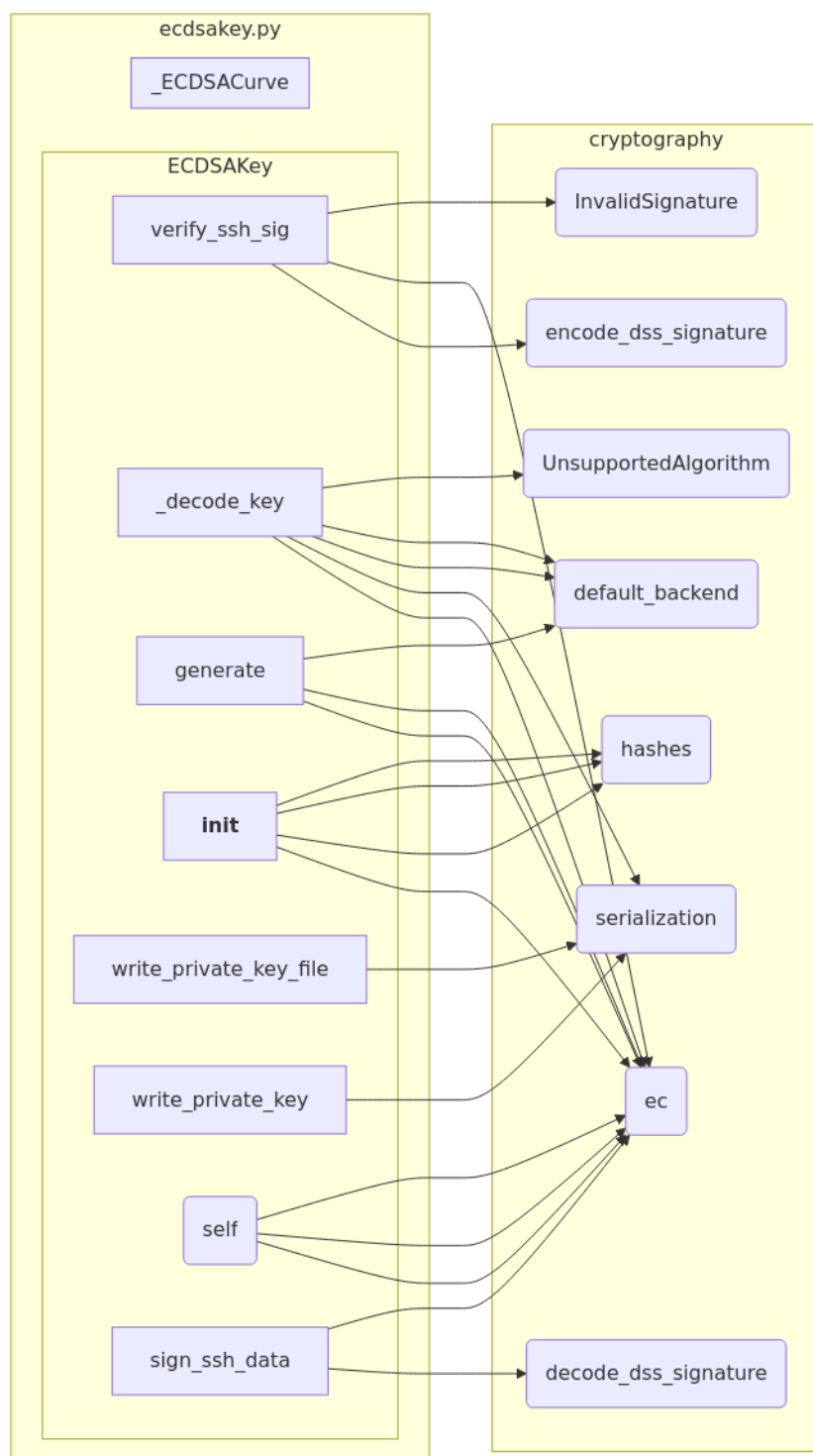


Figure 5: Use of `cryptography` in `ecdsa.py`.

`ecdsa.py` implements support for ECDSA with the `ECDSAKey` class, which is based on `PKey`.

- `cryptography` (Figure 5) is used for all cryptographic operations, including generating, storing, and loading ECDSA key pairs, as well as signing and verifying signatures.

Success

No issues were found.

9.1.3. `ed25519key.py`

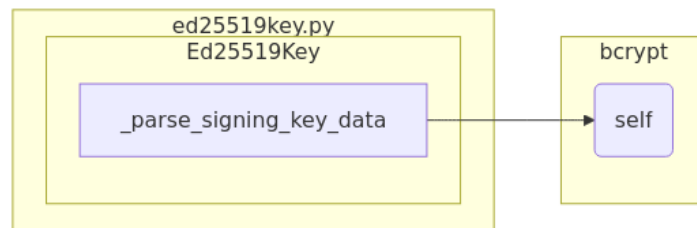


Figure 6: Use of `bcrypt` in `ed25519key.py`.

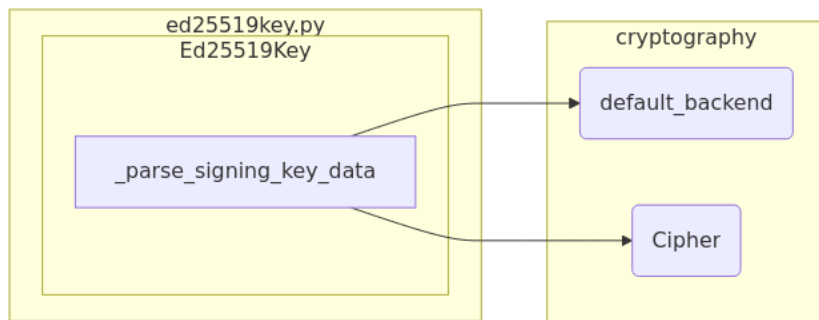


Figure 7: Use of `cryptography` in `ed25519key.py`.

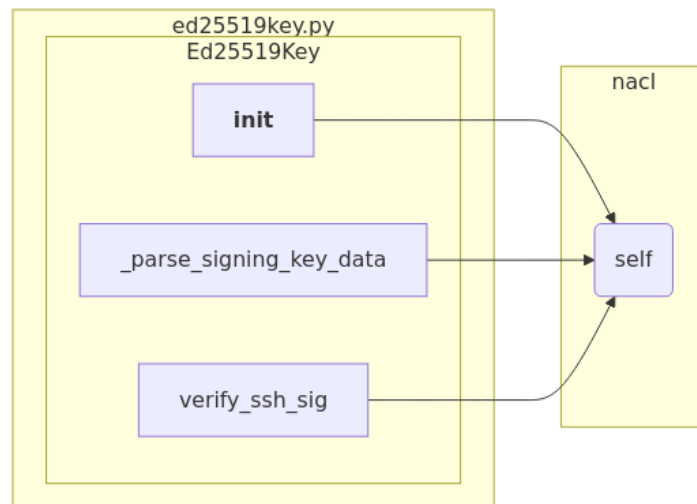


Figure 8: Use of `nacl` in `ed25519key.py`.

`ed25519` implements support for Ed25519 with the `Ed25519Key` class, which is based on `PKey`.

- `nacl` (Figure 8) is used for the core Ed25519 operations (key generation, signing, verifying).
- `cryptography` (Figure 7) and `bcrypt` (Figure 6) are used to decrypt private key files in OpenSSH format.

Success

No issues were found.

9.1.4. `hostkeys.py`

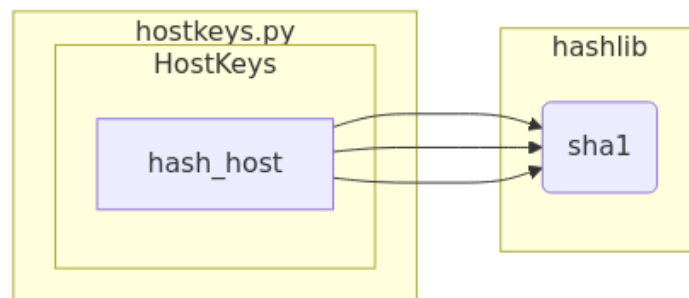


Figure 9: Use of `hashlib` in `hostkeys.py`.

`hostkeys` implements support for OpenSSH-style `known_hosts` file with the `HostKeys` class.

- `hashlib` (Figure 9) is used to hash host names with HMAC-SHA-1.

Use of SHA-1

In `hostkeys.py`, SHA-1 is used thrice in the `hash_host` method of `HostKeys`:

- to get the SHA-1 digest size (20 bytes) to determine the size of a random salt;
- to assert that the random salt is actually the same size as a SHA-1 digest;
- and to “hash” a host name with HMAC-SHA-1. Using SHA-1 with HMAC is acceptable and aligns with the behavior of OpenSSH.

```
284 @staticmethod
285 def hash_host(hostname, salt=None):
286     """
287     Return a "hashed" form of the hostname, as used by OpenSSH when
288     storing
289     hashed hostnames in the known_hosts file.
290     :param str hostname: the hostname to hash
291     :param str salt: optional salt to use when hashing
292                     (must be 20 bytes long)
293     :return: the hashed hostname as a `str`
294     """
295     if salt is None:
296         salt = os.urandom(shal().digest_size)
297     else:
298         if salt.startswith("|1|"):
299             salt = salt.split("|")[2]
300             salt = decodebytes(b(salt))
301         assert len(salt) == shal().digest_size
302         hmac = HMAC(salt, b(hostname), shal().digest())
303         hostkey = "|1|{}|{}".format(u(encodebytes(salt)),
304                                   u(encodebytes(hmac)))
304         return hostkey.replace("\n", "")
```

9.1.5. kex_curve25519.py

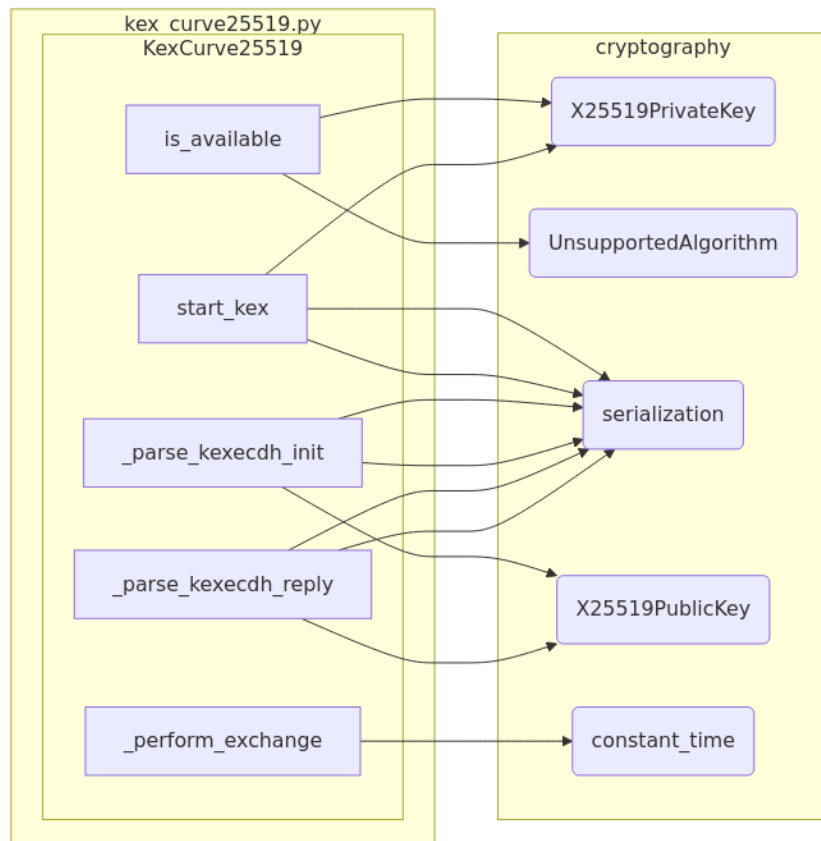


Figure 10: Use of `cryptography` in `kex_curve25519.py`.

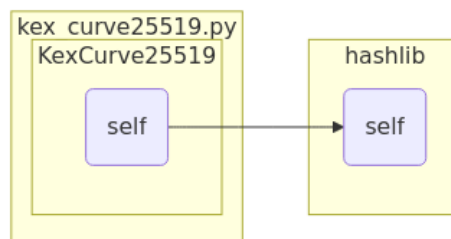


Figure 11: Use of `hashlib` in `kex_curve25519.py`.

`kex_curve25519` implements the X25519 key exchange with the `KexCurve25519` class.

- `cryptography` (Figure 10) is used to perform the key exchange with `X25519PrivateKey` and `X25519PublicKey`. Constant-time comparison verifies that the resulting shared secret is non-null.
- `hashlib.sha256` (Figure 11) is used as an hash algorithm for the exchange.

Success

No issues were found.

9.1.6. kex_ecdh_nist.py

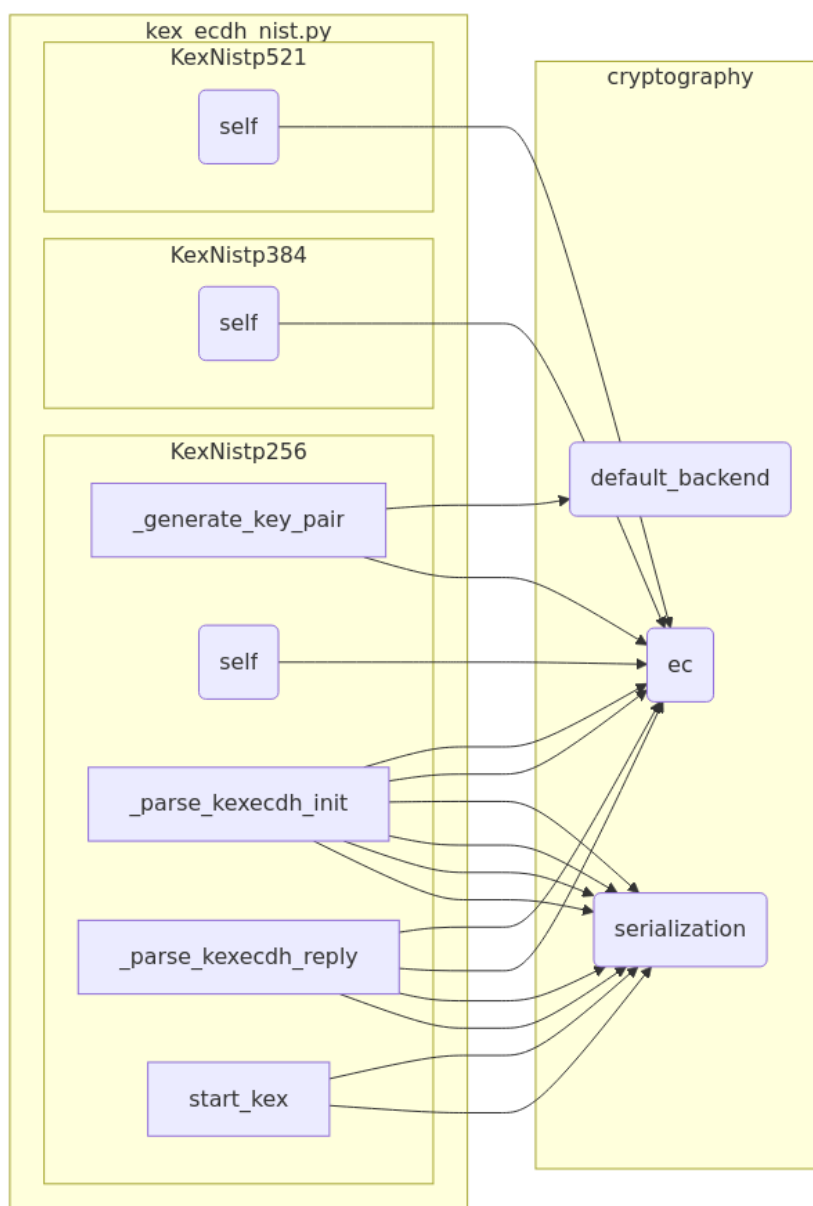


Figure 12: Use of cryptography in kex_ecdh_nist.py.

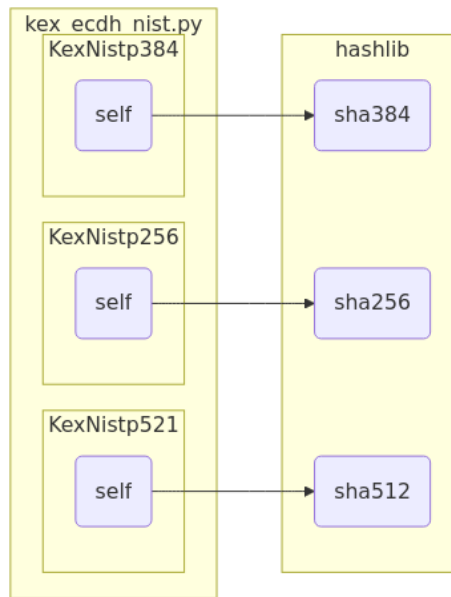


Figure 13: Use of `hashlib` in `kex_ecdh_nist.py`.

`kex_ecdh_nist` implements the ECDH key exchange using NIST P-curves.

- `cryptography` is used for the ECDH operations (key pair generation, public key loading, and the exchange itself) (Figure 12).
- Each of the three NIST curves supported (`nistp256`, `nistp384`, and `nistp521`) is associated with a hash algorithm from `hashlib` (Figure 13).

Success

No issues were found.

9.1.7. `kex_gex.py`

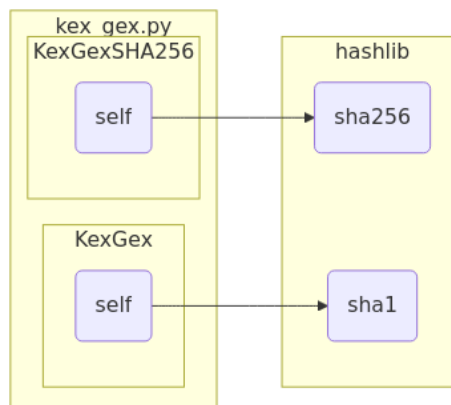


Figure 14: Use of `hashlib` in `kex_gex.py`.

`kex_gex` implements the Diffie-Hellman group exchange with the `KexGex` and `KexGexSHA256` classes.

- `hashlib` is used for SHA-1 and SHA-256 in `KexGex` and `KexGexSHA256` respectively (Figure 14).

MEDIUM-15		Deprecated group exchange method	
Likelihood		Impact	
Perimeter	paramiko/kex_gex.py		
Description			
The <code>KexGex</code> and <code>KexGexSHA256</code> classes implement the <code>diffie-hellman-group-exchange-sha1</code> and <code>diffie-hellman-group-exchange-sha256</code> key exchanges respectively, which were first described in RFC 4419. RFC 9142 now recommends that <code>diffie-hellman-group-exchange-sha1</code> should not be implemented as it uses the deprecated SHA-1 algorithm. <code>diffie-hellman-group-exchange-sha256</code> may be implemented instead. For reference, OpenSSH disabled the <code>diffie-hellman-group*</code> and <code>diffie-hellman-group-exchange-*</code> methods by default in version 10.0.			
Recommendation			
Remove support for <code>diffie-hellman-group-exchange-sha1</code> .			

MEDIUM-16		Insecure minimum modulus size in Diffie-Hellman group exchange			
Likelihood			Impact		
Perimeter	paramiko/kex_gex.py				
Description					
According to RFC 9142, the modulus size of the MODP group used needs to be at least 2048 bits, increasing the minimum of 1024 bits from RFC 4419. Both KexGex and KexGexSHA256 have a minimum of 1024 bits:					
<pre>51 class KexGex: 52 53 name = "diffie-hellman-group-exchange-sha1" 54 min_bits = 1024 55 max_bits = 8192 56 preferred_bits = 2048 57 hash_algo = sha1 58 ... 285 286 class KexGexSHA256(KexGex): 287 name = "diffie-hellman-group-exchange-sha256" 288 hash_algo = sha256</pre> Python					
OpenSSH increased the minimum modulus size supported for diffie-hellman-group-exchange to 2048 bits in version 7.2.					
Recommendation					
Increase the minimum modulus size to 2048 bits.					

9.1.8. kex_group1.py

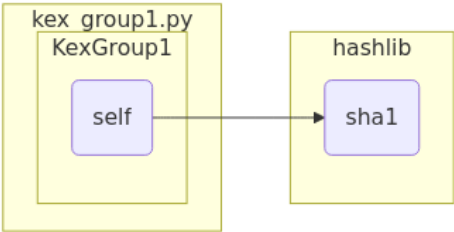


Figure 15: Use of hashlib in kex_group1.py.

`kex_group1` implements the `diffie-hellman-group1-sha1` key exchange with the `KexGroup1` class.

- The SHA-1 implementation of `hashlib` is used as the hash algorithm (Figure 15).

MEDIUM-17	Deprecated Diffie-Hellman group			
Likelihood			Impact	
Perimeter	paramiko/kex_group1.py.			
Description				
RFC 9142 changes the guidance on implementing the <code>diffie-hellman-group1-sha1</code> exchange from <i>MUST</i> to <i>SHOULD NOT</i>. OpenSSH disabled the <code>diffie-hellman-group1-sha1</code> key exchange by default in version 7.0.				
Recommendation				
Remove support for <code>diffie-hellman-group1-sha1</code> .				

9.1.9. `kex_group14.py`

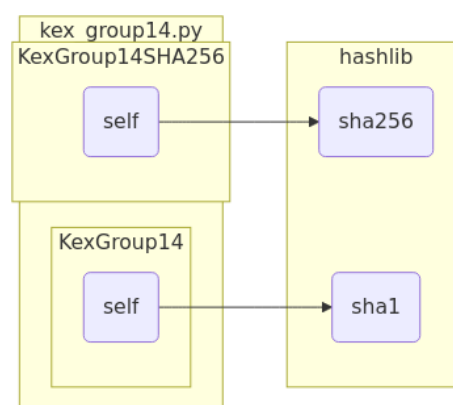


Figure 16: Use of `hashlib` in `kex_group14.py`.

`kex_group14` implements the `diffie-hellman-group14-*` key exchange methods with `KexGroup14` and `KexGroup14SHA256`, based on `KexGroup1`.

- `hashlib` is used for SHA-1 and SHA-256 in `KexGroup14` and `KexGroup14SHA256` respectively (Figure 16).

Use of SHA-1

RFC 9142 removed `diffie-hellman-group14-sha1` from *mandatory to implement* methods due to its use of SHA-1 and the fact that it shares the same group as `diffie-hellman-group14-sha256`, which must now be implemented. We recommend deprecating the `-sha1` method.

9.1.10. `kex_group16.py`

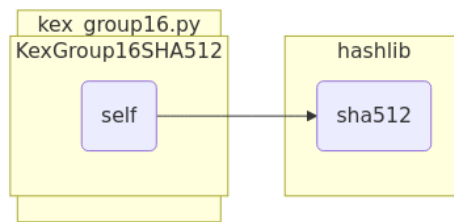


Figure 17: Use of `hashlib` in `kex_group16.py`.

`kex_group16` implements the `diffie-hellman-group16-sha512` key exchange method with `KexGroup16SHA512` class.

- `hashlib` is used for SHA-512 (Figure 17).

Success

No issues were found.

9.1.11. `kex_gss.py`

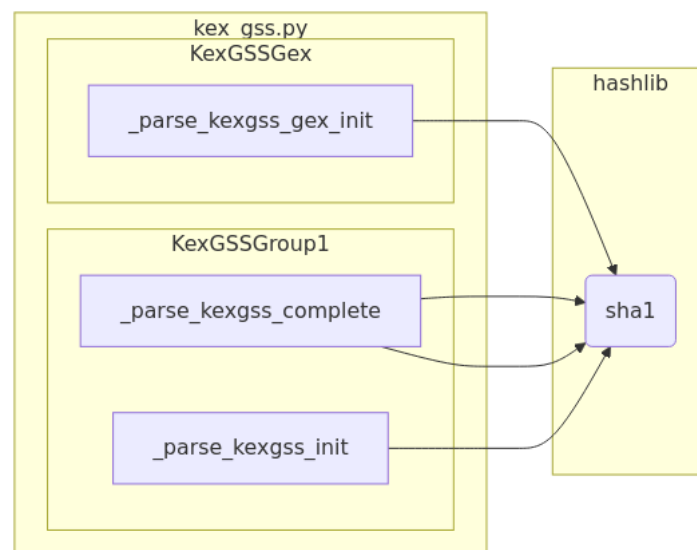


Figure 18: Use of `hashlib` in `kex_gss.py`.

`kex_gss` implements the GSS-API Authenticated Diffie-Hellman key exchange methods defined in [RFC 4462](#).

- `hashlib` is used for SHA-1 in all key exchange methods implemented (Figure 18).

MEDIUM-18	Deprecated GSS-API key exchange methods		
Likelihood		Impact	
Perimeter	paramiko/kex_gss.py.		
Description			
Paramiko implements the three methods defined in RFC 4462: <code>gss-group1-sha1-*</code> , <code>gss-group14-sha1-*</code> , and <code>gss-gex-sha1-*</code> . They have been deprecated in RFC 8732 in favor of methods using SHA-256 and SHA-512, as well as elliptic curve-based methods.			
Recommendation			
Remove the deprecated key exchange methods, replacing them with RFC 8732 additions.			

9.1.12. `pkey.py`

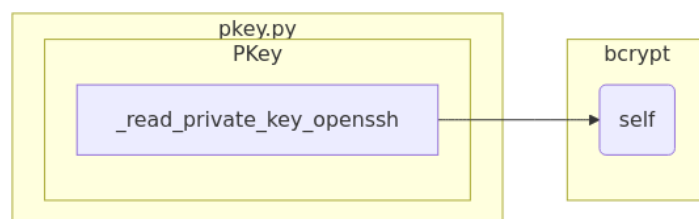


Figure 19: Use of `bcrypt` in `pkey.py`.

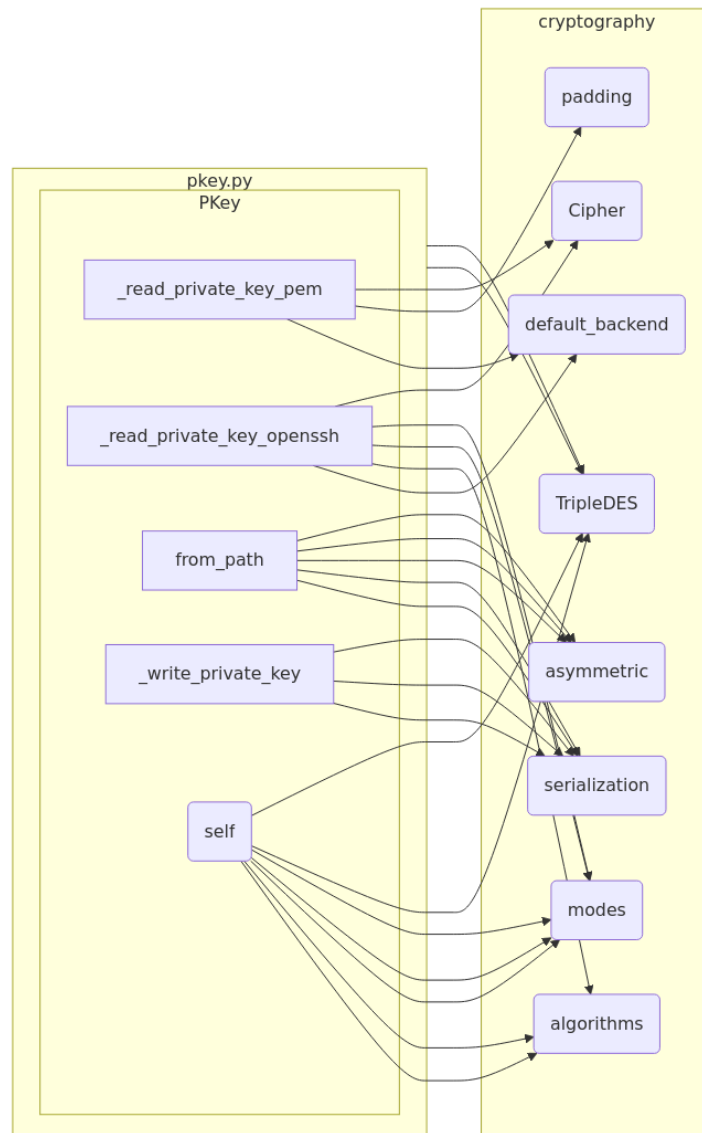


Figure 20: Use of `cryptography` in `pkey.py`.

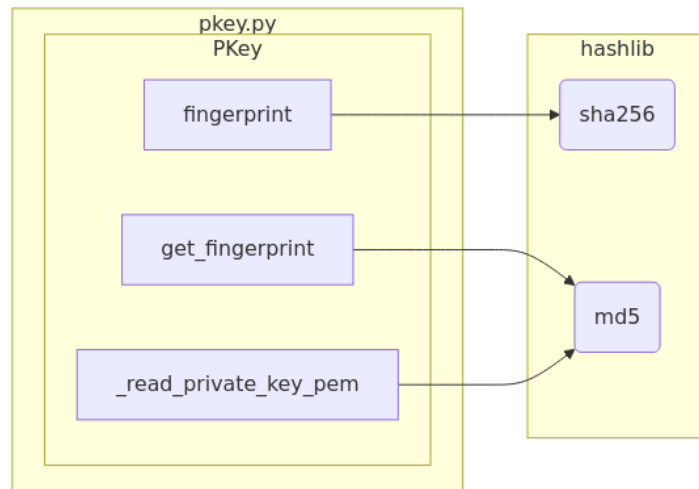


Figure 21: Use of hashlib in `pkey.py`.

`pkey` defines the `PKey` class, which serves as a base class for `*Key` classes in Paramiko. It has generic methods for storing and loading private key files in different formats (DER, PEM, etc.), including password-protected files.

- `bcrypt` is used as a Key Derivation Function to compute the decryption key for private keys in OpenSSH format (Figure 19).
- `cryptography` is used for reading and writing private key files, including password-protected ones. (Figure 20).
- `hashlib` is used for computing public key fingerprints with SHA-256 and MD5, as well as deriving decryption keys with MD5 for PEM private key files (Figure 21).

Use of MD5

In `pkey.py`, MD5 is used to compute the fingerprint of a public key when using the `get_fingerprint()` method of `PKey`. This usage relies on the second preimage resistance of MD5, which is not broken yet. Nevertheless, OpenSSH uses SHA-256 as the default hash algorithm for key fingerprints since version 6.8, released in 2015, which is the behavior of the `fingerprint` property of `PKey`. We recommend deprecating the `get_fingerprint()` method in favor of `fingerprint`.

LOW-19		Use of MD5 as a Key Derivation Function	
Likelihood		Impact	
Perimeter	paramiko/pkey.py		
Description			
Paramiko supports loading private keys from password-encrypted PEM files. It uses the <code>util.generate_key_bytes()</code> function to derive the decryption key from the password: <pre>142 def generate_key_bytes(hash_alg, salt, key, nbytes): 143 """ 144 ... 145 """ 155 """ 156 keydata = bytes() 157 digest = bytes() 158 if len(salt) > 8: 159 salt = salt[:8] 160 while nbytes > 0: 161 hash_obj = hash_alg() 162 if len(digest) > 0: 163 hash_obj.update(digest) 164 hash_obj.update(b(key)) 165 hash_obj.update(salt) 166 digest = hash_obj.digest() 167 size = min(nbytes, len(digest)) 168 keydata += digest[:size] 169 nbytes -= size 170 return keydata</pre>			
For PEM files, this function is called with MD5 as the hash algorithm. While it matches the behavior of OpenSSH, this custom KDF is not standard, nor recommended: the release notes for version 8.1 of OpenSSH mention that PKCS8 files are better when interoperability with non-OpenSSH software is required “as it may use a less insecure key derivation function than PEM’s”.			
Recommendation			
Warn when using this format, recommend the user to save their keys in PKCS8 or OpenSSH format instead.			

INFO-20	Invalid class method <code>from_private_key_file</code> for <code>AgentKey</code>
Perimeter	<code>paramiko/agent.py</code> , <code>paramiko/pkey.py</code>
Description	
The description of the <code>from_private_key_file()</code> method indicates that it exists on all subclasses of <code>PKey</code>. It creates a new instance of the (sub)class by calling <code>cls</code>: <pre> 434 @classmethod 435 def from_private_key(cls, file_obj, password=None): ... 452 key = cls(file_obj=file_obj, password=password) 453 return key </pre> <code>AgentKey</code> inherits from <code>PKey</code> but does not have <code>file_obj</code> and <code>password</code> arguments: the class method is available but leads to a <code>TypeError</code>.	
Recommendation	
Override <code>from_private_key_file()</code> in <code>AgentKey</code> to return an error indicating that this method is not supported.	

9.1.13. rsakey.py

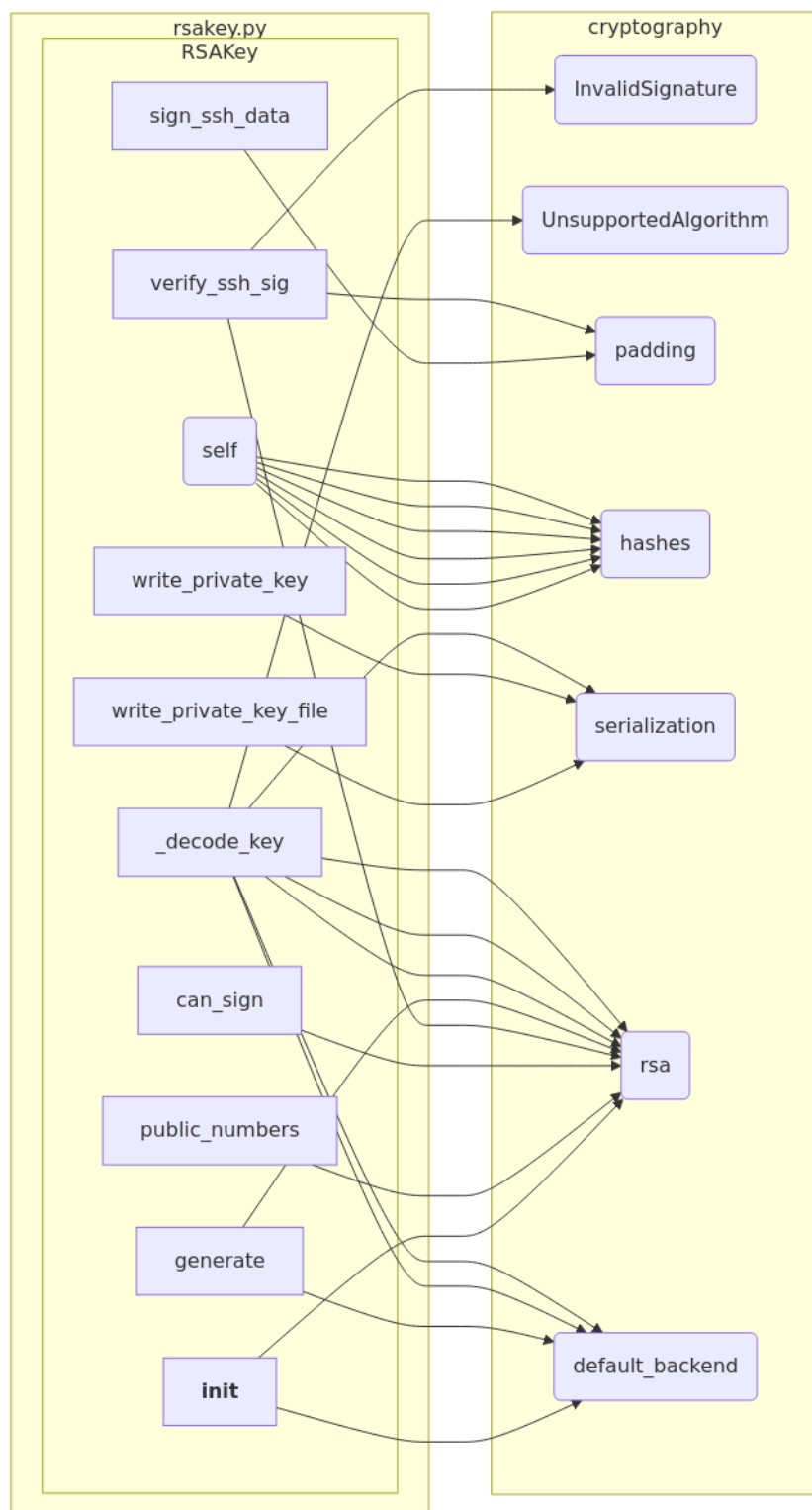


Figure 22: Use of `cryptography` in `rsakey.py`.

`rsa` implements support for managing RSA key pairs, signing messages, and validating signatures.

- `cryptography` (Figure 22) is used for all cryptographic operations, including generating, storing, and loading RSA key pairs, as well as signing and verifying signatures.

HIGH-21		Insecure parameters for digital signatures with RSA	
Likelihood		Impact	
Perimeter	paramiko/rsakey.py.		
Description			
RSAPKey supports the <code>ssh-rsa</code> public key algorithm for server and client authentication using RSA with SHA-1, as defined in RFC 4253. RFC 8332 updates the <code>ssh-rsa</code> algorithm to disallow the use of SHA-1 following NIST recommendations. Notably, NIST SP 800-131A disallows the use of SHA-1 for generating digital signatures. RFC 8332 added <code>rsa-sha2-256</code> and <code>rsa-sha2-512</code> to replace SHA-1, and both methods are supported by Paramiko. OpenSSH disabled RSA signatures with SHA-1 by default in version 8.8.			
Recommendation			
Remove support for RSA with SHA-1.			

No minimum RSA key size

Additionally, it is not recommended to use RSA keys shorter than 2048 bits per [NIST SP 800-131A Rev. 2](#). From a static analysis perspective, we determined that no minimum key size is enforced by `RSAPKey`. We confirmed this in Section 11.3.

9.1.14. `sftp_server.py`

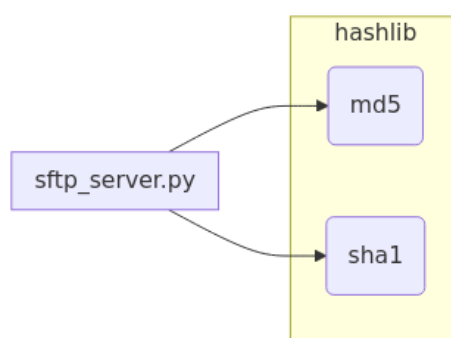


Figure 23: Use of `hashlib` in `sftp_server.py`.

`sftp_server` serves as an implementation of the server-side SFTP protocol.

- `hashlib` (Figure 23) is used to hash files for the file hashing extension (see [draft-ietf-secsh-filexfer-extensions-00](#) Section 3).

Use of MD5 and SHA-1

`sftp_server` implements the file hashing extension with SHA-1 and MD5. These algorithms are allowed per the specification, but the SHA-2 family is also supported. We recommend removing SHA-1 and MD5 in favor of SHA-2.

9.1.15. `ssh_gss.py`

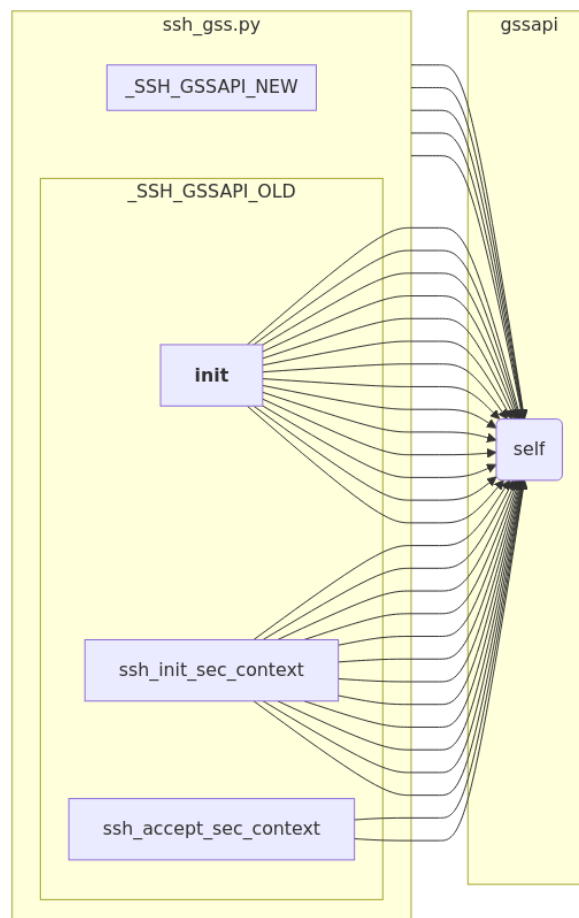


Figure 24: Use of `gssapi` in `ssh_gss.py`.

`ssh_gss` implements support for GSS-API/SPI authentication as defined in [RFC 4462](#).

- `gssapi` (Figure 24) is used for all GSS-API operations.

Success

No issues were found.

9.1.16. `transport.py`

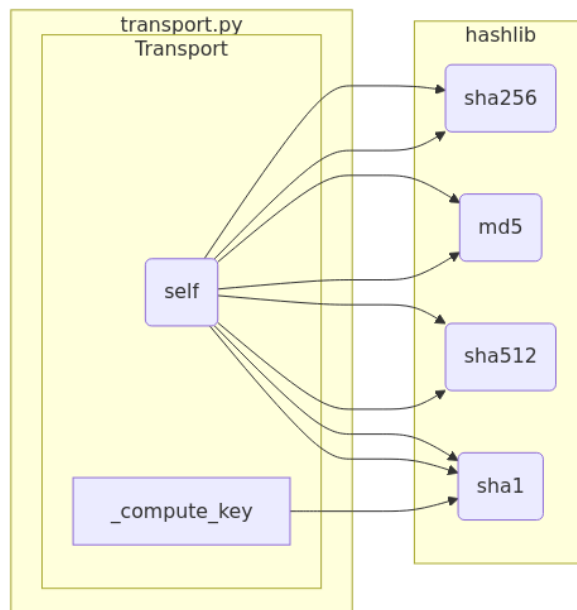


Figure 25: Use of `hashlib` in `transport.py`.

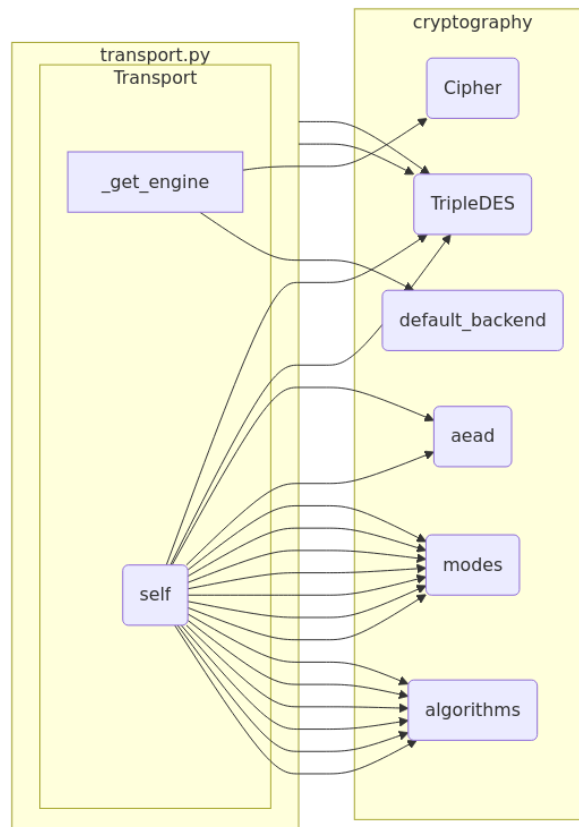


Figure 26: Use of `cryptography` in `transport.py`.

`transport` implements the Transport layer protocol with the `Transport` class, handling the protocol negotiation, starting the key exchange, calling the appropriate authentication methods of `AuthHandler`, and managing the channels.

- `cryptography` (Figure 26) is used when associating cipher names with their actual implementations and modes of operation in the `cipher_info` table of `Transport`. The implementation is instantiated in the `_get_engine` method, which calls the `Cipher` module.
- `hashlib` (Figure 25) is used when associating the MAC algorithm names to their actual implementations in the `_mac_info` table of `Transport`. `sha1` is also used as the fallback hash algorithm in `_compute_key`.

Success

No issues were found.

9.2. Random number generation

Paramiko relies on Python's built-in `os.urandom` function for random number generation, which uses an OS-dependent randomness source that is suitable for cryptographic use (see

<https://docs.python.org/3/library/os.html#os.urandom>). Listing 5 details the files, classes, and methods employ `os.urandom`.

```
paramiko
├─ channel.py
│   └─ Channel
│       └─ request_x11
│           └─ os.urandom @ 475
├─ hostkeys.py
│   └─ HostKeys
│       └─ hash_host
│           └─ os.urandom @ 296
├─ kex_gex.py
│   └─ KexGex
│       └─ _generate_x
│           └─ os.urandom @ 119
├─ kex_group1.py
│   └─ KexGroup1
│       └─ _generate_x
│           └─ os.urandom @ 88
├─ kex_gss.py
│   └─ KexGSSGroup1
│       └─ _generate_x
│           └─ os.urandom @ 154
│   └─ KexGSSGex
│       └─ _generate_x
│           └─ os.urandom @ 415
├─ packet.py
│   └─ Packetizer
│       └─ _build_packet
│           └─ os.urandom @ 691
├─ primes.py
│   └─ _roll_random
│       └─ os.urandom @ 43
└─ transport.py
    └─ Transport
        └─ send_ignore
            └─ os.urandom @ 1226
            └─ os.urandom @ 1227
        └─ _send_kex_init
            └─ os.urandom @ 2451
```

Listing 5: Uses of `os.urandom` in Paramiko

We analyzed the calls to `os.urandom` to verify proper usage. First, some instances use `urandom` to generate random byte strings that are used directly.

In `channel.py`, a 16-byte authentication cookie is generated if none is supplied (Listing 6).

```
426     def request_x11(Python
427         self,
428         screen_number=0,
429         auth_protocol=None,
430         auth_cookie=None,
431         single_connection=False,
432         handler=None,
433     ):
434         ...
435         ...
472         if auth_protocol is None:
473             auth_protocol = "MIT-MAGIC-COOKIE-1"
474         if auth_cookie is None:
475             auth_cookie = binascii.hexlify(os.urandom(16))
```

Listing 6: `urandom` call in `channel.py`

Success

This is compliant with RFC 4254 Section 6.3.1.

In `hostkeys.py:296`, a 20-byte salt is generated if none is given (Listing 7).

```
284     @staticmethodPython
285     def hash_host(hostname, salt=None):
286         ...
287         ...
295         if salt is None:
296             salt = os.urandom(shal().digest_size)
297         else:
298             if salt.startswith("|1|"):
299                 salt = salt.split("|")[2]
300                 salt = decodebytes(b(salt))
301             assert len(salt) == shal().digest_size
302             hmac = HMAC(salt, b(hostname), shal().digest())
303             hostkey = "|1|{}|{}".format(u(encodebytes(salt)), u(encodebytes(hmac)))
304             return hostkey.replace("\n", "")
```

Listing 7: `urandom` call in `hostkeys.py`

Success

This matches OpenSSH's behavior: when hashing a new host name, OpenSSH generates a random salt of the same length as a SHA-1 digest, using `arc4random_buf` as the PRNG.

In `packet.py`, random padding for a packet is generated if the packet is encrypted with any mode other than SDCTR (Listing 8).

```
676     def _build_packet(self, payload):
677         # pad up at least 4 bytes, to nearest block-size (usually 8)
678         bsize = self.__block_size_out
679         # do not include payload length in computations for padding in EtM mode
680         # (payload length won't be encrypted)
681         addlen = 4 if self.__etm_out or self.__aead_out else 8
682         padding = 3 + bsize - ((len(payload) + addlen) % bsize)
683         packet = struct.pack(">IB", len(payload) + padding + 1, padding)
684         packet += payload
685         if self.__sdctr_out or self.__block_engine_out is None:
686             # cute trick i caught openssh doing: if we're not encrypting or
687             # SDCTR mode (RFC4344),
688             # don't waste random bytes for the padding
689             packet += zero_byte * padding
690         else:
691             packet += os.urandom(padding)
692         return packet
```

Listing 8: `urandom` call in `packet.py`

Success

This is compliant with RFC 4253 Section 6: the padding is randomly generated, and its length is at least 4 bytes but no more than 255 bytes.

In `transport.py`, a random payload is generated for `MSG_IGNORE`. The length of the payload may also be random and also determined using `urandom` (Listing 9).

```
1212     def send_ignore(self, byte_count=None):
1213         """
1214         Send a junk packet across the encrypted link. This is sometimes used
1215         to add "noise" to a connection to confuse would-be attackers. It can
1216         also be used as a keep-alive for long lived connections traversing
1217         firewalls.
1218
1219         :param int byte_count:
```

```

1220         the number of random bytes to send in the payload of the ignored
1221         packet -- defaults to a random number from 10 to 41.
1222         """
1223         m = Message()
1224         m.add_byte(cMSG_IGNORE)
1225         if byte_count is None:
1226             byte_count = (byte_ord(os.urandom(1)) % 32) + 10
1227         m.add_bytes(os.urandom(byte_count))
1228         self._send_user_message(m)

```

Listing 9: `urandom` call in `transport.py`

Success

The specification does not say how to generate the random payload, nor its length. OpenSSH uses `MSG_IGNORE` to thwart traffic analysis by sending an ignore packet when echoing is disabled, usually during password input, see [here](#).

```

2259 if (c->isatty && dlen >= 1 && buf[0] != '\r') {
2260     if (tcgetattr(c->wfd, &tio) == 0 &&
2261         !(tio.c_lflag & ECHO) && (tio.c_lflag & ICANON)) {
2262         /*
2263          * Simulate echo to reduce the impact of
2264          * traffic analysis. We need to match the
2265          * size of a SSH2_MSG_CHANNEL_DATA message
2266          * (4 byte channel id + buf)
2267          */
2268         if ((r = sshpkt_msg_ignore(ssh, 4+len)) != 0 ||
2269             (r = sshpkt_send(ssh)) != 0)
2270             fatal_fr(r, "channel %i: ignore", c->self);
2271     }
2272 }

```

Implementers of interactive shells can use the `byte_count` argument to mimic this behavior.

In `transport.py`, a 16-byte random cookie is generated for `MSG_KEXINIT` (Listing 10).

```

2395 def _send_kex_init(self):
2396     """
2397     announce to the other side that we'd like to negotiate keys, and what
2398     kind of key negotiation we support.
2399     """
2400     ...

```



```

2449     m = Message()
2450     m.add_byte(cMSG_KEXINIT)
2451     m.add_bytes(os.urandom(16))

```

Listing 10: `urandom` call in `transport.py`

Success

This is compliant with RFC 4253 Section 7.1.

The other instances use `os.urandom` to generate integers in specific ranges: `kex_gex.py:119`, `kex_gss.py:154`, and `kex_group1.py:88` choose a value x such that $1 < x < q$ for the parameter q of the Diffie-Hellman group used, while the `_roll_random` function in `primes.py` chooses a value between 0 and $N - 1$.

All four of these instances use rejection sampling to pick an integer in the correct range. This is a widely used technique for generating uniformly random numbers from the output of a CSPRNG.

9.2.1. `_generate_x`

The `_generate_x()` methods (see Listing 11) of `KexGSSGex` and `KexGex` are the same. They implement a key generation algorithm based on [NIST SP 800-56A Rev.3](#), Section 5.6.1.1.4. It works by obtaining a random bit string of the required size (here, the bit length of q), converting it to an integer, and checking if it is part of the desired range. Since `os.urandom` returns `bytes`, a mask is applied to ensure each draw has the correct number of bits.

```

1  def _generate_x(self):
2      # generate an "x" ( $1 < x < (p-1)/2$ ).
3      q = (self.p - 1) // 2
4      qnorm = util.deflate_long(q, 0)
5      qhbyte = byte_ord(qnorm[0])
6      byte_count = len(qnorm)
7      qmask = 0xFF
8      while not (qhbyte & 0x80):
9          qhbyte <<= 1
10         qmask >>= 1
11     while True:
12         x_bytes = os.urandom(byte_count)
13         x_bytes = byte_mask(x_bytes[0], qmask) + x_bytes[1:]
14         x = util.inflate_long(x_bytes, 1)
15         if (x > 1) and (x < q):
16             break
17     self.x = x

```

Listing 11: `_generate_x` method from `KexGSSGex` (`kex_gss.py`) and `KexGex` (`kex_gex.py`)

Success

This is compliant with NIST SP 800-56A Rev.3.

Note

We tested the presence of a bias using TestU01 in Section 10.2.2.

Similar to the other `_generate_x()` methods, the one of `KexGSSGroup1` (Listing 12) generates a random number such that $1 < x < q$. It uses the same method of masking some bits to match the expected bit length, except instead of checking that the result is less than q , it rejects results whose 63 first bits are all ones or zeroes.

```
1  def _generate_x(self):
2      """
3      generate an "x" ( $1 < x < q$ ), where  $q$  is  $(p-1)/2$ .
4       $p$  is a 128-byte (1024-bit) number, where the first 64 bits are 1.
5      therefore  $q$  can be approximated as a  $2^{1023}$ . we drop the subset of
6      potential  $x$  where the first 63 bits are 1, because some of those will
7      be larger than  $q$  (but this is a tiny tiny subset of potential  $x$ ).
8      """
9      while 1:
10         x_bytes = os.urandom(128)
11         x_bytes = byte_mask(x_bytes[0], 0x7F) + x_bytes[1:]
12         first = x_bytes[:8]
13         if first not in (self.b7ffffffffffffffff, self.b0000000000000000):
14             break
15         self.x = util.inflate_long(x_bytes)
```

Listing 12: `KexGSSGroup1._generate_x()` method from `kex_gss.py`.

This technique removes two patterns out of 2^{63} , which introduces a negligible bias. It would still be preferable to use a standardized method, like the one used for the other instances of `_generate_x()`, or use Python's `secrets` module with its `randbelow()` function. This function uses the rejection method with `urandom`, like `_generate_x()` currently does, so it would not introduce compatibility issues.

Warning

`KexGSSGroup1._generate_x()` introduces a small bias in the random number generation: it is preferable to apply the same method as the other instances or use Python's `secrets.randbelow()`.

9.2.2. `_roll_random`

Similar to the previous section, although with a different implementation, `_roll_random()` tests candidates generated with `os.urandom`, masking bits to ensure the correct size, and verifying the result is in the required range.

```
1 def _roll_random(n):
2     """returns a random # from 0 to N-1"""
3     bits = util.bit_length(n - 1)
4     byte_count = (bits + 7) // 8
5     hbyte_mask = pow(2, bits % 8) - 1
6
7     # so here's the plan:
8     # we fetch as many random bits as we'd need to fit N-1, and if the
9     # generated number is >= N, we try again.  in the worst case (N-1 is a
10    # power of 2), we have slightly better than 50% odds of getting one that
11    # fits, so i can't guarantee that this loop will ever finish, but the odds
12    # of it looping forever should be infinitesimal.
13    while True:
14        x = os.urandom(byte_count)
15        if hbyte_mask > 0:
16            x = byte_mask(x[0], hbyte_mask) + x[1:]
17        num = util.inflate_long(x, 1)
18        if num < n:
19            break
20    return num
```

Listing 13: `_roll_random` from `primes.py`.

Success

This is compliant with NIST SP 800-56A Rev.3.

Note

We tested the presence of a bias using TestU01 in Section 10.2.2.

9.3. Cryptographic primitive implementations in (PYCA) Cryptography

9.3.1. Python API

The following is a comprehensive list of Python functions called by Paramiko in PYCA Cryptography. Many of these functions are abstract classes or wrappers around their corre-

sponding Rust implementations. To create this list, we developed a script and examined every function call mentioned in the output.

9.3.1.1. exceptions.InvalidSignature

N/A (abstract)

9.3.1.2. exceptions.UnsupportedAlgorithm

N/A (abstract)

9.3.1.3. hazmat.backends.default_backend

In `src/cryptography/hazmat/backends/__init__.py`, sets the default backend to the `OpenSSL` backend.

```
1 def default_backend() -> Any:
2     from cryptography.hazmat.backends.openssl.backend import backend
3
4     return backend
```

 Python

Success

No issues were found.

9.3.1.4. hazmat.decrepit.ciphers.algorithms.TripleDES

In `src/cryptography/hazmat/decrepti/ciphers/algorithms.py`, defines triple DES as a `BlockCipherAlgorithm`. We noticed several issues in the initialization of a `TripleDes` instance in the following code:

```
1 def __init__(self, key: bytes):
2     if len(key) == 8:
3         key += key + key
4     elif len(key) == 16:
5         key += key[:8]
6     self.key = _verify_key_size(self, key)
```

 Python

Warning

1. Expanding 8-byte keys like that is insecure. Many libraries explicitly reject 8-byte keys for 3DES.
2. Since January 2024, in [SP.800-67r2](#), Section 3.4, NIST specifies that triple DES with only two unique keys should *not be used to apply cryptographic protections*.

Note

We confirmed that 8-byte and 16-byte keys are accepted in Section 11.2

MEDIUM-22		Use of 8-byte seed for TripleDES key generation		
Likelihood			Impact	
Perimeter	Encryption.			
Description				
The Triple DES implementation allows 8-byte input for the key. This is an insecure way to initialize the key and expand it into 16-byte, and has been deprecated and labeled insecure by NIST since 2024.				
Recommendation				
Reject 8-byte input for the key initialization of Triple DES.				

9.3.1.5. `hazmat.primitives.asymmetric.ec.ECDH`

N/A (abstract)

9.3.1.6. `hazmat.primitives.asymmetric.ec.ECDSA`

In `src/cryptography/hazmat/primitives/asymmetric/ec.py`, defines `ECDSA` as a `EllipticCurveSignatureAlgorithm` class. Constructor for the signature algorithm on top of the OpenSSL backend.

Success

No issues were found.

9.3.1.7. `hazmat.primitives.asymmetric.ec.EllipticCurvePrivateKey`

N/A (abstract). Methods that need to be defined:

- `exchange` to perform a key exchange operation
- `public_key`, the public key for this private key
- `curve`, the curve that the key is on
- `key_size`, the bit size of a secret scalar for the curve
- `sign`, the signature function
- `private_numbers`, to return an `EllipticCurvePrivateNumbers`
- `private_bytes`, to return the key serialized as bytes

- `copy`, to copy the key

9.3.1.8. `hazmat.primitives.asymmetric.ec.EllipticCurvePublicKey.from_encoded_point`

Method in the Class `EllipticCurvePublicKey` in `primitives/asymmetric/ec.py`, returns an `EllipticCurvePublicKey` from an encoded point by calling Rust.

Success

No issues were found.

9.3.1.9. `hazmat.primitives.asymmetric.ec.SECP256R1`

Definition of the `SECP256R1` curve done in `primitives/asymmetric/ec.py`.

Success

No issues were found.

9.3.1.10. `hazmat.primitives.asymmetric.ec.SECP384R1`

Definition of the `SECP384R1` curve done in `primitives/asymmetric/ec.py`.

Success

No issues were found.

9.3.1.11. `hazmat.primitives.asymmetric.ec.SECP521R1`

Definition of the `SECP521R1` curve done in `primitives/asymmetric/ec.py`.

Success

No issues were found.

9.3.1.12. `hazmat.primitives.asymmetric.ec.derive_private_key`

In `primitives/asymmetric/ec.py`, returns an `EllipticCurvePrivateKey` from an integer by calling Rust.

Success

No issues were found.

9.3.1.13. `hazmat.primitives.asymmetric.ec.generate_private_key`

In `primitives/asymmetric/ec.py`, direct alias to the Rust function to generate a private key.

Success

No issues were found.

9.3.1.14. `hazmat.primitives.asymmetric.ed25519.Ed25519PrivateKey`

In `primitives/asymmetric/ed25519.py` it defines a class for Ed25519 private keys. It contains various methods, some of which are abstract, while others call Rust. The following methods are present:

- *generate*: generate a `Ed25519PrivateKey` calling Rust.
- *from_private_bytes*: generate a `Ed25519PrivateKey` from private data calling Rust.
- *public_key*: abstract method to derive the `Ed25519PublicKey` from the private key.
- *private_bytes*: abstract method to serialize the private key into encrypted bytes.
- *private_bytes_raw*: abstract method to serialize the private key into bytes.
- *sign*: abstract method to sign the data.
- *copy*: abstract method to copy the private key

Success

No issues were found.

9.3.1.15. `hazmat.primitives.asymmetric.padding.PKCS1v15`

N/A (defines the name to “EMSA-PKCS1-v1_5”)

9.3.1.16. `hazmat.primitives.asymmetric.rsa.RSAPrivateKey`

In `primitives/asymmetric/rsa.py`, it defines a class for RSA private keys. It consists of several abstract methods. The following methods are present:

- *decrypt*: abstract method to decrypt the provided ciphertext.
- *key_size*: abstract method to return the bit length of the public modulus.
- *sign*: abstract method to sign the data.
- *private_numbers*: abstract method to return `RSAPrivateNumbers`.
- *private_bytes*: abstract method to serialize the private key into bytes.
- *copy*: abstract method to copy the private key

Success

No issues were found.

9.3.1.17. `hazmat.primitives.asymmetric.rsa.RSAPrivateNumbers`

In `primitives/asymmetric/rsa.py`, direct alias to the Rust function that generates RSA private numbers (the raw integer components of an RSA private key).

Success

No issues were found.

9.3.1.18. `hazmat.primitives.asymmetric.rsa.RSAPublicNumbers`

In `primitives/asymmetric/rsa.py`, direct alias for the Rust function that generates RSA public numbers (the public-key counterpart of `RSAPrivateNumbers`).

Success

No issues were found.

9.3.1.19. `hazmat.primitives.asymmetric.rsa.generate_private_key`

In `primitives/asymmetric/rsa.py`, it ensures that the RSA parameters, such as the exponent, are correct. It must be either 3 for legacy or 65,537 as the default modern one. It then calls Rust.

Success

No issues were found.

9.3.1.20. `hazmat.primitives.asymmetric.utils.decode_dss_signature`

In `primitives/asymmetric/utils.py`, direct alias for the Rust function that decodes DSS signatures.

Success

No issues were found.

9.3.1.21. `hazmat.primitives.asymmetric.utils.encode_dss_signature`

In `primitives/asymmetric/utils.py`, direct alias of the Rust function for encoding DSS signatures.

Success

No issues were found.

9.3.1.22. `hazmat.primitives.asymmetric.x25519.X25519PrivateKey.generate`

In `primitives/asymmetric/x25519.py`, the method inside the `X25519PrivateKey` to generate a `X25519PrivateKey` by calling Rust.

Success

No issues were found.

9.3.1.23. `hazmat.primitives.asymmetric.x25519.X25519PublicKey.from_public_bytes`

In `primitives/asymmetric/x25519.py`, the method inside the `X25519PublicKey` to generate a `X25519PublicKey` from input bytes by calling Rust.

Success

No issues were found.

9.3.1.24. `hazmat.primitives.ciphers.Cipher`

In `primitives/ciphers/base.py`, it defines a class for ciphers. It contains various abstract methods. The following methods are present:

- *encryptor*: method to generate an encryption context for the cipher and its corresponding mode by calling Rust.
- *encryptor*: type overloading to return an `AEADEncryptionContext`
- *encryptor*: type overloading to return an `CipherContext`
- *decryptor*: method to generate a decryption context for the cipher and its corresponding mode by calling Rust.
- *decryptor*: type overloading to return an `AEADDecryptionContext`
- *decryptor*: type overloading to return an `CipherContext`

Success

No issues were found.

9.3.1.25. `hazmat.primitives.ciphers.aead.AESGCM`

In `primitives/ciphers/aead.py`, direct alias to the Rust function for AES in GCM mode.

9.3.1.26. `hazmat.primitives.ciphers.algorithms.AES`

In `primitives/ciphers/algorithms.py`, it defines a class for AES with its name, the supported key sizes, and the block size.

Success

No issues were found.

9.3.1.27. `hazmat.primitives.ciphers.algorithms.TripleDES`

In `primitives/ciphers/algorithms.py`, it defines `TripleDES` as a deprecated algorithm, warning of its removal in version 48.0.0. The current latest version is 45.0.7.

Success

No issues were found.

9.3.1.28. `hazmat.primitives.ciphers.modes.CBC`

In `primitives/ciphers/modes.py`, defines a class `CBC` from a class `ModeWithInitializationVector`.

Success

No issues were found.

9.3.1.29. `hazmat.primitives.ciphers.modes.CTR`

In `primitives/ciphers/modes.py`, defines a class `CTR` from a class `ModeWithNonce`.

Success

No issues were found.

9.3.1.30. `hazmat.primitives.constant_time.bytes_eq`

In `primitives/constant_time.py`, it uses the function `compare_digest` from Python's `hmac` packages. Since Python 3.3, this function uses an approach to prevent timing analysis.

Success

No issues were found.

Info

In the latest Python documentation, it is stated that since version 3.10, `compare_digest` uses OpenSSL's `CRYPTO_memcmp()` internally when available. Since PYCA Cryptography library moved most of its cryptographic building blocks into Rust bindings of OpenSSL, it would also make sense to make this change here instead of relying on the `hmac` package. This would allow to have a single source for all cryptographically related operations.

9.3.1.31. `hazmat.primitives.hashes.SHA1`

In `primitives/hashes.py`, defines a class `SHA1` from the `HashAlgorithm`, with digest size and block size.

Success

No issues were found.

9.3.1.32. `hazmat.primitives.hashes.SHA256`

In `primitives/hashes.py`, defines a class `SHA256` from a class `HashAlgorithm`, with digest size and block size.

Success

No issues were found.

9.3.1.33. `hazmat.primitives.hashes.SHA384`

In `primitives/hashes.py`, defines a class `SHA384` from a class `HashAlgorithm`, with digest size and block size.

Success

No issues were found.

9.3.1.34. `hazmat.primitives.hashes.SHA512`

In `primitives/hashes.py`, defines a class `SHA512` from a class `HashAlgorithm`, with digest size and block size.

Success

No issues were found.

9.3.1.35. hazmat.primitives.padding.PKCS7

In `primitives/padding.py`, it defines a class `PKCS7` for padding that relies on the `PaddingContext` class, composed of abstract methods only.

Success

No issues were found.

9.3.1.36. hazmat.primitives.serialization.BestAvailableEncryption

In `primitives/_serialization.py`, defines a class `BestAvailableEncryption`. This is a key serialization encryption type in the cryptography library. It automatically chooses the strongest available encryption algorithm to protect private keys during serialization.

Success

No issues were found.

9.3.1.37. hazmat.primitives.serialization.load_der_private_key

In `primitives/serialization/base.py`, direct alias of the Rust function.

Success

No issues were found.

9.3.1.38. hazmat.primitives.serialization.load_pem_private_key

In `primitives/serialization/base.py`, direct alias to the Rust function.

Success

No issues were found.

9.3.1.39. hazmat.primitives.serialization.load_ssh_private_key

In `primitives/serialization/base.py`, direct alias to the Rust function.

Success

No issues were found.

9.3.1.40. Miscellaneous

The following other portions of the code from the Cryptography library’s serialization module are called by Paramiko but are not considered relevant: `Encoding::Raw`, `x962`, `NoEncryption`, `PrivateFormat.TraditionalOpenSSL`, `PublicFormat.Raw`, `PublicFormat.UncompressedPoint`.

9.3.2. Rust bindings to OpenSSL

Here is a list of all the Rust cryptographic primitives called by the Python API. These implementations are external function calls to the OpenSSL library.

9.3.2.1. AES

Paramiko utilizes the three AES key sizes (128, 192, and 256) across three distinct modes: CTR, CBC, and GCM. These six underlying primitives – AES-128/192/256-CTR/CBC and AES-128/256-GCM – are registered in the Rust section of `src/rust/src/backend/cipher_registry.rs`. The specific encryption/decryption tasks are managed by the `CipherContext` object in `src/rust/src/backend/ciphers.rs`. This structure interacts with OpenSSL’s `CipherCtx` and can operate with any pre-registered cipher mode.

```
1 pub(crate) struct CipherContext {
2     ctx: openssl::cipher_ctx::CipherCtx,
3     py_mode: pyo3::PyObject,
4     py_algorithm: pyo3::PyObject,
5     side: openssl::symm::Mode,
6 }
```

 Rust

Success

No issues were found.

9.3.2.2. SHA-{256,384,512}

SHA-256, SHA-384, and SHA-512 are implemented in Rust through the `Hash` structure in `src/rust/src/backend/hashes.rs`. This structure wraps OpenSSL’s hash implementations.

```
1 pub(crate) struct Hash {
2     #[pyo3(get)]
3     algorithm: pyo3::Py<pyo3::PyAny>,
4     ctx: Option<openssl::hash::Hasher>,
5 }
```

 Rust

The fundamental building blocks, or primitives, are defined as enum variants in the `AlgorithmParameters` in `src/rust/cryptography-x509/src/common.rs`:

```

1 #[defined_by(oid::SHA256_OID)]
2 Sha256(Option<asn1::Null>),
3 #[defined_by(oid::SHA384_OID)]
4 Sha384(Option<asn1::Null>),
5 #[defined_by(oid::SHA512_OID)]
6 Sha512(Option<asn1::Null>),

```

 Rust

Success

No issues were found.

9.3.2.3. HMAC-SHA2-{256,512}

Rust implements the HMAC-SHA2-256 and HMAC-SHA2-512 algorithms through the `Hmac` structure in `src/rust/src/backend/hmac.rs`. This structure wraps OpenSSL's HMAC implementation.

```

1 pub(crate) struct Hmac {
2     #[pyo3(get)]
3     algorithm: pyo3::Py<pyo3::PyAny>,
4     ctx: Option<cryptography_openssl::hmac::Hmac>,
5 }

```

 Rust

The `HMAC` structure offers primary HMAC functionality by wrapping OpenSSL's `HMAC_CTX`.

```

1 pub(crate) fn new_bytes(
2     py: pyo3::Python<'_>,
3     key: &[u8],
4     algorithm: &pyo3::Bound<'_, pyo3::PyAny>,
5 ) -> CryptographyResult<Hmac> {
6     let md = message_digest_from_algorithm(py, algorithm)?;
7     let ctx = cryptography_openssl::hmac::Hmac::new(key, md).map_err(|_| {
8         exceptions::UnsupportedAlgorithm::new_err((
9             "Digest is not supported for HMAC",
10            exceptions::Reasons::UNSUPPORTED_HASH,
11        ))
12     })?;
13
14     Ok(Hmac {
15         ctx: Some(ctx),
16         algorithm: algorithm.clone().unbind(),
17     })
18 }

```

 Rust

The low-level integration of OpenSSL is located in `src/rust/cryptography-openssl/src/hmac.rs`. This file provides FFI bindings to OpenSSL's HMAC functions.

```
1 foreign_types::foreign_type! {  
2     type CType = ffi::HMAC_CTX;  
3     fn drop = ffi::HMAC_CTX_free;  
4  
5     pub struct Hmac;  
6     pub struct HmacRef;  
7 }
```

This module wraps OpenSSL's `HMAC_CTX` and offers secure Rust functions for HMAC operations.

```
1 pub fn new(key: &[u8], md: openssl::hash::MessageDigest) ->  
    OpenSSLResult<Hmac> {  
2     // SAFETY: All FFI conditions are handled.  
3     unsafe {  
4         let h = Hmac::from_ptr(cvt_p(ffi::HMAC_CTX_new())?);  
5         cvt(ffi::HMAC_Init_ex(  
6             h.as_ptr(),  
7             key.as_ptr().cast(),  
8             key.len(),  
9             .try_into()  
10            .expect("Key too long for OpenSSL's length type"),  
11            md.as_ptr(),  
12            ptr::null_mut(),  
13        ))?;  
14         Ok(h)  
15     }  
16 }
```

Success

No issues were found.

9.3.2.4. RSA

RSA is implemented in Rust in `src/rust/src/backend/rsa.rs`. It wraps OpenSSL's RSA implementation, providing two main structures: `RsaPrivateKey` and `RsaPublicKey`.

```

1 pub(crate) struct RsaPrivateKey {
2     pkey: openssl::pkey::PKey<openssl::pkey::Private>,
3 }
4 ...
5
6 pub(crate) struct RsaPublicKey {
7     pkey: openssl::pkey::PKey<openssl::pkey::Public>,
8 }

```

The processes of signing and verifying data using RSA, as implemented with the low-level OpenSSL integrations, are also specified in `src/rust/src/backend/rsa.rs`.

- Signing:

```

1 fn sign<'p>(
2     &self,
3     py: pyo3::Python<'p>,
4     data: CffiBuf<'>,
5     padding: &pyo3::Bound<'p, pyo3::PyAny>,
6     algorithm: &pyo3::Bound<'p, pyo3::PyAny>,
7 ) -> CryptographyResult<pyo3::Bound<'p, pyo3::types::PyAny>> {
8     ...
9 }

```

- Verification

```

1 fn verify(
2     &self,
3     py: pyo3::Python<'>,
4     signature: CffiBuf<'>,
5     data: CffiBuf<'>,
6     padding: &pyo3::Bound<'_, pyo3::PyAny>,
7     algorithm: &pyo3::Bound<'_, pyo3::PyAny>,
8 ) -> CryptographyResult<()> {
9     ...
10 }

```

Warning

In `setup_signature_ctx`, the key type is never checked before calling RSA-specific APIs. Make RSA-specific branches explicit and return a clear `UnsupportedAlgorithm` if the padding chosen is not compatible with the key type.

9.3.2.5. P-{256,384,521}

P-256, P-384, and P-521 are implemented in Rust in `src/rust/src/backend/ec.rs`. This file provides wrappers for OpenSSL's elliptic curve implementation.

```
1 pub(crate) struct ECPrivateKey {
2     pkey: openssl::pkey::PKey<openssl::pkey::Private>,
3     #[pyo3(get)]
4     curve: pyo3::Py<pyo3::PyAny>,
5 }
6 ...
7 pub(crate) struct ECPublicKey {
8     pkey: openssl::pkey::PKey<openssl::pkey::Public>,
9     #[pyo3(get)]
10    curve: pyo3::Py<pyo3::PyAny>,
11 }
```

The `curve_from_py_curve` function maps Python curve objects into OpenSSL's `EcGroup` by matching curve names with OpenSSL NIDs.

```
1 ...
2 "secp256r1" => openssl::nid::Nid::X9_62_PRIME256V1,
3 "secp384r1" => openssl::nid::Nid::SECP384R1,
4 "secp521r1" => openssl::nid::Nid::SECP521R1,
5 ...
```

The low-level OpenSSL integration for key generation, verification, signing and validation is also located in `src/rust/src/backend/ec.rs`.

Success

No issues were found.

Warning

It is a common practice to zeroize the private scalar, as it may remain in memory after use. This would not necessarily be a security issue but is worth noting.

9.3.2.6. Ed25519

Rust's implementation of Ed25519 is located in `src/rust/src/backend/ed25519.rs`. This file provides wrappers around OpenSSL's Ed25519 implementation.

```

1 pub(crate) struct Ed25519PrivateKey {
2     pkey: openssl::pkey::PKey<openssl::pkey::Private>,
3 }
4 ...
5 pub(crate) struct Ed25519PublicKey {
6     pkey: openssl::pkey::PKey<openssl::pkey::Public>,
7 }

```



The low-level OpenSSL integration for key generation, validation, signing and verification can be found in `src/rust/src/backend/ed25519.rs`.

Success

No issues were found.

Warning

Here again there is no zeroization of the private key buffers after use.

9.4. Conclusion

9.4.1. Paramiko

During the static analysis phase, we assessed Paramiko's usage of cryptographic primitives and random number generators. We did not find issues in the way Paramiko uses cryptographic libraries such as `cryptography`. However, we found uses of deprecated or obsolete primitives and parameters, such as 1024-bit Diffie-Hellman groups or RSA signatures with SHA-1. These choices come from the original set of RFCs, which have since been superseded by new specifications that updated the list of recommended primitives and parameters.

We recommend applying the necessary corrections for all instances noted in this section. We acknowledge that these changes are incompatible with older servers and clients. However, in most cases, OpenSSH has already implemented the recommended changes. We noted the specific version whenever possible.

Regarding random number generation, we did not find issues during this phase when the values generated of `urandom` are used directly, nor when they are used to generate integers in specific ranges. However, for most cases, key generation is not covered, as it is usually done by OpenSSL through `cryptography`. Both RNG use and key generation are covered by our additions to the test suite in Section 10.2.2.

In conclusion, we have some additional recommendations.

Recommendation: ChaCha20-Poly1305

Add support for ChaCha20-Poly1305 as a cipher. This is the default choice in OpenSSH since version 6.9 and the primitive is implemented by Cryptography (see [Authenticated encryption](#)).

Recommendation: hybrid PQ key agreement

Add support for the hybrid post-quantum key agreement algorithm `mlkem768x25519-sha256`. This is the default choice in OpenSSH since version 10.0 (released in April 2025) and version 10.1 will warn users when a non post-quantum key agreement algorithm is used. However, Cryptography does not offer ML-KEM yet but integration through OpenSSL 3.5 is planned (see [Issue 12610](#)).

Recommendation: deterministic ECDSA

Paramiko introduced deterministic ECDSA signatures in version 1.14.0 (see [PR 299](#)) in order to prevent issues that arise when generating the random parameter k (a poor entropy source can lead up to the compromise of the private key). However, when replacing PyCrypto with PYCA's `cryptography` in version 2.0.0, Paramiko also re-introduced “randomized” ECDSA as `cryptography` did not support deterministic ECDSA. Now that PYCA added deterministic ECDSA to `cryptography` in version 45.0.0, we recommend using it.

9.4.2. PYCA Cryptography

Many pieces of code rely on OpenSSL's default behavior. Setting it up correctly is essential, and this should happen, since PYCA Cryptography begins with the OpenSSL binding (`src/cryptography/hazmat/bindings/openssl/binding.py`).

Warning

We have discovered numerous instances in PYCA Cryptography Rust implementation where errors from OpenSSL are converted to a `unwrapor(false)` instead of specific Python exceptions. This makes errors from the code indistinguishable and could make an attacker tamper information.

INFO-23	Mapping Python errors to <code>unwrap_or(false)</code>
Perimeter	Cryptography Rust
Description	
By using <code>.unwrap_or(false)</code> instead of properly map Python errors, the code suppresses all verification errors, including critical misuse or failure conditions. This: • Makes debugging harder • Potentially misleads higher-level code into thinking the signature was simply invalid instead of something being broken • May mask attacker-induced errors or key corruption.	
Recommendation	
Properly propagate the error instead of converting it into <code>false</code> .	

10. Testing enhancement

Paramiko boasts a robust test suite that thoroughly examines the majority of its codebase. As noted in Section 6.3.3, a whopping 80% of the code is covered by at least one test. Some of these tests zero in on a specific function, such as `HostKeys.add`. Others, like the tests in `tests/test_client.py`, interact with multiple components at once.

10.1. Increasing test coverage

As the coverage is not a perfect 100%, we examined whether the functions that use cryptographic primitives were fully covered. To do so, we used the mapping from Section 9.1 in conjunction with the coverage information to identify any lines of code that were not covered in those functions. For each such line, we added specific test cases designed to cover it.

Note

A patch with our modifications to the tests was included in the supplemental material given to the client.

Some situations were easy to address. Typically, any adverse scenario involving incorrect input that results in an exception was not included in the testing. By doing so, we could guarantee that the software handled these situations effectively, avoiding undesirable outcomes such as service interruptions or faulty functioning.

Certain instances were simple to activate, yet they enabled us to execute more code. Notably, the inadequate coverage of `kex_gss.py` was due to the lack of testing for the methods in `'KexGSSGroup1'` and `'KexGSSGroup14'`. We modified the tests for `KexGex` to also use the other two key exchange methods, which significantly increased coverage and revealed a bug in `'KexGSSGroup1'` (and `'KexGSSGroup14'`, which inherits from `'KexGSSGroup1'`).

MEDIUM-24

Wrong type usage in SHA-1 in KexGSSGroup1 and KexGSSGroup14

Likelihood**Impact****Perimeter**

paramiko/kex_gss.py

Description

In KexGSSGroup1, the `_parse_kexgss_complete` method generates the exchange hash from a Message:

```
197 def _parse_kexgss_complete(self, m):  
    ...  
217     # okay, build up the hash H of  
218     # (V_C || V_S || I_C || I_S || K_S || e || f || K)  
219     hm = Message()  
220     hm.add(  
221         self.transport.local_version,  
222         self.transport.remote_version,  
223         self.transport.local_kex_init,  
224         self.transport.remote_kex_init,  
225     )  
226     hm.add_string(self.transport.host_key.__str__())  
227     hm.add_mpint(self.e)  
228     hm.add_mpint(self.f)  
229     hm.add_mpint(K)  
230     H = sha1(str(hm)).digest()
```

`hashlib.sha1` takes as input some bytes, not `str`, causing a `TypeError`. Since `KexGSSGroup14` is derived from `KexGSSGroup1`, it shares the same method.

Recommendation

Change `str(hm)` to `hm.asbytes()` in `KexGSSGroup1`.

10.2. CI improvements

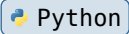
Currently, there are no tests on entropy sources or constant-timeness in either of the projects. As such, one of the goals of this audit was to add a suite of CI tests targeting these two issues.

10.2.1. crypto-condor plugin

[crypto-condor](#), is a compliance-testing framework for cryptographic primitives, available both as a Python library and as a command-line interface (CLI). For a given algorithm, crypto-condor executes the relevant test vectors: a set of predefined inputs are given to the implementation, and the outputs are compared to the test vector values.

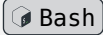
As Paramiko is written in Python, we use crypto-condor's [Python wrapper API](#): a Python module that exposes functions following the API naming convention can be tested with crypto-condor directly. For example, to test the X25519 key exchange, the Python module must export a function with the following name and signature:

```
1 def CC_x25519_exchange(secret_key: bytes, peer_key: bytes) -> bytes:
2     ...
```



This function will automatically be recognized by crypto-condor and tested with the corresponding module. To install and run crypto-condor:

```
1 python -m pip install crypto-condor
2 crypto-condor-cli test auto .
```



To test the cryptographic implementations, the relevant functions were identified and exposed as crypto-condor harness functions. The following sections describe the harnesses added and the results obtained.

Note

The harnesses were part of the supplemental material given to the client.

10.2.2. Randomness generation

In order to test randomness generation, Quarkslab uses a battery of statistical tests based on [TestU01](#), a C library for empirical testing of random number generators. This specific battery is composed of 29 statistical tests that correspond to those used by [NIST Statistical Test Suite](#), with several sets of parameters. They are available through crypto-condor.

We have identified three categories of randomness generation: direct uses of `os.urandom`, functions that generate random numbers using `os.urandom`, and third-party functions that require a good random number generator (e.g. a key generation function). As detailed in Section 9.2, we assume that `os.urandom` is a strong CSPRNG using a good source of entropy. In contrast, the functions such as `_roll_random()` that build upon the output of `os.urandom` to generate numbers in specific intervals require a more detailed study. Third-party libraries are out of the scope of this audit, but the quality of the random values generated by them must be tested to ensure that no issues exist.

To test these two generators of randomness, we can sample considerable amounts of data and treat it as the output of a PRNG to test it with TestU01. Particularly, we are looking for indications that the output of these functions may be biased.

Since crypto-condor offers a Python API, we created two test files. The first one, `test_rng.py`, tests the four functions identified in Section 9.2 that use `os.urandom` to generate random numbers in certain intervals: `_roll_random()`, `KexGroup1._generate_x()`, `KexGex._generate_x()`, and `KexGSSGroup1._generate_x()`. The second one, `cctest_keygen.py`, covers the key generation functions used by the different primitives.

The test for the random number generators calls the functions to output a large set of numbers. As a general rule with TestU01, the more data, the better. This is often limited by the speed at which these generators work: when writing the tests we aimed for 1MB of data for a short feedback loop, which may be enough for detecting a definite bias. If the generator doesn't fail at that mark, we recommend aiming for 1GB but we note that generating 10MB with `KexGroup1._generate_x()` already takes more than four hours.

`_roll_random`

To test `_roll_random()`, we generate 32-bit numbers (between 0 and $2^{32} - 1$), which are the exact type of numbers that TestU01 deals with. We expect the numbers to be uniformly distributed over $[0, 2^{32} - 1]$, so all tests should pass.

Success

The test for `_roll_random()` passes without errors.

`_generate_x`

The case for the `_generate_x()` methods is different: the numbers are drawn from the interval $[2, q - 1]$, where q has a bit length of $2^N - 1$ but is not equal to $2^N - 1$. This means that the numbers generated by these functions are *not* uniformly distributed over $[0, 2^N - 1]$, which is what TestU01 expects². As such, we expect some of the tests to **fail**. By increasing the size of the data generated more tests should fail, as the bias introduced by the modulo becomes more apparent.

Warning

The `_generate_x()` methods fail some but not all of the TestU01 tests. This is expected, as the numbers generated are not uniformly distributed over all 32-bit integers.

Key generation

`cctest_keygen.py` adds tests for the key generation functions found in `ECDSAKey` and `KexNistp*`, as well as tests for the `Ed25519PrivateKey` and `X25519PrivateKey` classes from `cryptography`.

For `ECDSAKey` and `KexNistp*` we use the `ECDSA` module of crypto-condor, as it tests elliptic curve key pairs. It works by generating thousands of keys. If it includes the public key, crypto-condor verifies that it forms a valid key pair with the private key, and whether it is a valid

²More precisely, TestU01 deals with 32-bit integers, so in practice the 2^N -bit numbers are split into 32-bit chunks and the bits of each of those must be uniformly distributed.

public key for the elliptic curve in use. Then all the private keys, which are scalars, are encoded as bytes and concatenated. This output is tested with TestU01.

For `Ed25519PrivateKey`, the key exported with the `private_bytes_raw()` method corresponds to the randomly generated 32-byte seed. 10MB worth of keys are generated, concatenated and tested with TestU01.

Finally, `X25519PrivateKey` has the same `private_bytes_raw()` method, but this time it returns the actual private key, which is a randomly generated 32-byte scalar that has been clamped³. This is by design: the bit manipulation fulfills a security purpose⁴ without introducing a vulnerability due to the added bias. However, this bias is detected by TestU01. To properly test the key generation, crypto-condor generates a key and checks if it appears to have been clamped. If it is, it removes the first and last bytes before concatenating the key to the final output, otherwise the entire key is used.

Success

The tests for key generation functions pass.

We did not test the key generation of `RSAPrivateKey` because the private key is not drawn from a uniform distribution.

10.2.3. Cryptographic primitives

While it can test random number generators with TestU01, the main goal of crypto-condor is to verify the compliance of cryptographic primitives to the relevant specifications. This section identifies the cryptographic primitives used by Paramiko that are supported by crypto-condor. For each one, we created a harness that can be tested directly with crypto-condor.

Note

The harnesses were included in the supplemental material given to the client.

10.2.3.1. AES

AES is used by the transport layer. Concretely, the `Packetizer` class can send and receive encrypted packets with the `send_message()` and `read_message()` methods.

Encryption

The encryption of packets not only includes the payload, but also the packet length, the padding and its length, so the value generated by `send_message()` is not equivalent to encrypting the payload alone. This means that we cannot use test vectors to test these methods, as the ciphertexts will not match. Instead, we use the crypto-condor functions to test the *output* of an implementation: we use the `_build_packet()` method to create the packet with an arbitrary

³The 3 least significant bits are cleared, the most significant bit is cleared, and the next most significant bit is set.

⁴See for example <https://neilmadden.blog/2020/05/28/whats-the-curve25519-clamping-all-about/>

payload *before* it is encrypted, then compute the encrypted packet by calling `send_message()`. With both values, and control over the key and IV used, we write these values to a text file that crypto-condor can parse and replay the operation to verify that the outputs match.

Success

The tests pass for all three modes of operation and for all supported key sizes.

Decryption

Warning

We were unable to test decryption: the `read_message()` method expects a valid SSH packet with a specific structure that cannot be randomly generated. Trying to decrypt packets generated by `send_message()` resulted in `SSHException: invalid packet blocking`.

10.2.3.2. ECDH

The Elliptic Curve Diffie-Hellman (ECDH) key exchange is used by the transport layer to establish a secure communication channel. This is implemented by the `KexNistp256`, `KexNistp384`, and `KexNistp521` classes for their corresponding elliptic curves.

These classes are *not* part of the public API of Paramiko, but we can still access the `kex_ecdh_nist` module where they are defined. There are two operations to test:

- The key generation with `_generate_key_pair()`;
- The actual exchange with the `start_kex()` and `parse_next()` methods.

The key generation function is covered in Section 10.2.2. We added two crypto-condor harnesses to test the key exchange, one from the client perspective and one from the server. In either case, the exchange is performed with the `start_kex()` and `parse_next()` methods. The first one calls `_generate_key_pair()`, which will generate a key pair for the corresponding elliptic curve. To test the exchange, crypto-condor gives the function the two keys required to perform the exchange (the private key of one party and the public key of the other). To use these keys, we mock the key generation to return the ones given by crypto-condor. The output of the exchange should match the value from the test vector.

Success

Both tests for client and server key exchange pass.

10.2.3.3. ECDSA

Paramiko supports ECDSA with the `ECDSAKey` class, which inherits from `PKey`. It exposes three functions related to ECDSA operations:

- `ECDSAKey.generate` for private key generation;
- `ECDSAKey.sign_ssh_data` for signing data;

- `ECDSAKey.verify_ssh_sig` for verifying signatures generated with `sign_ssh_data`.

The crypto-condor harness for ECDSA adds tests for signing and verifying over the three elliptic curves that are supported by Paramiko (`nistp256`, `nistp384`, and `nistp521`). Key generation is covered in Section 10.2.2.

To test signature generation, crypto-condor uses test vectors to verify that the function is compliant with the specification. The implementation is given a private key and a message to sign. In order to test the function, we parse the `Message` returned by `sign_ssh_data` to retrieve the values (r, s) that form the signature and encode it correctly for crypto-condor, which checks if the signature is valid.

Success

The test passes for all three curves supported.

Similar to signing, to test signature verification, the implementation is given a public key, a message, and a signature to verify. The test passes if valid signatures are correctly verified, and invalid signatures are rejected.

Warning

Unlike signing, there are resilience test vectors from Project Wycheproof. They include several invalid signatures, from bit flips to malformed DER encoding.

`verify_ssh_sig` expects a `Message` containing the (r, s) values from the signature as two `mpint` following RFC 5656 Section 3.1.2. However, the signatures included in the test vectors are ASN.1 `Dss-Sig-Value` sequences, so the test must convert from one type to the other. This requires decoding the signature to construct the `Message`: if the signature is malformed, this step fails before calling `verify_ssh_sig`.

We have opted for catching the exceptions raised by the decoder and returning `False`, which crypto-condor counts as a successful rejection of an invalid signature, but we emphasize that `verify_ssh_sig` has not been used to test those values.

Success

The test passes for all three curves supported.

Note that invalid signatures from Wycheproof test vectors may not be tested with `verify_ssh_sig`, see warning above.

10.2.3.4. Ed25519

The Ed25519 signature scheme is available through the `Ed25519Key` class. Like `ECDSAKey`, it has methods for signing and verifying signatures: `sign_ssh_data` and `verify_ssh_sig`. However,

unlike `ECDKey` there is no key generation method, `Ed25519Key` can only be instantiated from an existing key.

We added a crypto-condor harness to test both methods using test vectors. For signing, `sign_ssh_data` is called to sign a message and the results are compared to the test vector values. For verifying, `verify_ssh_sig` is called to verified signatures. There are valid signatures that must be correctly validated and invalid signatures that should be rejected.

Success

The test for Ed25519 signing passes without errors.

Fail

The test for Ed25519 verification fails on signatures that are not exactly 64 bytes: some invalid tests include signatures that have different lengths, including an empty signature, which causes `pynacl` to raise an error that is not handled by Paramiko.

LOW-25		Invalid Ed25519 signature causes mishandled exception		
Likelihood			Impact	
Perimeter	ed25519key.py			
Description				
The <code>verify_ssh_sig</code> method of <code>Ed25519Key</code> does not check that the signature is exactly 64 bytes long before calling <code>nacl.signing.VerifyKey.verify()</code>. A signature of different lengths causes <code>verify()</code> to raise <code>nacl.exceptions.ValueError</code>, which is not handled by <code>verify_ssh_sig</code>.				
Recommendation				
Either check the length of the signature before calling <code>verify()</code> or handle the exception.				

Note

We verified the impact of the invalid signature in Section 11.1.

10.2.3.5. X25519

The X25519 key exchange is an alternative to ECDH for establishing a shared symmetric key. It is implemented in Paramiko by the `KexCurve25519` class. There are three steps: generating a new ephemeral key pair, sending the public key to the peer, and receiving the public key of the peer to perform the exchange.

A new ephemeral key pair is generated for each exchange so the key generation function of the `X25519PrivateKey` class from `cryptography` is called directly by `KexCurve25519.start_exchange()`. This is tested in Section 10.2.2.

To test the key exchange using test vectors, we added two crypto-condor harnesses, one from the perspective of the client and the other from the server. `crypto-condor` calls the function with a private key and a peer public key, and compares the returned value with the shared secret from the test vector. To use the given private key, the `X25519PrivateKey.generate()` method is mocked to return that specific key.

Success

The test passes for both the client and server.

Note that there are **acceptable** tests that fail: this is expected, as Wycheproof includes tests for cases that are not forbidden by the RFC but are rejected by some implementations. See the Flag notes.

10.3. Conclusion

Overall, the tests developed by Paramiko cover most of the happy paths, with the exception of the GSS-API modules. Our suggestions add tests for invalid cases: while they are currently handled well by Paramiko, these types of tests are useful to catch future regressions. Moreover, `crypto-condor` harnesses give additional assurance on the correctness of the cryptographic primitives implemented by PYCA Cryptography.

11. Dynamic testing

To perform dynamic tests based on the results of the static analysis in Section 9, we created a base server configuration to be modified for each specific test (see [here](#)). We used OpenSSH 9.9 as the client.

When we were setting up a base server, we noticed the following documentation error:

INFO-26	Wrong documented type in <code>check_channel_exec_request</code>
Perimeter	<code>paramiko/server.py</code>
Description	
The documentation indicates that the <code>command</code> argument is of type <code>str</code> but it actually is of type <code>bytes</code> .	
Recommendation	
Update the docstring to indicate the correct type.	

11.1. Invalid Ed25519 signature size

From [LOW-25](#), if an Ed25519 signature is not exactly 64 bytes long, `pynacl` raises an exception that is not handled by Paramiko.

LOW-27	Invalid Ed25519 signature cause transport thread to crash		
Likelihood		Impact	
Perimeter	paramiko/ed25519key.py		
Prerequisites	A working Paramiko SSH server and a SSH client that can generate invalid signatures.		
Description			
To reproduce, start a Paramiko server. The script in Appendix C.2 creates a client that generates Ed25519 signatures that are not 64 bytes by modifying the <code>sign_ssh_data</code> method. Using this client to connect to a Paramiko server crashes the transport thread, but does not cause the server script to exit.			
Recommendation			

Handle the exception: either catch the `nacl.exception.ValueError` exception or check that the signature has the correct length before calling `verify()`.

11.2. DES keys

As noted in Section 9.3.1.4, `cryptography` appears to accept 8- and 16-byte keys for `TripleDES`. We verified that `TripleDES` accepts keys that are not 24 bytes long, as identified during static analysis.

HIGH-28		Insecure key sizes accepted for Triple DES	
Likelihood		Impact	
Perimeter	TripleDES in Cryptography		
Description			
Appendix C.6 creates instances of TripleDES using various key sizes. As expected, 24-byte keys are accepted. However, 8-byte and 16-byte keys are also accepted and expanded. Following NIST SP 800-67 Rev.2, Section 3.4, these key sizes should be rejected. For comparison, the script also tries to generate a TripleDES instance from a 20-byte key, which fails with ValueError.			
Recommendation			
Reject key sizes that are not 24 bytes.			

11.3. RSA keys

From our static analysis of `RSAPrivateKey` in Section 9.1.13, it appears that `RSAPrivateKey` can be instantiated with keys that are fewer than 2048 bits long. Appendix C.3 uses both a 512-bit and a 1024-bit key to instantiate `RSAPrivateKey` and sign an arbitrary message. The expected result is that `RSAPrivateKey` should fail to instantiate, but the instance is created successfully and the message is signed. Since `RSAPrivateKey` internally uses `RSAPrivateKey` from `cryptography`, it also means that `cryptography` does not reject these keys.

To test the impact in the context of an SSH server based on Paramiko, we created two server instances using each of the keys (see Appendix C.4 and Appendix C.5) and try to connect with the OpenSSH client. The server with the 512-bit key encountered an issue during the key exchange, as evidenced by the following exception:

```

1 ERROR:paramiko.transport:Unknown exception: Digest or salt length too long for
  key size. Use a larger key or shorter salt length if you are specifying a PSS
  salt
2 ERROR:paramiko.transport:Traceback (most recent call last):
3 ERROR:paramiko.transport:  File "/dynamic_testing/.venv/lib64/python3.13/site-
  packages/paramiko/transport.py", line 2222, in run
4 ERROR:paramiko.transport:      self.kex_engine.parse_next(ptype, m)
5 ERROR:paramiko.transport:      ~~~~~^~~~~~
6 ERROR:paramiko.transport:  File "/dynamic_testing/.venv/lib64/python3.13/site-
  packages/paramiko/kex_curve25519.py", line 62, in parse_next
7 ERROR:paramiko.transport:      return self._parse_kexecdh_init(m)
8 ERROR:paramiko.transport:      ~~~~~^
9 ERROR:paramiko.transport:  File "/dynamic_testing/.venv/lib64/python3.13/site-
  packages/paramiko/kex_curve25519.py", line 92, in _parse_kexecdh_init
10 ERROR:paramiko.transport:      sig =
    self.transport.get_server_key().sign_ssh_data(
11 ERROR:paramiko.transport:          H, self.transport.host_key_type
12 ERROR:paramiko.transport:      )
13 ERROR:paramiko.transport:  File "/dynamic_testing/.venv/lib64/python3.13/site-
  packages/paramiko/rsa.py", line 125, in sign_ssh_data
14 ERROR:paramiko.transport:      sig = self.key.sign(
15 ERROR:paramiko.transport:          data,
16 ERROR:paramiko.transport:      ...<3 lines>...
17 ERROR:paramiko.transport:          algorithm=self.HASHES[algorithm](),
18 ERROR:paramiko.transport:      )

```

The key is used with `rsa-sha2-512`, which generates a digest that is too big for the key. We disabled this algorithm to force the server to use `rsa-sha2-256`. The key exchange starts without errors but OpenSSH refuses to connect with `ssh_dispatch_run_fatal: Connection to 127.0.0.1 port 2222: Invalid key length`.

The server with the 1024-bit key starts correctly and OpenSSH is able to connect to it, provided that the `RequiredRSASize` option is not set to 1024.

LOW-29	Insecure RSA key size allowed		
Likelihood		Impact	
Perimeter	RSA Keys in Paramiko and Cryptography		
Prerequisites	Valid 512-bit and 1024-bit RSA keys.		
Description			
RSAPrivateKey, and by extension RSAPrivateKey from cryptography, accept 512-bit and 1024-bit RSA keys. Per NIST SP 800-131A Rev.2, these key sizes are insecure and should not be used at all. In the case of Paramiko, this issue is partially mitigated by the fact that the 512-bit key is too short to sign the 512-bit digest generated by the default <code>rsa-sha2-512</code> algorithm and because OpenSSH (version 9.9 at least) rejects host keys that are shorter than 1024 bits.			
Recommendation			
Reject RSA keys that are shorter than 2048 bits.			

11.4. Constant-timeness

Cryptographic implementations should be constant time to prevent timing side-channel attacks, where an attacker can infer secret information—such as encryption keys—by measuring how long operations take to execute. Variations in execution time caused by data-dependent branches or memory accesses can unintentionally reveal patterns linked to sensitive values. By ensuring that all operations run in constant time, regardless of input data, implementations can protect against these subtle but powerful attacks and maintain the confidentiality and integrity of cryptographic systems.

To test for the use of sensitive material in conditional branches, we used [Pysa](#), the Python Static Analyzer feature of Pyre, a Python type-checker. This analyzer implements **taint analysis**: it tracks values that come from user-defined sources to check if they can reach a “dangerous location” called sink.

While Pysa generally deals with functions as sinks, it can also use conditional branches as [implicit sinks](#). This fits our use-case: we identified all sensitive values (static and ephemeral keys, passwords), marked them as tainted sources, and used Pysa to verify if any of those values ends up in a conditional branch.

The current approach suffers from two drawbacks:

- The implicit sinks feature relies only on *forward analysis*, while explicit sinks also rely on *backward analysis*. From our testing, forward analysis misses cases in which the conditional branch is not in the same module as the tainted source while backward analysis can find them.
- Most keys are not raw byte strings but are opaque attributes of a class (particularly in `cryptography` keys, which come from the `rust_openssl` bindings). This can result in many false positives, as we can only mark the class as tainted, which is flagged whenever the class itself or an innocuous method is used in a conditional branch, but the key material is not exposed.

The following sections indicate the instances found by Pysa: all are false positives. Illustrating the second drawback, these instances are mostly false positives due to the use of the class itself in a conditional branch (e.g., checking if the object has a particular type). However one instance is notable: in Section 11.4.3, the result of the key exchange is used in a (constant-time) comparison, and has been flagged because the `key` attribute of `KexCurve25519` is tainted. This showcases the power of taint analysis for sensitive variable propagation.

To reproduce these results, we have included the configuration in Appendix D. The paths are relative to the root of Paramiko. Both `cryptography` and `pynacl` should be downloaded.

```
1 pysa_tests
2 |— paramiko
3 |— cryptography
4 |— pynacl
```

From the root of `paramiko`, run `pyre analyze` to launch Pysa, the results are displayed at the end in JSON format.

11.4.1. agent.py

```
458     def asbytes(self):
459         # Prefer inner_key.asbytes, since that will differ for eg RSA-CERT
460         return self.inner_key.asbytes() if self.inner_key else self.blob
461     ...
462     ...
465     def get_bits(self):
466         # Have to work around PKey's default get_bits being crap
467         if self.inner_key is not None:
468             return self.inner_key.get_bits()
469         return super().get_bits()
470
471     def __getattr__(self, name):
472         """
473         Proxy any un-implemented methods/properties to the inner_key.
474         """
475         if self.inner_key is None: # nothing to proxy to
476             raise AttributeError(name)
477         return getattr(self.inner_key, name)
478
479     @property
480     def _fields(self):
481         fallback = [self.get_name(), self.blob]
482         return self.inner_key._fields if self.inner_key else fallback
```

Success

All four cases check whether `AgentKey` has the `inner_key` attribute set, the key material is not used in a conditional branch.

11.4.2. auth_strategy

```
115     def __repr__(self):
116         # NOTE: most of interesting repr-bits for private keys is in PKey.
117         # TODO: tacking on agent-ness like this is a bit awkward, but, eh?
118         rep = super()._repr(pkey=self.pkey)
119         if isinstance(self.pkey, AgentKey):
120             rep += " [agent]"
121         return rep
```

Success

This checks that the key of the `InMemoryPrivateKey` class is an `AgentKey`, the key material is not used in a conditional branch.

11.4.3. `kex_curve25519.py`

```
36     def _perform_exchange(self, peer_key):
37         secret = self.key.exchange(peer_key)
38         if constant_time.bytes_eq(secret, b"\x00" * 32):
39             raise SSHException(
40                 "peer's curve25519 public value has wrong order"
41             )
42         return secret
```

Success

The shared secret that results from the X25519 exchange is used in a conditional branch. However, the `constant_time.bytes_eq` function from `cryptography` is used.

11.4.4. `rsakey.py`

There are two hits in `paramiko/rsakey.py`.

```
91     def public_numbers(self):
92         if isinstance(self.key, rsa.RSAPrivateKey):
93             return self.key.private_numbers().public_numbers
94         else:
95             return self.key.public_numbers()
```

```
138    def verify_ssh_sig(self, data, msg):
139        sig_algorithm = msg.get_text()
140        if sig_algorithm not in self.HASHES:
141            return False
142        key = self.key
143        if isinstance(key, rsa.RSAPrivateKey):
144            key = key.public_key()
```

Success

Both cases are checking if the key is a private key: the key material itself is not used in a conditional branch.

11.5. Differential Fuzzing of PYCA Cryptography

DeltAFLy is Quarkslab’s internal differential fuzzer, developed in Rust and built on top of libAFL for API-level fuzzing. It is specifically designed for testing cryptographic libraries and operations. Inspired by the cryptofuzz project, DeltAFLy addresses several of its limitations through a more modular architecture where each harness is packaged as an independent shared library.

It supports differential fuzzing across multiple languages, including Rust, C, C++, and Java. Furthermore, thanks to libAFL’s QEMU mode, it can also target binary-only implementations. Leveraging libAFL’s Observer modularity, DeltAFLy features a constant-time analysis mode capable of detecting secret-dependent control flow. DeltAFLy notably led to the discovery of vulnerabilities in the HQC post-quantum algorithm implementation within LibOQS.

DeltAFLy operates as a greybox differential fuzzer, combining both coverage feedback and input/output comparison to efficiently guide its exploration. The results of the performed tests are presented in the following sections.

Since PYCA Cryptography is primarily used through binding, we decided to create a few harnesses to ensure the Rust bindings’ correct behavior against OpenSSL. The harnesses were straightforward to implement once the Py03 crate was understood.

Success

Our fuzzing campaign on the following primitives did not show any flaws:

- AES-128/192/256 in CTR mode,
- AES-128/192/256 in CBC mode,
- AES-128/256 in GCM mode,
- 3DES,
- HMAC-SHA2-256/512 for modern security,
- HMAC-SHA1 and HMAC-MD5 for legacy compatibility,
- SHA-256, SHA-384, SHA-512 for ECDSA and modern key exchange,
- SHA-1 for legacy Diffie-Hellman and DSA operations,
- MD5 for fingerprint generation,
- RSA with PKCS#1 v1.5 padding,
- RSA-SHA2-256/512 for modern signatures,
- DSA (Digital Signature Algorithm) for legacy key support,
- P-256, P-384, P-521 curves for modern elliptic curve cryptography,
- Ed25519 for modern elliptic curve signatures.

11.6. Server over UDP

As noted in Section 8.1.2.6, the `Transport` class only verifies that the socket is a TCP one when creating a new instance from a `(host, port)` tuple. Otherwise, if the user passes an

existing socket, it is used directly. We were able to create a server instance that runs on a UDP socket (see Appendix C.7).

LOW-30		Server can be instantiated over UDP socket	
Likelihood		Impact	
Perimeter	paramiko/transport.py		
Description			
The Transport class can be instantiated using an UDP socket instead of TCP one. This is marked as low, as both OpenSSH and Paramiko client are unable to connect to this server.			
Recommendation			
Add a check in Transport.__init__() to verify that sock is a TCP socket.			

11.7. Conclusion

Dynamic testing was performed on a best-effort basis, as the focus of this audit was the code review of Paramiko and PYCA Cryptography. We used it mainly to validate issues found during the static analysis phase of the audit.

This section also includes the usage of Pysa to detect the presence of possible constant-time issues: it showcased the ability of taint analysis to track sensitive material into conditional branches, with only a few results that were all false positives due to how the classes deriving from `PKey` store their sensitive material differently.

12. Build Systems, CI, and Supply chain Review

12.1. Paramiko's CI

CircleCI is a continuous integration and deployment (CI/CD) platform that automates software development cycles. Its operation relies on a configuration file (`.circleci/config.yml`) defining pipelines executed in ephemeral environments (Docker containers or virtual machines), automatically triggered by Git-related events. These pipelines consist of jobs (tests, linting, deployment) orchestrated by workflows, with secrets and environment variables managed through contexts and project-level settings.

A key aspect involves SSH integration. CircleCI supports storing dedicated SSH keys to clone private repositories or manage submodules, with secure injection during pipeline execution. Temporary SSH sessions can also be opened to debug running jobs. Security best practices recommend using CircleCI-specific SSH keys with limited permissions and validating remote hosts via the `known_hosts` mechanism to mitigate man-in-the-middle attacks.

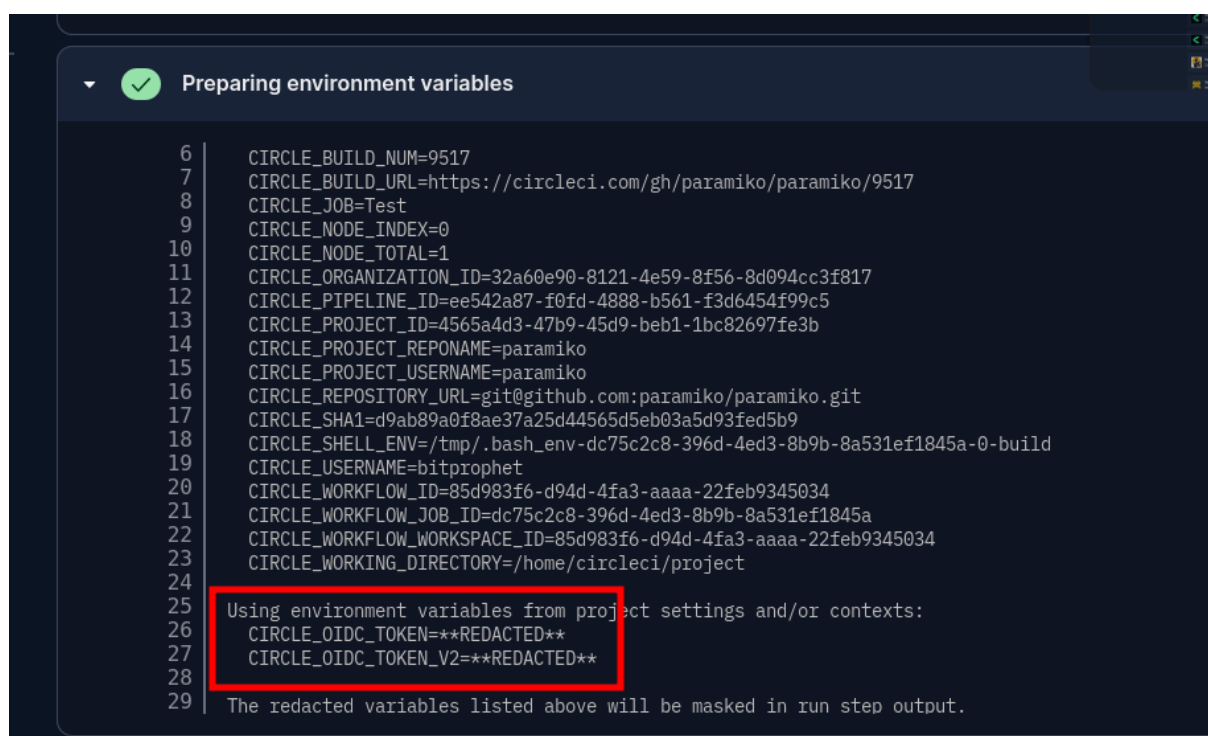
Pull request jobs run inside ephemeral Docker containers without exposed Docker sockets or elevated privileges, providing appropriate isolation. The residual risk is primarily configuration-related (improper injection of secrets, overly verbose logs). Recommended measures include PR/trusted separation, prohibition of secrets in PRs, opt-in debug filtering, and maintaining non-privileged execution.

The project being open source, it is possible to find the CircleCI pipeline on the following URL: <https://app.circleci.com/pipelines/github/paramiko/paramiko>.

However, certain limitations are inherent to CircleCI. For instance, API keys do not support fine-grained permissions: users either have full access or none at all. This lack of granularity makes the compromise of an API key particularly critical.

12.2. Fork to Pull Requests

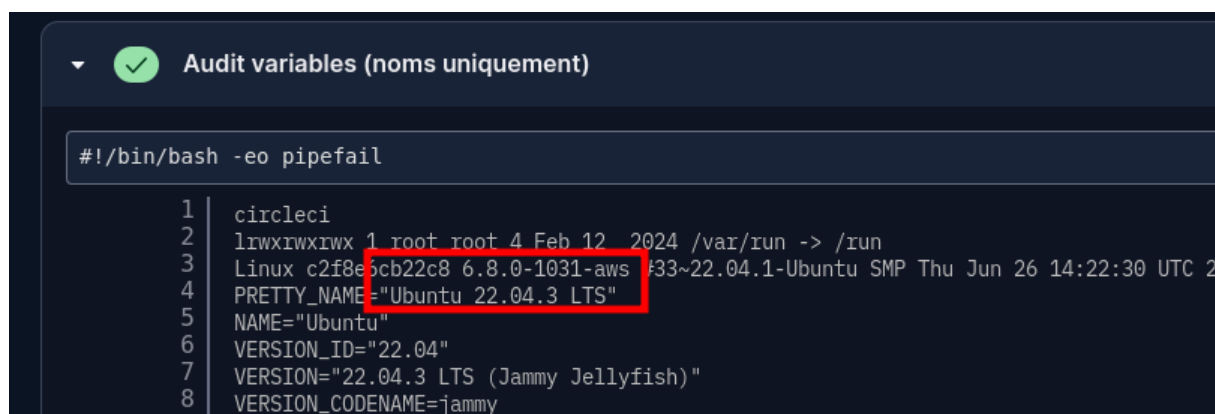
Several attack vectors may nonetheless lead to CI compromise. For instance, it has been demonstrated that pipelines can be triggered through a pull request originating from a fork. In such a scenario, an attacker may execute arbitrary runners. However, these runners are not part of a trusted workflow and therefore do not inherit the OIDC token in their environment variables, unlike workflows classified as trusted.



```
6 CIRCLE_BUILD_NUM=9517
7 CIRCLE_BUILD_URL=https://circleci.com/gh/paramiko/paramiko/9517
8 CIRCLE_JOB=Test
9 CIRCLE_NODE_INDEX=0
10 CIRCLE_NODE_TOTAL=1
11 CIRCLE_ORGANIZATION_ID=32a60e90-8121-4e59-8f56-8d094cc3f817
12 CIRCLE_PIPELINE_ID=ee542a87-f0fd-4888-b561-f3d6454f99c5
13 CIRCLE_PROJECT_ID=4565a4d3-47b9-45d9-beb1-1bc82697fe3b
14 CIRCLE_PROJECT_REPONAME=paramiko
15 CIRCLE_PROJECT_USERNAME=paramiko
16 CIRCLE_REPOSITORY_URL=git@github.com:paramiko/paramiko.git
17 CIRCLE_SHA1=d9ab89a0f8ae37a25d44565d5eb03a5d93fed5b9
18 CIRCLE_SHELL_ENV=/tmp/.bash_env-dc75c2c8-396d-4ed3-8b9b-8a531ef1845a-0-build
19 CIRCLE_USERNAME=bitprophet
20 CIRCLE_WORKFLOW_ID=85d983f6-d94d-4fa3-aaaa-22feb9345034
21 CIRCLE_WORKFLOW_JOB_ID=dc75c2c8-396d-4ed3-8b9b-8a531ef1845a
22 CIRCLE_WORKFLOW_WORKSPACE_ID=85d983f6-d94d-4fa3-aaaa-22feb9345034
23 CIRCLE_WORKING_DIRECTORY=/home/circleci/project
24
25 Using environment variables from project settings and/or contexts:
26 CIRCLE_OIDC_TOKEN=**REDACTED**
27 CIRCLE_OIDC_TOKEN_V2=**REDACTED**
28
29 The redacted variables listed above will be masked in run step output.
```

Figure 27: CircleCI OIDC's credentials.

An attacker could nevertheless execute arbitrary commands to gather information about the environment used by Paramiko and CircleCI, for example the runner's operating system and kernel version (see Figure 28).



```
#!/bin/bash -eo pipefail
1 circleci
2 lrwxrwxrwx 1 root root 4 Feb 12 2024 /var/run -> /run
3 Linux c2f8e5cb22c8 6.8.0-1031-aws #33~22.04.1-Ubuntu SMP Thu Jun 26 14:22:30 UTC 2
4 PRETTY_NAME="Ubuntu 22.04.3 LTS"
5 NAME="Ubuntu"
6 VERSION_ID="22.04"
7 VERSION="22.04.3 LTS (Jammy Jellyfish)"
8 VERSION_CODENAME=jammy
```

Figure 28: Leakage of operating system and kernel version.

It was not possible to further exploit CircleCI's untrusted pipelines. These mechanisms therefore appear to be properly implemented. However, an attacker could still execute arbitrary code on a third-party runner, see for instance Figure 29.

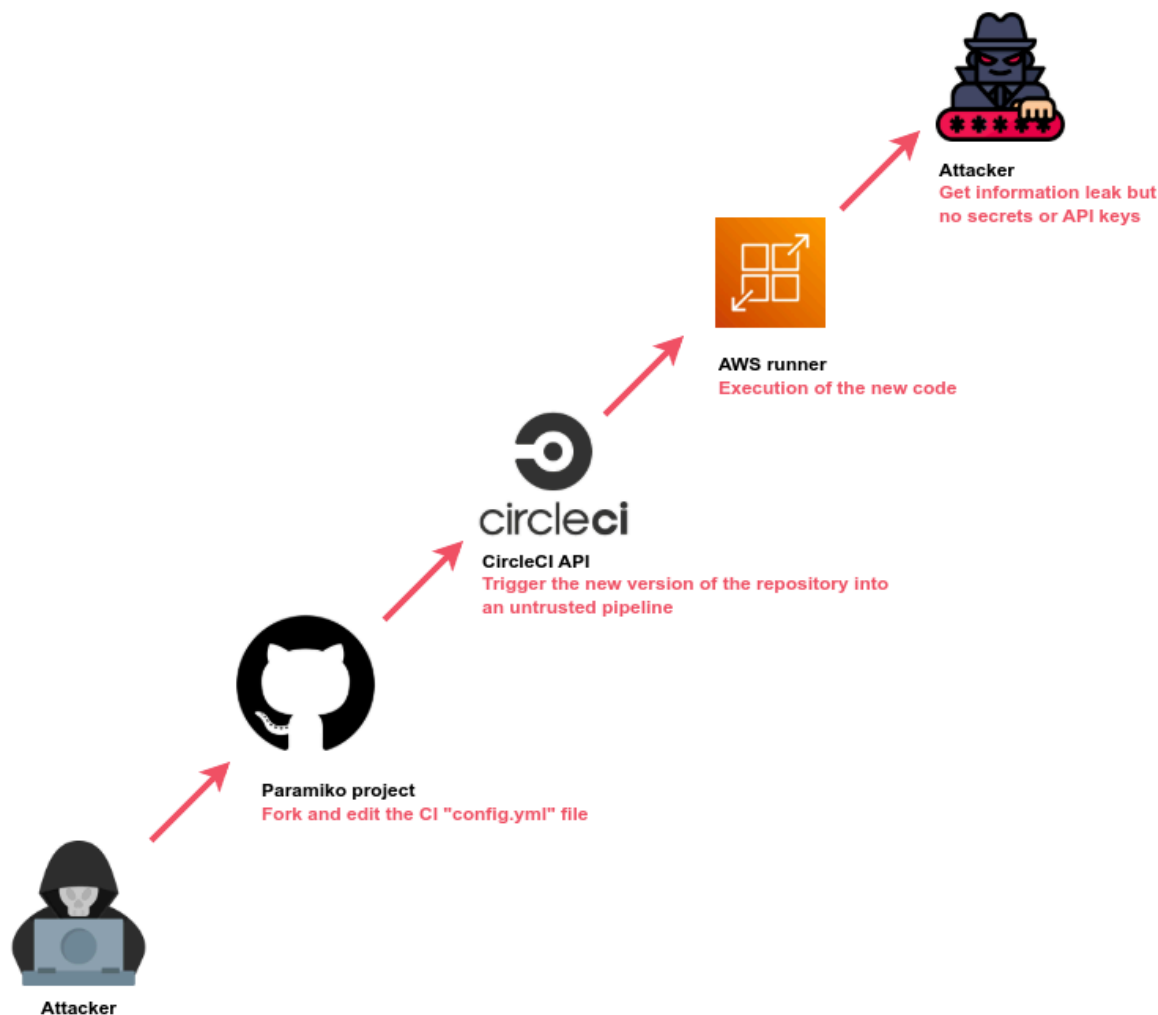


Figure 29: Execution of arbitrary code on a third-party runner.

Runners being time-limited, attack scenarios involving heavy workloads such as crypto-miners or other resource-intensive programs that would incur significant costs to the organization were not considered.

From Figure 28, the kernel version in use appears out of date and includes a number of known vulnerabilities (CVEs), one of which allows privilege escalation, such as obtaining elevated privileges illegitimately. It is possible to determine that the runners make use of the Docker service. Containerization strongly isolates privileges between the host and the container, but the Docker daemon and containers still rely on the host kernel. Historically, there have been exploits that allowed escape from a container context via Linux kernel vulnerabilities.

To date there is no publicly known, reliable exploit enabling such an escape from a Docker container in this environment. However, a motivated and skilled attacker could adapt public exploits or develop a bespoke one. A plausible attack chain is: an attacker forks the Paramiko GitHub repository, modifies the CircleCI configuration to include a tailored exploit, and opens a pull request; the modified pipeline is then executed by Paramiko's CircleCI. If the exploit succeeds, the attacker could execute commands on the runner host, potentially exfiltrating

OIDC tokens, credentials, or installing a persistent backdoor. With such access, the attacker could inject a backdoor into the official Paramiko repository and thereby distribute the vulnerability to any systems that update the compromised release.

During the audit, such an attempt was detected by the development teams and the attacker's account was banned, demonstrating good operational responsiveness.

12.3. Supply Chain Attacks

It is also possible to target Paramiko through its dependencies, in particular the PYCA/Cryptography library. Paramiko has a single primary maintainer, which reduces the likelihood of supply-chain abuse originating from its own contributor base. Cryptography, on the other hand, is maintained by a significantly larger community with more than 300 contributors, which increases the potential attack surface (see Figure 30).

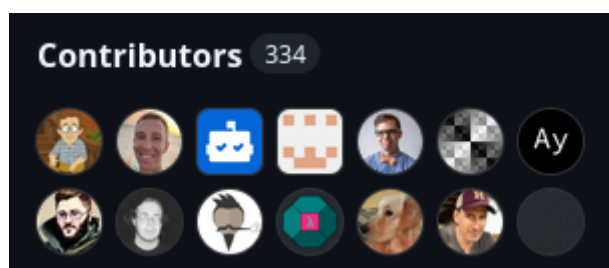


Figure 30: Contributors of PYCA Cryptography.

A script was developed to enumerate a project's Python dependencies and assess whether they contain known vulnerabilities (see Appendix A). For both Paramiko and Cryptography, the scan did not reveal any vulnerable dependencies. Nevertheless, an attacker could infiltrate the contributor community of Cryptography and inject malicious code that would subsequently propagate to Paramiko through its direct dependency.

Several high-profile supply-chain compromises in recent years illustrate this threat model. The XZ Utils backdoor campaign, for example, involved the introduction of malicious code into a widely used compression library following a long-term infiltration of the project's trust perimeter. This case demonstrates how a persistent and targeted campaign against a dependency can enable remote code execution or privilege escalation on downstream systems.

Mitigation measures include regular dependency audits, strict peer reviews, multi-person approval for modifications to build or release tools, minimizing direct dependency exposure where possible, and monitoring contributors' activity for signs of sock puppet accounts or long-term infiltration attempts.

12.4. Conclusion

During the audit of the Paramiko and Cryptography repositories, it was determined that the configuration files do not contain significant vulnerabilities. The main risk identified concerns

supply chain attacks. Paramiko has a small number of external dependencies, while libraries like Cryptography have many contributors, increasing the overall exposure.

It was also observed that arbitrary commands can be executed in the Paramiko pipeline when forking the project. However, these executions take place within an untrusted pipeline. An attacker could attempt to exploit potential vulnerabilities in the runners to escape their environment, but would not be able to access sensitive secrets such as the CircleCI API key.

The tests have finally confirmed that the operational teams quickly responded to malicious behavior on CircleCI, effectively blocking unsophisticated attempts. You can find the full list of tests conducted on the infrastructure here:

- <https://github.com/paramiko/paramiko/pull/2546/files>
- <https://app.circleci.com/pipelines/github/paramiko/paramiko/958/workflows/865d97ce-251e-41b2-8e61-37b684b1ccaa/jobs/9648>

A. Dependencies script

```
#!/usr/bin/env python3
"""
Dependency Mapper & Freshness Auditor for Python Packages
-----
- Prompts for a root package name and a max depth (e.g., 10, 20, 30).
- Builds a dependency graph from the *installed environment* (actual supply chain).
- Correlates each node with the *installed version* and the *latest version on PyPI*.
- Exports:
  * A PNG graph (matplotlib) with labels showing versions and freshness status.
  * A CSV and JSON report listing all nodes with installed/latest/behind-by.
  * A DOT file (Graphviz) for further rendering if desired.

Notes
-----
- The graph follows what your environment *actually imports* based on installed
distributions.
- If a dependency isn't installed, it's still added as a node (installed_version = N/A)
but will still fetch the latest version from PyPI if possible.
- For robust parsing of requirements and markers, we use `packaging`.
"""

import sys
import json
import csv
import subprocess
import textwrap
from pathlib import Path
from typing import Dict, Set, Tuple, List, Optional

# --- Bootstrap deps if missing ---
def ensure(package: str, import_name: Optional[str] = None):
    try:
        __import__(import_name or package)
    except Exception:
        print(f"[setup] Installing missing dependency: {package} ...",
              file=sys.stderr)
        subprocess.check_call([sys.executable, "-m", "pip", "install", package])

for pkg, name in [
    ("packaging", "packaging"),
    ("requests", "requests"),
    ("networkx", "networkx"),
    ("matplotlib", "matplotlib"),
]:
    ensure(pkg, name)

# After ensuring, import them
import requests
```

```

import networkx as nx
from packaging.requirements import Requirement
from packaging.markers import Marker
from packaging.utils import canonicalize_name
from packaging.version import Version, InvalidVersion

# importlib.metadata is stdlib in 3.8+
try:
    import importlib.metadata as importlib_metadata
except Exception:
    import importlib_metadata # type: ignore

import matplotlib
matplotlib.use("Agg") # headless
import matplotlib.pyplot as plt # noqa: E402

# --- Utility functions ---
def prompt_nonempty(prompt: str) -> str:
    while True:
        val = input(prompt).strip()
        if val:
            return val

def yesno(prompt: str, default: bool = True) -> bool:
    suffix = " [Y/n] " if default else " [y/N] "
    ans = input(prompt + suffix).strip().lower()
    if not ans:
        return default
    return ans in ("y", "yes")

def get_installed_distributions() -> Dict[str, str]:
    """Return map canonical_name -> installed_version"""
    result = {}
    for dist in importlib_metadata.distributions():
        name = canonicalize_name(dist.metadata["Name"] if dist.metadata and "Name" in
dist.metadata else dist.metadata["Summary"])
        version = dist.version
        result[name] = version
    return result

def get_requires_for_dist(name: str) -> List[str]:
    """Return raw requirement strings from installed metadata for `name` if
installed; else empty list."""
    try:
        dist = importlib_metadata.distribution(name)
    except importlib_metadata.PackageNotFoundError:
        return []
    requires = dist.requires or []
    return list(requires)

def evaluate_marker(req: Requirement) -> bool:

```

```

    """Return True if environment markers evaluate to True in the current
    environment."""
    if req.marker is None:
        return True
    try:
        return req.marker.evaluate()
    except Exception:
        return True # be permissive

def parse_dependencies(raw_requires: List[str]) -> List[str]:
    """Parse requires_dist lines and return list of dependency project names that
    apply to current env (ignores extras/markers not matching)."""
    deps: List[str] = []
    for raw in raw_requires:
        try:
            r = Requirement(raw)
        except Exception:
            # malformed requirement line; skip
            continue
        if not evaluate_marker(r):
            continue
        deps.append(canonicalize_name(r.name))
    return deps

def latest_on_pypi(project: str) -> Optional[str]:
    url = f"https://pypi.org/pypi/{project}/json"
    try:
        r = requests.get(url, timeout=10)
        if r.status_code != 200:
            return None
        data = r.json()
        ver = data.get("info", {}).get("version")
        if ver:
            return ver
    except Exception:
        return None
    return None

def version_cmp(installed: Optional[str], latest: Optional[str]) -> Tuple[bool,
Optional[int]]:
    """
    Return (is_outdated, behind) where behind is None if cannot compute.
    We don't compute the numeric distance between versions (semantic), but flag
    outdated via PEP 440 comparison.
    """
    if not installed or not latest:
        return (False, None)
    try:
        vi = Version(installed)
        vl = Version(latest)
        return (vi < vl, None)
    except InvalidVersion:

```

```

        return (False, None)

def build_graph(root: str, max_depth: int, installed_map: Dict[str, str]) ->
nx.DiGraph:
    G = nx.DiGraph()
    root_canon = canonicalize_name(root)
    # BFS
    queue: List[Tuple[str, int]] = [(root_canon, 0)]
    seen: Set[str] = set()
    while queue:
        node, depth = queue.pop(0)
        if node in seen:
            continue
        seen.add(node)
        G.add_node(node)
        if depth >= max_depth:
            continue

        # Get dependencies from installed metadata if present
        child_names: List[str] = []
        # importlib.metadata expects original name, not canonicalized in lookup
        # Try best-effort: find any installed dist matching canonical name.
        lookup_name = None
        for n, v in installed_map.items():
            if canonicalize_name(n) == node:
                lookup_name = n
                break
        raw = get_requires_for_dist(lookup_name or node)
        child_names = parse_dependencies(raw)

        for dep in child_names:
            G.add_edge(node, dep)
            queue.append((dep, depth + 1))
    return G

def annotate_versions(G: nx.DiGraph, installed_map: Dict[str, str]) -> Dict[str,
Dict[str, Optional[str]]]:
    meta: Dict[str, Dict[str, Optional[str]]] = {}
    for n in G.nodes:
        installed_version = installed_map.get(n) or installed_map.get(n.replace("-",
"_")) # best effort
        latest_version = latest_on_pypi(n)
        outdated, _ = version_cmp(installed_version, latest_version)
        meta[n] = {
            "installed": installed_version,
            "latest": latest_version,
            "outdated": "yes" if outdated else "no",
        }
    return meta

def draw_graph(G: nx.DiGraph, meta: Dict[str, Dict[str, Optional[str]]], png_path:
Path, title: str):

```

```

plt.figure(figsize=(12, 10))
pos = nx.spring_layout(G, seed=42)
labels = {}
for n in G.nodes:
    inst = meta[n]["installed"] or "N/A"
    lat = meta[n]["latest"] or "N/A"
    outdated = meta[n]["outdated"]
    lab = f"{n}\n{inst} → {lat}"
    if outdated == "yes":
        lab += "\nOUTDATED"
    labels[n] = lab

nx.draw_networkx_nodes(G, pos)
nx.draw_networkx_edges(G, pos, arrows=True, arrowstyle='->', arrowsize=10)
nx.draw_networkx_labels(G, pos, labels=labels, font_size=8)

plt.title(title)
plt.axis('off')
plt.tight_layout()
plt.savefig(png_path) # No explicit colors/styles as requested
plt.close()

def export_reports(G: nx.DiGraph, meta: Dict[str, Dict[str, Optional[str]]],
out_prefix: Path):
    # CSV
    csv_path = out_prefix.with_suffix(".csv")
    with csv_path.open("w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["package", "installed", "latest", "outdated"])
        for n in sorted(G.nodes):
            w.writerow([n, meta[n]["installed"] or "", meta[n]["latest"] or "",
meta[n]["outdated"]])
    # JSON
    json_path = out_prefix.with_suffix(".json")
    with json_path.open("w", encoding="utf-8") as f:
        json.dump({"nodes": meta, "edges": list(G.edges())}, f, indent=2)
    # DOT
    try:
        from networkx.drawing.nx_pydot import write_dot
        dot_path = out_prefix.with_suffix(".dot")
        write_dot(G, dot_path)
    except Exception:
        # pydot not installed; skip DOT
        pass

def main():
    print(textwrap.dedent("""
    === Python Dependency Mapper & Freshness Auditor ===
    - Analyse installed env(what your jobs really use).
    - Compare each packet to the latest version on PyPI.
    - Generate a graph PNG + CSV + JSON (+ DOT if pydot installed).
    """))

```



```

root = prompt_nonempty("Name of root packet (ex: paramiko) : ").strip()
try:
    depth = int(prompt_nonempty("Max depth (ex: 10, 20, 30) : ").strip())
    if depth < 0:
        raise ValueError
except Exception:
    print("Invalid depth, default value = 10", file=sys.stderr)
    depth = 10

out_dir = Path.cwd() / "depmap_out"
out_dir.mkdir(parents=True, exist_ok=True)

installed_map = get_installed_distributions()

G = build_graph(root, depth, installed_map)
if not G.nodes:
    print(f"No nodes found: packet '{root}' is probably not installed on this
env.", file=sys.stderr)
    print("Tip: instal-it (ex: `pip install paramiko`) then rerun it.",
file=sys.stderr)
    sys.exit(2)

meta = annotate_versions(G, installed_map)

prefix = out_dir / f"{canonicalize_name(root)}_depth{depth}"
png_path = prefix.with_suffix(".png")
draw_graph(G, meta, png_path, f"Dependencies of {root} (depth {depth})")

export_reports(G, meta, prefix)

# Résumé console
outdated = [n for n in G.nodes if meta[n]['outdated'] == 'yes']
print(f"[OK] Graph: {png_path}")
print(f"[OK] Reports: {prefix.with_suffix('.csv')},
{prefix.with_suffix('.json')}")
if outdated:
    print(f"[INFO] Packets that are potentially outdated ({len(outdated)}): {'',
'.join(sorted(outdated))}")
else:
    print("[INFO] All graph packets are up-to-date according to PyPI.")

if __name__ == "__main__":
    main()

```

Listing 14: `deps.py`

B. Patch to fix GSS-API tests

```
diff --git a/.coveragerc b/.coveragerc
index 90c7ab09..7477b7cb 100644
--- a/.coveragerc
+++ b/.coveragerc
@@ -4,3 +4,4 @@ include =
     paramiko/*
     tests/*
omit = paramiko/_winapi.py
+dynamic_context = test_function
diff --git a/tests/test_gssapi.py b/tests/test_gssapi.py
index da62fd97..9b096931 100644
--- a/tests/test_gssapi.py
+++ b/tests/test_gssapi.py
@@ -54,7 +54,7 @@ def test_pyasn1(self):

     oid = encoder.encode(ObjectIdentifier(self.krb5_mech))
     mech, __ = decoder.decode(oid)
-    self.assertEqual(self.krb5_mech, mech.__str__())
+    self.assertEqual(self.krb5_mech, mech.__str__())

def _gssapi_ssapi_test(self):
    """
@@ -107,27 +107,27 @@ def _gssapi_ssapi_test(self):
    if self.server_mode:
        c_token = gss_ctxt.step(c_token)
        gss_ctxt_status = gss_ctxt.established
-        self.assertEqual(False, gss_ctxt_status)
+        self.assertEqual(False, gss_ctxt_status)
        # Accept a GSS-API context.
        gss_srv_ctxt = gssapi.AcceptContext()
        s_token = gss_srv_ctxt.step(c_token)
        gss_ctxt_status = gss_srv_ctxt.established
-        self.assertNotEquals(None, s_token)
-        self.assertEqual(True, gss_ctxt_status)
+        self.assertNotEqual(None, s_token)
+        self.assertEqual(True, gss_ctxt_status)
        # Establish the client context
        c_token = gss_ctxt.step(s_token)
-        self.assertEqual(None, c_token)
+        self.assertEqual(None, c_token)
    else:
        while not gss_ctxt.established:
            c_token = gss_ctxt.step(c_token)
-            self.assertNotEquals(None, c_token)
+            self.assertNotEqual(None, c_token)
        # Build MIC
        mic_token = gss_ctxt.get_mic(mic_msg)
```

```

        if self.server_mode:
            # Check MIC
            status = gss_srv_ctxt.verify_mic(mic_msg, mic_token)
-           self.assertEqual(0, status)
+           self.assertEqual(0, status)
    elif _API == "PYTHON-GSSAPI-NEW":
        if self.server_mode:
            gss_flags = (
@@ -157,27 +157,27 @@ def _gssapi_ssapi_test(self):
            if self.server_mode:
                c_token = gss_ctxt.step(c_token)
                gss_ctxt_status = gss_ctxt.complete
-               self.assertEqual(False, gss_ctxt_status)
+               self.assertEqual(False, gss_ctxt_status)
                # Accept a GSS-API context.
                gss_srv_ctxt = gssapi.SecurityContext(usage="accept")
                s_token = gss_srv_ctxt.step(c_token)
                gss_ctxt_status = gss_srv_ctxt.complete
-               self.assertNotEquals(None, s_token)
-               self.assertEqual(True, gss_ctxt_status)
+               self.assertNotEqual(None, s_token)
+               self.assertEqual(True, gss_ctxt_status)
                # Establish the client context
                c_token = gss_ctxt.step(s_token)
-               self.assertEqual(None, c_token)
+               self.assertEqual(None, c_token)
            else:
                while not gss_ctxt.complete:
                    c_token = gss_ctxt.step(c_token)
-                   self.assertNotEquals(None, c_token)
+                   self.assertNotEqual(None, c_token)
            # Build MIC
            mic_token = gss_ctxt.get_signature(mic_msg)

        if self.server_mode:
            # Check MIC
            status = gss_srv_ctxt.verify_signature(mic_msg, mic_token)
-           self.assertEqual(0, status)
+           self.assertEqual(0, status)
    else:
        gss_flags = (
            sspicon.ISC_REQ_INTEGRITY
@@ -192,7 +192,7 @@ def _gssapi_ssapi_test(self):
        if self.server_mode:
            error, token = gss_ctxt.authorize(c_token)
            c_token = token[0].Buffer
-           self.assertEqual(0, error)
+           self.assertEqual(0, error)
            # Accept a GSS-API context.
            gss_srv_ctxt = ssapi.ServerAuth("Kerberos", spn=target_name)
            error, token = gss_srv_ctxt.authorize(c_token)
@@ -200,8 +200,8 @@ def _gssapi_ssapi_test(self):

```

```

        # Establish the context.
        error, token = gss_ctxt.authorize(s_token)
        c_token = token[0].Buffer
-       self.assertEqual(None, c_token)
-       self.assertEqual(0, error)
+       self.assertEqual(None, c_token)
+       self.assertEqual(0, error)
        # Build MIC
        mic_token = gss_ctxt.sign(mic_msg)
        # Check MIC
@@ -209,7 +209,7 @@ def _gssapi_ssapi_test(self):
        else:
            error, token = gss_ctxt.authorize(c_token)
            c_token = token[0].Buffer
-            self.assertNotEquals(0, error)
+            self.assertNotEqual(0, error)

    def test_gssapi_ssapi_client(self):
        """
diff --git a/tests/test_kex_gss.py b/tests/test_kex_gss.py
index a81b1959..0fe521c7 100644
--- a/tests/test_kex_gss.py
+++ b/tests/test_kex_gss.py
@@ -113,11 +113,11 @@ def _test_gsskex_and_auth(self, gss_host, rekey=False):
    )

    self.event.wait(1.0)
-    self.assert_(self.event.is_set())
-    self.assert_(self.ts.is_active())
-    self.assertEqual(self.username, self.ts.get_username())
-    self.assertEqual(True, self.ts.is_authenticated())
-    self.assertEqual(True, self.tc.get_transport().gss_kex_used)
+    self.assertTrue(self.event.is_set())
+    self.assertTrue(self.ts.is_active())
+    self.assertEqual(self.username, self.ts.get_username())
+    self.assertEqual(True, self.ts.is_authenticated())
+    self.assertEqual(True, self.tc.get_transport().gss_kex_used)

    stdin, stdout, stderr = self.tc.exec_command("yes")
    schan = self.ts.accept(1.0)
@@ -128,10 +128,10 @@ def _test_gsskex_and_auth(self, gss_host, rekey=False):
    schan.send_stderr("This is on stderr.\n")
    schan.close()

-    self.assertEqual("Hello there.\n", stdout.readline())
-    self.assertEqual("", stdout.readline())
-    self.assertEqual("This is on stderr.\n", stderr.readline())
-    self.assertEqual("", stderr.readline())
+    self.assertEqual("Hello there.\n", stdout.readline())
+    self.assertEqual("", stdout.readline())
+    self.assertEqual("This is on stderr.\n", stderr.readline())
+    self.assertEqual("", stderr.readline())

```

```
stdin.close()
stdout.close()
```

Listing 15: Diff patch to fix broken GSS-API tests

C. Dynamic testing scripts

C.1. Base instances

```
#!/usr/bin/env python
"""Base server."""
import logging
import socket
import threading

import paramiko

from common import HOSTKEYS, CLIENT_FINGERPRINTS

# Set up logging globally.
logging.basicConfig(level=logging.DEBUG)
logger = logging.getLogger(__name__)

class BaseServer(paramiko.ServerInterface):
    """Base SSH server.

    This class should have everything available. Other test servers must inherit from
    this class and override the corresponding methods.

    It provides the `listen()` method that creates the socket, loads the host keys,
    and
    creates a channel waiting for authentication and a command.
    """

    def __init__(self):
        self.event = threading.Event()

    def check_channel_request(self, kind, chanid):
        if kind == "session":
            return paramiko.OPEN_SUCCEEDED

    def get_allowed_auths(self, username: str):
        return "publickey,password,keyboard-interactive"

    def check_auth_none(self, username: str):
        return paramiko.AUTH_FAILED

    def check_auth_password(self, username: str, password: str):
        if username == "user" and password == "password":
            return paramiko.AUTH_SUCCESSFUL
        elif username == "2fa_user" and password == "password":
            return paramiko.AUTH_SUCCESSFUL
        else:
            return paramiko.AUTH_FAILED
```

```

def check_auth_publickey(self, username, key):
    if username == "user" and key.fingerprint in CLIENT_FINGERPRINTS:
        return paramiko.AUTH_SUCCESSFUL
    elif username == "2fa_user" and key.fp in CLIENT_FINGERPRINTS:
        return paramiko.AUTH_PARTIALLY_SUCCESSFUL
    else:
        return paramiko.AUTH_FAILED

def check_auth_interactive(self, username: str, submethods: str):
    query = paramiko.server.InteractiveQuery(
        "Keyboard interactive method", "Testing auth_interactive"
    )
    query.add_prompt("Do you like coffee? ")
    return query

def check_auth_interactive_response(self, responses: list[str]):
    return paramiko.AUTH_SUCCESSFUL

def check_auth_gssapi_with_mic(
    self, username: str, gss_authenticated: int = 2, cc_file: str | None = None
):
    if username == "kerberos_user":
        return paramiko.AUTH_SUCCESSFUL
    else:
        return paramiko.AUTH_FAILED

def check_auth_gssapi_keyex(
    self, username: str, gss_authenticated: int = 2, cc_file: str | None = None
):
    if username == "kerberos_user":
        return paramiko.AUTH_SUCCESSFUL
    else:
        return paramiko.AUTH_FAILED

def enable_auth_gssapi(self):
    return True

def check_port_forward_request(self, address: str, port: int):
    return False

def cancel_port_forward_request(self, address: str, port: int):
    return

def check_global_request(self, kind: str, msg: paramiko.Message):
    return False

def check_channel_pty_request(
    self,
    channel: paramiko.Channel,
    term: str,
    width: int,

```

```

        height: int,
        pixelwidth: int,
        pixelheight: int,
        modes,
    ):
        return False

def check_channel_shell_request(self, channel: paramiko.Channel):
    return False

def check_channel_exec_request(self, channel: paramiko.Channel, command: bytes):
    self.event.set()
    cmd = command.decode()
    logger.info("channel_exec request received: %s", cmd)

    if cmd == "banner":
        self.banner(channel)
    elif cmd.startswith("echo "):
        self.echo(channel, cmd)
    else:
        return False
    return True

def check_channel_subsystem_request(self, channel: paramiko.Channel, name: str):
    return False

def check_channel_window_change_request(
    self, channel, width, height, pixelwidth, pixelheight
):
    return False

def check_channel_x11_request(
    self, channel, single_connection, auth_protocol, auth_cookie, screen_number
):
    return False

def check_channel_forward_agent_request(self, channel):
    return False

def check_channel_direct_tcpip_request(self, chanid, origin, destination):
    return paramiko.OPEN_FAILED_ADMINISTRATIVELY_PROHIBITED

def check_channel_env_request(self, channel, name, value):
    return False

def get_banner(self):
    return ("Paramiko test server", "en-US")

def _load_host_keys(self, transport):
    for host_key in HOSTKEYS:
        transport.add_server_key(host_key)

```



```

def listen(self):

    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    sock.bind("", 2222)
    sock.listen(100)
    client, addr = sock.accept()

    t = paramiko.Transport(client)
    t.set_gss_host(socket.getfqdn(""))
    t.load_server_moduli()

    self._load_host_keys(t)

    # Starts the server and negotiates a new session as server. Either returns
    # successfully or raises SSHException.
    t.start_server(server=self)

    chan = t.accept(20)
    if chan is None:
        raise ValueError("No channel")
    # Wait for client to send a command.
    self.event.wait(30.0)
    if not self.event.is_set():
        raise ValueError("Client did not send a command")

    chan.close()
    t.close()

def banner(self, chan: paramiko.Channel):
    chan.send(
        """===== Test server banner =====
We should include some ascii art...
"""
    )

def echo(self, chan: paramiko.Channel, cmd: str):
    _, s = cmd.split("echo ")
    chan.send(s)

if __name__ == "__main__":
    while True:
        try:
            server = BaseServer()
            server.listen()
        except KeyboardInterrupt:
            exit(0)
        except OSError:
            logger.exception("Caught OSError, usually address already in use")
            exit(1)

```

```

except Exception:
    logger.exception("Caught exception from listener")

```

Listing 16: base_server.py

```

#!/usr/bin/env python
"""Base client with all three types of public keys."""

from paramiko import SSHClient, ECDSAKey, RSAKey, Ed25519Key, WarningPolicy

from common import KEY_ED25519, HOST, PORT, USER


class BaseClient:
    """Base SSH client.

    This is the class to inherit from. Any new methods should call _connect() first.

    Args:
        auth_method:
            How to connect to the server. Can be a private key or a password.
        host:
            The server host name.
        port:
            The server port.
        user:
            The username.
    """

    def __init__(
        self,
        auth_method: ECDSAKey | Ed25519Key | RSAKey | str,
        host: str = HOST,
        port: int = PORT,
        user: str = USER,
    ):
        match auth_method:
            case ECDSAKey() | Ed25519Key() | RSAKey() | str():
                self.auth_method = auth_method
            case _:
                raise ValueError(
                    "Invalid auth method (expected private key or password)"
                )
        self.host = host
        self.port = port
        self.user = user

    def _connect(self):
        client = SSHClient()
        client.load_system_host_keys()
        client.set_missing_host_key_policy(WarningPolicy)

        conn_args = {"port": self.port, "username": self.user}

```

```

match self.auth_method:
    case Ed25519Key() | ECDSAKey() | RSAKey():
        conn_args["pkey"] = self.auth_method
    case str():
        conn_args["password"] = self.auth_method

client.connect(self.host, **conn_args)
self.client = client

def exec_command(self, command: str, timeout: int = 15):
    self._connect()
    stdin, stdout, stderr = self.client.exec_command(command, timeout=timeout)
    data = stdout.read(1024)
    print(data.decode())
    stdin.close()
    stdout.close()
    stderr.close()
    self.client.close()

if __name__ == "__main__":
    client = BaseClient(KEY_ED25519)
    client.exec_command("banner")
    client.exec_command("echo This is a string")

```

Listing 17: base_client.py

```

"""Common values for servers and clients."""

```

```

from paramiko import ECDSAKey, Ed25519Key, RSAKey

HOSTKEY_RSA = RSAKey(filename="keys/server_rsa")
HOSTKEY_ECDSA = ECDSAKey(filename="keys/server_ecdsa")
HOSTKEY_ED25519 = Ed25519Key(filename="keys/server_ed25519")
HOSTKEYS = [HOSTKEY_ECDSA, HOSTKEY_ED25519, HOSTKEY_RSA]

CLIENT_RSA_FINGERPRINT = "SHA256:ntdKS9juQ5wathKtzZAI+04T0P7dz71gK2Pi5+kluwE"
CLIENT_ECDSA_FINGERPRINT = "SHA256:1avVrGVfGZBHj5wxbfkyb012cIRoLpkrp31rIzG5em0"
CLIENT_ED25519_FINGERPRINT = "SHA256:jr6puB3cDf5Yf0FdbqRlcjmJonZAszjTIXR0UUdTPBQ"
CLIENT_FINGERPRINTS = [
    CLIENT_RSA_FINGERPRINT,
    CLIENT_ECDSA_FINGERPRINT,
    CLIENT_ED25519_FINGERPRINT,
]

USER = "user"
PASSWORD = "password"
HOST = "127.0.0.1"
PORT = 2222

with open("keys/client_ed25519", "r") as file:

```

```

KEY_ED25519 = Ed25519Key.from_private_key(file, None)
with open("keys/client_ecdsa", "r") as file:
    KEY_ECDSA = ECDSAKey.from_private_key(file, None)
with open("keys/client_rsa", "r") as file:
    KEY_RSA = RSAKey.from_private_key(file, None)

```

Listing 18: common.py

C.2. Bad Ed25519 signature

"""Test for invalid Ed25519 signatures.

This test creates a regular client but replaces the `sign_ssh_data()` method from `Ed25519Key` with one that returns signatures that are not 64 bytes long.

The expected result is that the server has an uncaught exception when performing the authentication.

"""

```

from unittest import mock

```

```

from paramiko import SSHClient, Ed25519Key, Message

```

```

from common import KEY_ED25519

```

```

def _mock_sig(self, data, algorithm=None):
    m = Message()
    m.add_string(b"ssh-ed25519")
    m.add_string(b"\xcc" * 32)
    return m

```

```

with mock.patch.object(Ed25519Key, "sign_ssh_data", new=_mock_sig):
    client = SSHClient()
    client.load_system_host_keys()
    client.connect("127.0.0.1", port=2222, username="user", pkey=KEY_ED25519)

```

Listing 19: test_rsakey_minimum_size.py

C.3. RSA keys

"""Testing if a minimum key size is enforced for RSA keys."""

```

import pytest

```

```

from paramiko import RSAKey

```

```

MSG = b"This is a message."

```

```

def test_512_bits_key():

```

```

    """Imports a 512-bit RSA key.

    Expected:
        The key is not imported.
    """
    with open("keys/client_rsa_512", "r") as file:
        with pytest.raises(Exception):
            key = RSAKey.from_private_key(file)
            sig = key.sign_ssh_data(MSG)
            key.verify_ssh_sig(MSG, sig)

def test_1024_bits_key():
    """Imports a 1024-bit RSA key.

    Expected:
        The key is not imported.
    """
    with open("keys/client_rsa_1024", "r") as file:
        with pytest.raises(Exception):
            key = RSAKey.from_private_key(file)
            sig = key.sign_ssh_data(MSG)
            key.verify_ssh_sig(MSG, sig)

def test_2048_bits_key():
    """Imports a 2048-bit key.

    Expected:
        The key is imported and can be used to sign an arbitrary message.
    """
    file = open("keys/client_rsa", "r")
    key = RSAKey.from_private_key(file)
    sig = key.sign_ssh_data(MSG)
    key.verify_ssh_sig(MSG, sig)
    file.close()

```

Listing 20: test_rsa_key_minimum_size.py

C.4. RSA 512-bit server

```

import logging
import socket

from paramiko import RSAKey, Transport

from base_server import BaseServer

logging.basicConfig(level=logging.DEBUG)
logger = logging.getLogger(__name__)

```

```

class Rsa512Server(BaseServer):

    def _load_host_keys(self, transport):
        with open("keys/server_rsa_512", "r") as file:
            key = RSAKey.from_private_key(file)
            transport.add_server_key(key)

    def listen(self):

        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        sock.bind(("", 2222))
        sock.listen(100)
        client, addr = sock.accept()

        t = Transport(client, disabled_algorithms={"keys": ["rsa-sha2-512"]})
        t.set_gss_host(socket.getfqdn(""))
        t.load_server_moduli()

        self._load_host_keys(t)

        # Starts the server and negotiates a new session as server. Either returns
        # successfully or raises SSHException.
        t.start_server(server=self)

        chan = t.accept(20)
        if chan is None:
            raise ValueError("No channel")
        # Wait for client to send a command.
        self.event.wait(30.0)
        if not self.event.is_set():
            raise ValueError("Client did not send a command")

        chan.close()
        t.close()

if __name__ == "__main__":
    while True:
        try:
            server = Rsa512Server()
            server.listen()
        except KeyboardInterrupt:
            exit(0)
        except OSError:
            logger.exception("Caught OSError, usually address already in use")
            exit(1)
        except Exception:
            logger.exception("Caught exception from listener")

```

Listing 21: test_rsa_key_minimum_size.py

C.5. RSA 1024-bit server

```
import logging
import socket

from paramiko import RSAKey, Transport

from base_server import BaseServer

logging.basicConfig(level=logging.DEBUG)
logger = logging.getLogger(__name__)

class Rsa1024Server(BaseServer):

    def _load_host_keys(self, transport):
        with open("keys/server_rsa_1024", "r") as file:
            key = RSAKey.from_private_key(file)
            transport.add_server_key(key)

if __name__ == "__main__":
    while True:
        try:
            server = Rsa1024Server()
            server.listen()
        except KeyboardInterrupt:
            exit(0)
        except OSError:
            logger.exception("Caught OSError, usually address already in use")
            exit(1)
        except Exception:
            logger.exception("Caught exception from listener")
            Listing 22: test_rsakey_minimum_size.py
```

C.6. TDES keys

```
"""Testing TDES keys."""

import os

import pytest

from cryptography.hazmat.decrepit.ciphers.algorithms import TripleDES

def test_correct_size():
    """Create a new instance from a valid key size.

    Expected:
```

```

        The key is correctly created.
    """
    key = os.urandom(24)
    tdes_key = TripleDES(key)
    assert tdes_key.name == "3DES"

def test_invalid_size():
    """Create a new instance from an invalid key size.

    Expected:
        The key is not created.
    """
    key = os.urandom(20)
    with pytest.raises(ValueError):
        tdes_key = TripleDES(key)

def test_8byte_key():
    """Expanding an 8-byte key to a full-sized TDES key is insecure.

    Expected:
        Instantiation raises an exception because the key size is invalid.
    """
    key = os.urandom(8)
    with pytest.raises(ValueError):
        tdes_key = TripleDES(key)

def test_two_keys():
    """Using two unique keys instead of three is obsolete.

    Expected:
        Instantiation raises an exception because the key size is invalid.
    """
    key = os.urandom(16)
    with pytest.raises(ValueError):
        tdes_key = TripleDES(key)

```

Listing 23: test_tdes_keys.py

C.7. UDP server

```

"""Tests creating a server with an UDP socket.

```

As noted during the static analysis, it seems that Server does not check whether a given socket is actually a TCP socket instead of an UDP one.

```

The expected result is that the server starts listening on an UDP socket.
"""

```



```

import logging
import socket

import paramiko

from base_server import BaseServer
from common import HOSTKEYS

# Set up logging globally.
logging.basicConfig(level=logging.DEBUG)
logger = logging.getLogger(__name__)

class UdpServer(BaseServer):

    def listen(self):

        sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        sock.bind(("", 2222))
        # sock.listen(100)
        # client, addr = sock.accept()
        client, addr = sock.recvfrom(1024)

        t = paramiko.Transport(client)
        t.set_gss_host(socket.getfqdn(""))
        t.load_server_moduli()
        for host_key in HOSTKEYS:
            t.add_server_key(host_key)

        # Starts the server and negotiates a new session as server. Either returns
        # successfully or raises SSHException.
        t.start_server(server=self)

        chan = t.accept(20)
        if chan is None:
            raise ValueError("No channel")
        # Wait for client to send a command.
        self.event.wait(30.0)
        if not self.event.is_set():
            raise ValueError("Client did not send a command")

        chan.close()
        t.close()

if __name__ == "__main__":
    while True:
        try:
            server = UdpServer()
            server.listen()
        except KeyboardInterrupt:

```

```
        exit(0)
    except Exception:
        logger.exception("Caught exception from listener")
```

Listing 24: server_over_udp.py

D. Pysa configuration files

D.1. `.pyre_configuration`

```
{
  "source_directories": [
    ".", "./paramiko"
  ],
  "taint_models_path": [
    "."
  ],
  "search_path": [
    ".", "./paramiko", "../cryptography/src/", "../pynacl/src/"
  ],
  "site_package_search_strategy": "pep561"
}
```

Listing 25: `.pyre_configuration`

D.2. `sinks.pysa`

```
# Holds the keys that are loaded in the agent
agent.AgentSSH._keys: TaintSource[CustomUserControlled] = ...

# blob is raw private key data: may be a certificate that is later parsed into PKey
agent.AgentKey.blob: TaintSource[CustomUserControlled] = ...
# inner_key is a PKey instance loaded from blob
agent.AgentKey.inner_key: TaintSource[CustomUserControlled] = ...

# private_key is a PKey instance
auth_handler.AuthHandler.private_key: TaintSource[CustomUserControlled] = ...

# password is a string
auth_handler.AuthHandler.password: TaintSource[CustomUserControlled] = ...

# pkey is a PKey instance
auth_strategy.OnDiskPrivateKey.pkey: TaintSource[CustomUserControlled] = ...
# pkey is a PKey instance
auth_strategy.InMemoryPrivateKey.pkey: TaintSource[CustomUserControlled] = ...

# signing_key is an EllipticCurvePrivateKey
ecdsakey.ECDSAKey.signing_key: TaintSource[CustomUserControlled] = ...

# ed25519key.Ed25519Key._signing_key is a nacl.signing.SigningKey which contains the
raw
# seed and a _signing_key attribute which is (sk || pk)
nacl.signing.SigningKey._seed: TaintSource[CustomUserControlled] = ...
nacl.signing.SigningKey._signing_key: TaintSource[CustomUserControlled] = ...
```

```

# key is a rsa.RSAPrivateKey or rsa.RSAPublicKey from cryptography
rsakey.RSAKey.key: TaintSource[CustomUserControlled] = ...

# key is a x25519.X25519PrivateKey from cryptography
kex_curve25519.KexCurve25519.key: TaintSource[CustomUserControlled] = ...

# P is ec.EllipticCurvePrivateKey
kex_ecdh_nist.KexNistp256.P: TaintSource[CustomUserControlled] = ...

# x is the ephemeral secret key (scalar)
kex_gex.KexGex.x: TaintSource[CustomUserControlled] = ...
# kex_gex.KexGexSHA256.x not defined as it inherits from KexGex.x

# x is the ephemeral secret key
kex_group1.KexGroup1.x: TaintSource[CustomUserControlled] = ...
# kex_group14.KexGroup14.x not defined as it inherits from KexGroup1.x
# kex_group14.KexGroup14SHA256.x not defined as it inherits from KexGroup14.x
# kex_group16.KexGroup16SHA256.x not defined as it inherits from KexGroup1.x

# x is the ephemeral secret key (scalar)
kex_gss.KexGSSGroup1.x: TaintSource[CustomUserControlled] = ...
# kex_gss.KexGSSGroup14.x not defined as it inherits from KexGSSGroup1.x

# x is the ephemeral secret key (scalar)
kex_gss.KexGSSGex.x: TaintSource[CustomUserControlled] = ...

# K is the root key as bytes, used to derive other session keys
transport.Transport.K: TaintSource[CustomUserControlled] = ...

transport.Transport.session_id: TaintSource[CustomUserControlled] = ...

# toto.KEY: TaintSource[CustomUserControlled] = ...
# def cryptography.hazmat.bindings._rust.openssl.hashes.Hash.update(self, data:
TaintSink[CodeExecution]): ...
#
# def toto._test_sink(x: TaintSink[CodeExecution]): ...

```

Listing 26: sinks.pysa

D.3. taint.config

```

{
  "sources": [
    {
      "name": "CustomUserControlled",
      "comment": "use to annotate user input"
    }
  ],
  "sinks": [
    {
      "name": "ConditionalBranch",

```

```

        "comment": "use to detect conditional branches"
    },
    {
        "name": "CodeExecution",
        "comment": "use to detect branches"
    }
],
"implicit_sinks": {
    "conditional_test": ["ConditionalBranch"]
},

"features": [],

"rules": [
    {
        "name": "Possible timing attack:",
        "code": 683874,
        "sources": [ "CustomUserControlled" ],
        "sinks": [ "ConditionalBranch" ],
        "message_format": "[{$sources}] may reach a conditional branch"
    },
    {
        "name": "Possible test",
        "code": 5001,
        "sources": [ "CustomUserControlled" ],
        "sinks": [ "CodeExecution" ],
        "message_format": "test"
    }
],

"maximum_overrides_to_analyze": 1000
}

```

Listing 27: taint.config